

*Communauté Économique et Monétaire de
l'Afrique Centrale
(CEMAC)*



*Institut Sous-régional de Statistique
et d'Économie Appliquée*

Organisation Internationale

BP : 294 Yaoundé - Tél : 222 220 134

Cameroun AI Enthousiaste



*Direction du Développement et de la
Recherche*

ONG camerounaise

BP : 83000 Toulon, France

*Mémoire professionnel de fin d'étude présenté en vue de l'obtention du diplôme d'Ingénieur
Statisticien Économiste(ISE)*

OPTION : Data Science et Marketing

**CONCEPTION D'UN SYSTEME DE RECOMMANDATION HYBRIDE POUR LE
MATCHING EMPLOI-COMPETENCES AU CAMEROUN PAR INTEGRATION DES
LLMS ET DES GRAPHS DE CONNAISSANCES**

Rédigé par :

NGOULOU NGOUBILI Irch Defluviaire

Élève Ingénieur Statisticien Économiste cycle long – 5^e année

Sous l'encadrement de :

Dr. FOUTSE Yuehgoh

Présidente & Directrice exécutive adjointe, PhD Computer Science & Data Scientist

Année académique 2025-2026

Dédicace

À mes parents... (Max 1 page)

Remerciements

Sommaire

Dédicace	i
Remerciements	ii
Liste des sigles et abréviations	v
Liste des tableaux	vi
Liste des figures	viii
Avant-propos	ix
Résumé	x
Abstract	xi
Introduction générale	1
I Cadre théorique et conceptuel	6
1 Fondements théoriques et revue de la littérature sur l'IA générative et les systèmes de recommandation	7
1.1 Vocabulaire opérationnel : du matching à l'Agentic RAG	7
1.2 Fondements économiques et problématique d'appariement	9
1.3 Systèmes de recommandation et person-job fit	10
1.4 LLMs, embeddings et graphes de connaissances	12
1.5 RAG, GraphRAG et orchestration agentique	14
2 Source de données et Approche méthodologique	19
2.1 Cadre méthodologique et sources de données	19
2.2 Modélisation sémantique et structuration des connaissances	25
2.3 Orchestration agentique et génération contrôlée	28
2.4 Évaluation prévue, limites et synthèse méthodologique	30
II Présentation des résultats	32
3 Construction opérationnelle du système hybride emploi-compétences	33
3.1 Données exploitables et contraintes de qualité	33

3.2	Adaptation du modèle d’embeddings au domaine emploi–compétences	38
3.3	Mémoire vectorielle pgvector pour le retrieval	44
3.4	Neo4j : mémoire relationnelle du matching	50
3.5	Score hybride et reranking des recommandations	57
3.6	Orchestration agentique, critic et exposition du système	60
4	Évaluation de la performance du système	61
4.1	Cadre d’évaluation et protocole expérimental	61
4.2	Retrieval sémantique et apport mesuré du graphe	62
4.3	Lecture fonctionnelle du graphe Neo4j et du GraphRAG	65
4.4	Reranking hybride et calibration des poids	68
4.5	Contrôle de génération par critic et décision accept/revise	72
5	Discussion et recommandations	75
	Conclusion générale	I
	Bibliographie	I
	Bibliographie	III
A	Annexes	VII
A.1	Compléments techniques du chapitre 3	X

Liste des sigles et abréviations

ANN :	Approximate Nearest Neighbors
API :	Application Programming Interface
BERT :	Bidirectional Encoder Representations from Transformers
BI :	Business Intelligence
CEMAC :	Communauté Économique et Monétaire de l'Afrique Centrale
CV :	Curriculum Vitae
DCG :	Discounted Cumulative Gain
ESCO :	European Skills, Competences, Qualifications and Occupations
FAISS :	Facebook AI Similarity Search
FNE :	Fonds National de l'Emploi
FRED :	Federal Reserve Economic Data
GAT :	Graph Attention Network
GCN :	Graph Convolutional Network
GNN :	Graph Neural Network
GPT :	Generative Pre-trained Transformer
GPU :	Graphics Processing Unit
HNSW :	Hierarchical Navigable Small World
IA :	Intelligence Artificielle
IDCG :	Ideal Discounted Cumulative Gain
ISE :	Ingénieur Statisticien Économiste
ISSEA :	Institut Sous-régional de Statistique et d'Économie Appliquée
IVF :	Inverted File Index
KG :	Knowledge Graph
LLM :	Large Language Model
LoRA :	Low-Rank Adaptation
LSTM :	Long Short-Term Memory
MAP :	Mean Average Precision
MEPC :	Nomenclature Camerounaise des Métiers, Emplois et Professions
MRR :	Mean Reciprocal Rank
NCF :	Nomenclature Camerounaise des Formations
NCM :	Nomenclature Camerounaise des Métiers
NDCG :	Normalized Discounted Cumulative Gain
NLP :	Natural Language Processing
OIT :	Organisation Internationale du Travail
ONG :	Organisation Non Gouvernementale
QLoRA :	Quantized Low-Rank Adaptation
RAG :	Retrieval-Augmented Generation
RNN :	Recurrent Neural Network
RWKV :	Receptance Weighted Key Value
SEO :	Search Engine Optimization
SGPT :	Sentence Generative Pre-trained Transformer
SQL :	Structured Query Language
SR :	Système de Recommandation

Liste des tableaux

Tableau 2.1	Correspondance entre objectifs spécifiques et choix méthodologiques	22
Tableau 2.2	Sources de données mobilisées et utilisation méthodologique	23
Tableau 3.1	Volumes réellement exploités par les modules de recommandation . .	34
Tableau 3.2	Protocole de construction des paires de fine-tuning	39
Tableau 3.3	Hyperparamètres du fine-tuning SentenceTransformer	40
Tableau 3.4	Distribution des embeddings par type d'entité	45
Tableau 3.5	Structure opérationnelle des tables pgvector	45
Tableau 3.6	Résultats de calibration des poids du score hybride	58
Tableau 4.1	Benchmark des modèles d'embeddings sur 473 requêtes de test	63
Tableau 4.2	Comparaison retrieval : pgvector seul versus pgvector + graphe	64
Tableau 4.3	Pondérations retenues selon la fonction objectif du score hybride . . .	70
Tableau 4.4	Seuil critique retenu après calibration contrastive	73
Tableau A.1	Labels de nœuds et relations structurantes du graphe	VIII
Tableau A.2	Volume d'entités indexées dans la base vectorielle pgvector	IX
Tableau A.3	Pipeline général implémenté : étapes, entrées et sorties	IX
Tableau A.4	Synthèse des traitements de nettoyage utiles au moteur de recomman- dation	XI
Tableau A.5	Comparaison directe entre le baseline et le modèle fine-tuné	XI
Tableau A.6	Diagnostic statistique du chunking documentaire	XII
Tableau A.7	Couverture documentaire et duplication des chunks	XII
Tableau A.8	Cohérence sémantique intra-chunk	XII
Tableau A.9	Indicateurs techniques pgvector	XIII
Tableau A.10	Index pgvector et index complémentaires	XIV
Tableau A.11	Hétérogénéité des degrés par famille de noeuds	XVI
Tableau A.12	Exemples de compétences centrales selon trois métriques	XVII
Tableau A.13	Résumé du déplacement de rang après reranking Neo4j	XX

Table des figures

Figure 1.1	Évolution simplifiée des modèles de langage et des représentations textuelles	12
Figure 1.2	Différence fonctionnelle entre RAG, GraphRAG et Agentic RAG	15
Figure 2.1	Architecture méthodologique du workflow agentique de recommandation	21
Figure 3.1	Répartition des offres par source après nettoyage	35
Figure 3.2	Compétences fréquentes et disponibilité descriptive	36
Figure 3.3	Diagnostic du fine-tuning : perte, généralisation, métriques de validation et calendrier du learning rate	41
Figure 3.4	Comparaison des modèles d'embeddings : qualité de classement, rappel et latence	42
Figure 3.5	Projection UMAP des embeddings indexés	43
Figure 3.6	Distribution des tailles, sources et couverture du chunking documentaire	47
Figure 3.7	Distribution de la cohérence sémantique intra-chunk par source	48
Figure 3.8	Schéma physique de la base pgvector : tables documentaires, embeddings et index de recherche	49
Figure 3.9	Schéma logique du graphe de connaissances	51
Figure 3.10	Lecture conjointe des noeuds et relations Neo4j mobilisés par le matching	52
Figure 3.11	Distribution comparée des scores pgvector et Neo4j utilisés dans le reranking	58
Figure 3.12	Effet du reranking Neo4j sur les métriques et les déplacements de rang	59
Figure 4.1	Comparaison des modèles d'embeddings sur le jeu de test	63
Figure 4.2	Effet de l'enrichissement graphe sur les métriques de retrieval	65
Figure 4.3	Évaluation fonctionnelle des requêtes Neo4j utiles au GraphRAG	66
Figure 4.4	Distribution des déplacements de rang après reranking par le graphe	69
Figure 4.5	Évolution de l'objectif NDCG@10 au cours des essais Optuna	71
Figure 4.6	Comparaison des pondérations retenues selon la fonction objectif	71
Figure 4.7	Distribution des scores du critic avec contexte réel et contexte mélangé	73
Figure 4.8	Effet du seuil du critic sur les décisions accept/revise	74
Figure A.1	Localisation nettoyée des offres	X
Figure A.2	Tableau de bord statistique du graphe Neo4j	XV
Figure A.3	Composition du graphe par types de noeuds et relations	XV
Figure A.4	Distribution des degrés dans la projection matching	XVI
Figure A.5	Centralités des compétences : degré, PageRank et intermédiarité approximée	XVII

Figure A.6	Communautés Louvain, distances et composantes connexes	XVIII
Figure A.7	Robustesse de la projection matching après retrait des hubs	XVIII
Figure A.8	Compétences par offre, candidat et métier dans Neo4j	XIX
Figure A.9	Distribution échantillonnée du skill gap candidat-offre	XIX

Avant-propos

L'Institut Sous-régional de Statistique et d'Économie Appliquée (ISSEA), institution spécialisée de la CEMAC créée en 1984, a pour mission principale la formation de cadres supérieurs en statistique, économie et data science, capables de répondre aux enjeux contemporains de décision fondée sur les données. Dans ce cadre, l'ISSEA propose plusieurs filières de formation, parmi lesquelles figure celle des Ingénieurs Statisticiens Économistes (ISE), alliant rigueur mathématique, modélisation économétrique et outils modernes d'analyse de données (Data Science).

Arrivés à un niveau avancé de leur formation, les élèves Ingénieurs Statisticiens Économistes sont tenus d'effectuer un stage professionnel d'une durée minimale de trois (03) mois. Ce stage constitue une phase essentielle du cursus, permettant de confronter les connaissances théoriques acquises à des problématiques réelles, tout en favorisant l'acquisition de compétences opérationnelles en environnement professionnel. Il est sanctionné par la rédaction d'un mémoire professionnel, soutenu devant un jury.

C'est dans ce cadre que nous avons effectué un stage du 9 mars 2026 au 30 juin 2026 au sein de KmerAI, où nous avons été intégrés à la Direction du Développement et de la Recherche. Cette immersion nous a permis de travailler sur une problématique appliquée située au croisement de l'analyse de données, de l'intelligence artificielle générative, du traitement automatique du langage naturel et de l'aide à la décision : l'amélioration du matching entre offres d'emploi, profils de candidats et compétences recherchées sur le marché camerounais.

Le présent mémoire s'inscrit dans cette dynamique et porte sur le thème : « Conception d'un système de recommandation hybride pour le matching emploi-compétences au Cameroun par intégration des LLMs et des graphes de connaissances ». Il vise principalement à concevoir une architecture capable de structurer les données d'offres et de profils, d'exploiter les référentiels métiers et formations, puis de produire des recommandations plus explicables que les approches fondées uniquement sur des mots-clés. Plus spécifiquement, il s'agit de normaliser les données disponibles, d'aligner les métiers et compétences avec les référentiels ESCO, MEPC et NCF, d'adapter un modèle d'embeddings au domaine emploi-compétences, de construire un graphe de connaissances sous Neo4j et de poser les bases d'un moteur de recommandation hybride intégrant similarité sémantique, relations de graphe et analyse du *skill gap*.

Ce travail répond à un double enjeu. Sur le plan scientifique et méthodologique, il cherche à montrer comment les modèles de représentation sémantique, les bases vectorielles et les graphes de connaissances peuvent être combinés pour traiter un problème réel d'appariement sur le marché du travail. Sur le plan opérationnel, il propose une démarche reproductible susceptible d'aider KmerAI à développer des outils de recommandation, d'orientation et de montée en compétences adaptés aux besoins des candidats, des recruteurs et des acteurs de la formation.

Résumé

Mots-clés : Économétrie, Statistique, Développement...

Abstract

INTRODUCTION GÉNÉRALE

Contexte et justification

Le fonctionnement du marché de l'emploi constitue un déterminant central de la performance économique et sociale d'un pays. Au Cameroun, ce marché reste marqué par des déséquilibres structurels persistants, notamment l'inadéquation entre les compétences disponibles et les besoins des entreprises, la forte informalité et les difficultés d'insertion professionnelle des jeunes. Les données disponibles montrent que le chômage des jeunes demeure une réalité importante, avec un taux estimé à 6,6% en 2024 et 6,5% en 2025 selon les estimations modélisées de l'OIT relayées par la Banque Mondiale et la base FRED. Par ailleurs, plusieurs travaux sur le Cameroun soulignent que le problème ne tient pas uniquement au manque d'emplois, mais aussi à un mismatch entre formation, compétences et exigences du marché.

Dans ce contexte, la question du matching emploi-compétences devient centrale. En théorie, un marché du travail efficient devrait permettre d'associer rapidement les profils disposant des compétences requises aux postes vacants correspondants. En pratique, plusieurs frictions informationnelles persistent : asymétrie d'information entre recruteurs et candidats, faible structuration des profils de compétences, hétérogénéité des offres d'emploi et faiblesse des systèmes d'intermédiation comme le FNE et certaines agences de recrutement. Ces dysfonctionnements conduisent à une situation paradoxale où coexistent des postes non pourvus et des chercheurs d'emploi, traduisant une inefficacité allocative du marché du travail.

Par ailleurs, l'essor des technologies numériques et de l'intelligence artificielle ouvre des perspectives nouvelles pour améliorer l'appariement entre offres et profils. Dans cette dynamique, KmerAI se positionne comme une plateforme camerounaise dédiée à la promotion de l'intelligence artificielle, à la vulgarisation de ses usages et à l'accompagnement des étudiants, chercheurs et entreprises dans l'adoption de solutions innovantes adaptées au contexte local. Ses missions consistent notamment à former aux bases de l'IA, à offrir un espace de partage d'expériences et à encourager la collaboration autour de cas d'usage appliqués à des secteurs stratégiques tels que la santé, l'agriculture, le transport, l'éducation et le marketing.

Cette orientation est particulièrement pertinente pour le marché de l'emploi camerounais, où les données liées aux offres, aux CV et aux compétences restent souvent dispersées et peu exploitées. Une approche fondée sur l'intelligence artificielle peut permettre d'analyser plus

efficacement ces données hétérogènes afin de proposer des recommandations d'emploi plus pertinentes, plus rapides et plus personnalisées.

Problématique

Le marché de l'emploi camerounais est marqué par une forte asymétrie d'information, une hétérogénéité des profils professionnels et des offres d'emploi, ainsi qu'une faible capacité des outils classiques à évaluer les écarts réels de compétences. Les systèmes fondés sur des mots-clés ou des filtres statiques ne parviennent pas à détecter les compétences implicites, les équivalences sémantiques et les trajectoires de montée en compétences.

Par ailleurs, si des référentiels structurants existent notamment la Nomenclature Camerounaise des Métiers (NCM 2013), le référentiel européen ESCO et la Nomenclature Camerounaise des Formations (NCF), leur intégration opérationnelle dans les plateformes et outils d'appariement reste encore limitée. Ils sont peu exploités pour aligner les intitulés locaux de métiers sur des standards internationaux, harmoniser les compétences attendues, qualifier les niveaux et domaines de formation, et rendre les correspondances profil-offre plus transparentes et comparables. Il en résulte un décalage entre la richesse potentielle de ces référentiels et leur usage effectif dans les systèmes d'information dédiés à l'emploi et à l'orientation.

Dans ce contexte, la question centrale n'est plus seulement d'apparier un profil à une offre, mais de concevoir un système capable (i) d'exploiter conjointement les référentiels NCM, ESCO et NCF pour structurer les métiers, les compétences et les formations, (ii) de mesurer finement le *skill gap* entre un candidat et un poste cible, et (iii) de proposer un parcours de montée en compétences compatible avec le niveau, le domaine de formation et les trajectoires de qualification du candidat. La question principale de recherche peut ainsi être formulée comme suit :

Comment concevoir un système de recommandation hybride, fondé sur les LLMs, les graphes de connaissances, les embeddings vectoriels et le filtrage collaboratif, capable de mesurer le skill gap entre un profil et une offre d'emploi, puis de recommander un parcours de montée en compétences cohérent avec la nomenclature des formations, le niveau du candidat et le métier visé pour les membres de la communauté KmerAi ?

Cette question générale se décline en plusieurs questions de recherche sous-jacentes :

- **Q1** : Comment identifier et formaliser, dans le contexte camerounais, les compétences, métiers, formations et relations pertinentes pour un matching emploi compétences fiable et opérationnel à partir des référentiels NCM, ESCO et NCF ?

- Q2 : Dans quelle mesure l'intégration d'un graphe de connaissances améliore-t-elle la qualité et la précision du matching par rapport aux approches purement vectorielles ?

Objectifs de l'étude

L'objectif général de cette étude est de concevoir et d'évaluer un système de recommandation hybride emploi-compétences capable d'aider à la décision dans le recrutement et la montée en compétence. Le système vise à rapprocher les profils de candidats et les offres d'emploi en combinant la recherche sémantique, les référentiels métiers et formations, le graphe de connaissances et une logique explicable de *skill gap*. Il ne s'agit donc pas seulement de retrouver les offres textuellement proches d'un profil, mais de produire un verdict opérationnel : identifier les offres immédiatement accessibles, distinguer celles qui nécessitent une montée en compétence, signaler les profils à conserver en vivier et expliciter les compétences à développer.

Plus spécifiquement, il sera question de :

- normaliser et aligner les offres d'emploi et profils candidats avec les référentiels ESCO, MEPC et NCF afin de produire des variables structurées intitulé de poste, compétences, niveau NCF, secteur, localisation exploitables par le système de matching ;
- adapter par *fine-tuning* contrastif un modèle *SentenceTransformer* au domaine emploi-compétences à partir de paires offre-profil, puis indexer les représentations produites dans une base vectorielle pgvector pour le retrieval sémantique ;
- construire un graphe de connaissances sous Neo4j modélisant les relations entre candidats, offres, compétences, métiers, secteurs et niveaux NCF, et concevoir un score hybride combinant similarité sémantique, couverture de compétences, compatibilité NCF et alignement sectoriel ;
- orchestrer, via un workflow *LangGraph* multi-étapes, l'ensemble du pipeline de recommandation retrieval sémantique, enrichissement graphe, analyse de *skill gap*, scoring hybride, critique et replanification afin de produire des verdicts interprétables (profil prêt, montée en compétence, vivier, hors cible) accompagnés d'une trajectoire de formation, et d'évaluer la qualité du retrieval par les métriques Recall@K, NDCG@K, MRR@K et MAP@K.

Intérêt de l'étude

L'intérêt d'une telle approche est renforcé par les limites des méthodes classiques de recrutement, souvent fondées sur des critères statiques ou sur des mots-clés, qui ne captent pas toujours la richesse des parcours et des compétences. En combinant différentes logiques de re-

commandation, un système hybride peut mieux prendre en compte les dimensions explicites et implicites des profils candidats, tout en améliorant la qualité des correspondances proposées. Cela rejoint la mission de KmerAI, qui vise à favoriser l'appropriation locale de l'IA pour résoudre des problèmes concrets du développement camerounais.

Hypothèses de recherche

Afin de répondre à la problématique, cette étude repose sur les hypothèses suivantes.

- **H1** : L'intégration conjointe de la NCM, d'ESCO et de la NCF dans un référentiel unifié métiers-compétences-formations améliore la structuration et les performances d'appariement profil-offre, par rapport à une approche sans ces référentiels ;
- **H2** : La modélisation du marché de l'emploi sous forme de graphe de connaissances (Neo4j), incluant explicitement les niveaux et domaines de formation de la NCF, améliore la qualité du matching par rapport à une approche purement vectorielle ;
- **H3** : Le *fine-tuning* d'un modèle de représentation sémantique sur le domaine emploi-compétences améliore la qualité des embeddings et la similarité sémantique entre profils et offres, par rapport au même modèle non adapté ;
- **H4** : La combinaison d'un RAG vectoriel, d'un GraphRAG et d'un score de recommandation hybride améliore la pertinence et l'explicabilité des recommandations de *skill gap* et de parcours de montée en compétences, notamment lorsque ces parcours sont contraints par la nomenclature des formations.

Méthodologie de recherche

La démarche adoptée s'inscrit dans un cadre d'ingénierie des données combinant structuration des connaissances et modélisation hybride. Elle repose sur la collecte et la normalisation des données issues d'offres d'emploi sur les différentes sites web et de profils, alignées sur la NCM 2013 afin d'assurer la cohérence sémantique. Un graphe de connaissances est ensuite construit sous Neo4j en intégrant les référentiels ESCO et NCM, à partir d'un processus d'extraction automatique de triplets à partir des données textuelles. Parallèlement, les descriptions sont vectorisées et indexées dans un espace sémantique permettant la mesure de similarité. Le moteur de recommandation combine ces différentes sources d'information à travers un score hybride intégrant similarité structurelle, similarité sémantique et signal collaboratif. Le système permet ainsi d'identifier les écarts de compétences entre profils et offres, et de générer des recommandations sous forme de trajectoires d'apprentissage. L'évaluation repose sur des métriques standard telles que la précision@K, le rappel et le NDCG, ainsi que sur l'analyse de

la cohérence des recommandations générées.

Plan du travail

Le mémoire est structuré en trois parties. La première présente le cadre théorique relatif aux systèmes de recommandation, aux modèles de langage et aux graphes de connaissances. La deuxième détaille l'approche méthodologique, incluant la modélisation du graphe et la conception du système hybride. La troisième analyse les résultats obtenus et discute les performances du modèle ainsi que ses implications dans le contexte du marché du travail camerounais.

Première partie

Cadre théorique et conceptuel

Fondements théoriques et revue de la littérature sur l'IA générative et les systèmes de recommandation

Ce chapitre situe le mémoire dans la littérature sur l'appariement emploi-compétences, les systèmes de recommandation, l'intelligence artificielle générative, les graphes de connaissances et la génération augmentée par récupération. Il ne vise pas à reprendre l'ensemble des raisonnements des ouvrages ou articles consultés. Il sélectionne les résultats qui éclairent directement le thème du stage : concevoir un système capable de recommander des offres, d'identifier des écarts de compétences et de produire des explications fondées sur des données structurées et non structurées.

La revue est guidée par une hypothèse centrale : le matching emploi-compétences n'est pas un simple problème de similarité textuelle. Il combine des frictions du marché du travail, une forte variabilité du vocabulaire, des contraintes de qualification, des relations métier-compétence-formation et un besoin d'explicabilité. Cette nature composite explique pourquoi la littérature pertinente relève à la fois de l'économie du travail, de la recommandation hybride, de la recherche sémantique, des graphes de connaissances et de la génération contrôlée.

1.1 Vocabulaire opérationnel : du matching à l'Agentic RAG

Avant d'entrer dans la revue de littérature, il est nécessaire de stabiliser les concepts utilisés dans la suite du mémoire. Cette clarification évite une confusion fréquente : employer indifféremment IA générative, LLM, RAG, GraphRAG, moteur de recommandation et chatbot. Dans ce travail, ces termes désignent des fonctions complémentaires, mais non interchangeables.

Matching emploi-compétences.

Le matching emploi-compétences désigne le processus qui consiste à rapprocher un profil candidat d'une offre, d'un métier ou d'une trajectoire de formation à partir de caractéristiques observables : compétences, niveau de formation, expérience, secteur, localisation et objectif

professionnel. Il est traité ici comme un problème d'appariement économique, mais aussi comme un problème de représentation des connaissances.

Système de recommandation.

Un système de recommandation est un dispositif algorithmique qui classe des éléments selon leur pertinence probable pour un utilisateur ou un cas d'usage [1]. Dans ce mémoire, les éléments recommandés peuvent être des offres d'emploi, des compétences, des métiers ou des pistes de formation. La pertinence ne renvoie donc pas seulement à une préférence ; elle renvoie aussi à une compatibilité professionnelle explicable.

Skill gap.

Le *skill gap*, ou écart de compétences, correspond à la différence entre les compétences déjà détenues par un candidat et celles attendues pour une offre ou un métier cible. Il permet de dépasser une logique binaire compatible/non compatible : un candidat peut être proche d'un poste tout en nécessitant une montée en compétences.

Embedding et recherche sémantique.

Un embedding est une représentation vectorielle d'un texte, d'un profil, d'une offre ou d'une compétence. La recherche sémantique utilise cette représentation pour retrouver des objets proches en sens, même lorsque les formulations diffèrent. Elle répond à un problème récurrent des données d'emploi : des expressions comme « data analyst », « analyste de données » et « reporting statistique » peuvent désigner des zones de compétences voisines.

Base vectorielle pgvector.

pgvector désigne l'extension PostgreSQL utilisée pour stocker et rechercher des embeddings [2]. Dans ce mémoire, elle sert de couche de recherche rapide pour retrouver les offres, métiers, compétences ou documents proches d'une requête. Son rôle est de présélectionner des candidats à la recommandation. Elle ne suffit cependant pas à décider seule, car un bon score vectoriel ne garantit pas que le niveau NCF, l'expérience ou les compétences obligatoires soient compatibles.

Graphe de connaissances Neo4j.

Un graphe de connaissances représente des entités et leurs relations : candidat, offre, compétence, métier, secteur, niveau, formation et référentiel [3]. Dans ce mémoire, Neo4j sert à représenter les relations explicites entre ces entités. Il permet de répondre à des questions que la seule similarité vectorielle ne traite pas correctement : quelles compétences une offre exige-t-elle ? quelles compétences le candidat possède-t-il ? quel niveau de formation est demandé ?

quels chemins relient un métier à une formation ?

IA générative, LLM et prompting.

L'intelligence artificielle générative désigne les modèles capables de produire du contenu, notamment du texte. Un LLM est un grand modèle de langage entraîné sur de grands corpus textuels et capable de générer, reformuler ou synthétiser des réponses. Dans ce mémoire, le LLM n'est pas le moteur de vérité du système. Il intervient comme générateur contrôlé : il reçoit un contexte construit à partir de pgvector, Neo4j et des documents, puis rédige une réponse en français. Le prompting désigne l'ensemble des consignes données au modèle pour encadrer son comportement, limiter les inventions et structurer la réponse.

RAG, GraphRAG et Agentic RAG.

La RAG combine récupération d'information et génération : le système recherche d'abord des éléments pertinents, puis les transmet au LLM pour produire une réponse [4]. Le GraphRAG étend cette logique en ajoutant des faits et chemins issus d'un graphe de connaissances. L'Agentic RAG ajoute une couche d'orchestration : l'agent analyse l'intention, choisit les outils, récupère les données, construit le contexte, génère la réponse et peut demander une révision si le contexte est insuffisant. Dans ce mémoire, cette orchestration est assurée par LangGraph.

Critic et traçabilité.

Le critic est un mécanisme de contrôle qui examine si la réponse générée semble suffisamment couverte par le contexte récupéré. Il ne remplace pas une validation humaine, mais il réduit le risque d'une réponse fluide et insuffisamment fondée. La traçabilité désigne la capacité à afficher les outils mobilisés, les sources consultées et le chemin suivi par le workflow. Elle est indispensable dans un système d'aide à la décision, car une recommandation professionnelle doit être vérifiable.

1.2 Fondements économiques et problématique d'appariement

1.2.1 Marché du travail, recherche d'emploi et frictions

Le problème traité dans ce mémoire relève d'abord de l'économie du travail. Les modèles de recherche et d'appariement montrent que le chômage peut coexister avec des postes vacants lorsque la rencontre entre travailleurs et entreprises est coûteuse, lente ou imparfaitement informée [5, 6]. Cette lecture est importante : un système de recommandation emploi-compétences n'est pas seulement un outil informatique ; c'est un dispositif d'intermédiation

qui cherche à réduire les coûts de recherche et à améliorer la qualité des mises en relation.

La littérature sur l'asymétrie d'information renforce ce diagnostic. Akerlof montre que l'incertitude sur la qualité peut dégrader les échanges lorsque les acteurs ne disposent pas de signaux fiables [7]. Sur le marché de l'emploi, cette asymétrie apparaît des deux côtés : le recruteur ne voit pas toujours les compétences réelles du candidat, et le candidat ne connaît pas toujours les exigences effectives du poste. La théorie du signalement de Spence explique alors le rôle des diplômes, certifications et expériences comme signaux observables [8]. Mais ces signaux restent incomplets lorsque les métiers évoluent rapidement, notamment dans les domaines numériques où les compétences techniques sont souvent décrites par des formulations instables.

1.2.2 Capital humain, tâches et inadéquation des compétences

La théorie du capital humain rappelle que la formation et l'expérience sont des investissements productifs [9]. Toutefois, la possession d'un capital humain ne garantit pas l'appariement. Encore faut-il que les compétences acquises soient visibles, comparables et alignées sur les compétences demandées. Autor montre que le progrès technique transforme surtout les tâches composant les métiers, et non uniquement les intitulés d'emploi [10]. Pour un système de recommandation, cela impose de descendre au niveau des compétences et des tâches plutôt que de s'arrêter aux titres de poste.

L'apport de cette littérature est de montrer que l'inadéquation emploi-compétences relève autant de la circulation imparfaite de l'information que d'un manque de formation. Sa limite, pour notre objet, est qu'elle ne fournit pas directement les outils de traitement des données nécessaires pour extraire, normaliser, comparer et expliquer les compétences. La section suivante examine donc les systèmes de recommandation comme réponse algorithmique à ce problème d'appariement.

1.3 Systèmes de recommandation et person-job fit

1.3.1 Approches par contenu, collaboratives et hybrides

Les systèmes de recommandation cherchent à classer des items selon leur pertinence probable pour un utilisateur. La littérature distingue classiquement les approches fondées sur le contenu, le filtrage collaboratif et les approches hybrides [11, 1]. Dans le contexte de ce mémoire, l'utilisateur peut être un candidat, un conseiller emploi ou une plateforme ; les items peuvent être des offres, compétences, métiers ou formations.

Les approches fondées sur le contenu utilisent les caractéristiques explicites des items et des profils. Elles sont adaptées au matching emploi-compétences parce que les offres et les candidats possèdent des descriptions textuelles, des secteurs, des niveaux et des compétences. Pazzani et Billsus montrent que ces systèmes exploitent les attributs des contenus pour produire des recommandations personnalisées [12]. Leur avantage est l'interprétabilité ; leur limite est la dépendance au vocabulaire observé.

Le filtrage collaboratif exploite les interactions passées entre utilisateurs et items. Les approches de factorisation matricielle ont fortement structuré ce domaine en apprenant des facteurs latents à partir des matrices d'interactions [13]. Les modèles neuronaux ont ensuite généralisé cette logique, par exemple avec le Neural Collaborative Filtering [14]. Ces méthodes sont puissantes lorsque l'on dispose d'un historique dense : clics, candidatures, recrutements, évaluations ou parcours de formation. Dans le cas étudié ici, cette hypothèse est fragile, car les données d'interaction restent limitées et de nombreux profils se trouvent en situation de démarrage à froid.

Les approches hybrides combinent plusieurs sources de signal pour compenser les limites propres à chaque famille. Burke montre que l'hybridation peut améliorer la robustesse en associant contenu, collaboration, connaissances et règles métier [15]. Cette conclusion prépare directement le choix d'un système hybride : les signaux textuels, relationnels et métier doivent être séparés puis combinés, car chacun ne capture qu'une partie de la compatibilité.

1.3.2 Spécificité du person-job fit

La recommandation d'emploi se distingue de la recommandation de films ou de produits. Une offre n'est pas seulement un item susceptible de plaire ; elle impose des contraintes de qualification, d'expérience, de localisation, de disponibilité et de compétences. La littérature récente sur le *person-job fit* insiste sur cette double sélection : le candidat doit être intéressé par l'offre, mais l'employeur doit aussi pouvoir considérer le profil comme compatible [16]. La recommandation devient donc un problème bilatéral.

Cette spécificité change l'interprétation des scores. Dans le commerce électronique, une recommandation peut viser la probabilité de clic ou d'achat. Dans l'emploi, une recommandation doit distinguer au moins trois situations : le candidat est immédiatement compatible ; le candidat est proche mais nécessite une montée en compétences ; le candidat est trop éloigné du poste cible. La notion de *skill gap* devient alors centrale.

Les travaux récents mobilisant les graphes et les LLM dans les ressources humaines vont dans ce sens. HRGraph propose par exemple d'exploiter des graphes de connaissances enrichis

par LLM pour la recommandation d'emploi [17]. D'autres travaux s'intéressent à la prédiction des compétences et à la construction de graphes de talents pour les ressources humaines [18]. Ces contributions confirment l'intérêt des graphes pour représenter les relations entre compétences, métiers et profils, mais elles ne résolvent pas automatiquement la question de l'ancrage local : nomenclatures camerounaises, données réelles de plateforme, contraintes de formation et explication opérationnelle.

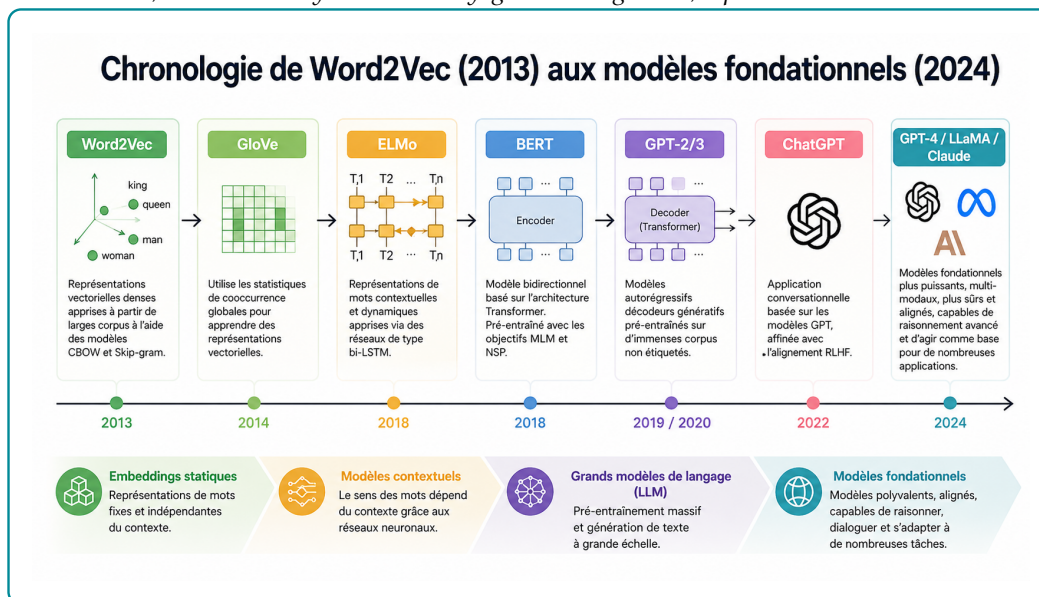
1.4 LLMs, embeddings et graphes de connaissances

1.4.1 Grands modèles de langage et génération contrôlée

Les grands modèles de langage ont transformé le traitement automatique du langage naturel en permettant des tâches de génération, synthèse, reformulation et raisonnement assisté. Leur rupture technique vient principalement de l'architecture Transformer proposée par Vaswani et al., fondée sur le mécanisme d'attention [19]. Avant cette rupture, les représentations textuelles étaient passées des sacs de mots aux embeddings denses comme Word2Vec, qui apprennent des vecteurs capturant des régularités sémantiques et syntaxiques [20]. Les mécanismes d'attention introduits en traduction neuronale ont ensuite permis aux modèles de pondérer dynamiquement les parties pertinentes d'une séquence [21].

Figure 1.1 – Évolution simplifiée des modèles de langage et des représentations textuelles

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.



Deux grandes familles de modèles sont utiles pour ce mémoire. La première regroupe les modèles de représentation, comme BERT et Sentence-BERT. BERT apprend des repré-

tions contextuelles bidirectionnelles adaptées à la compréhension du texte [22]. Sentence-BERT adapte cette logique pour produire des embeddings de phrases comparables efficacement par similarité cosinus [23]. La seconde famille regroupe les modèles génératifs, comme GPT, T5 ou les modèles instruction-tuned. GPT-3 a montré qu'un modèle de grande taille pouvait effectuer des tâches variées à partir de quelques exemples dans le prompt [24]. T5 a proposé une formulation unifiée des tâches NLP comme transformation texte-vers-texte [25]. L'apprentissage par retour humain a ensuite amélioré la capacité des modèles à suivre des instructions [26].

L'acquis de cette littérature est considérable : les LLM savent reformuler, synthétiser et produire des explications lisibles. Sa limite, pour notre problématique, tient au statut de la vérité : un modèle génératif peut produire une réponse plausible sans être fondée sur les données locales. Dans un système d'emploi, cette limite impose une génération contrôlée par des sources récupérées.

1.4.2 Embeddings, recherche sémantique et bases vectorielles

Après la clarification conceptuelle, l'enjeu est de savoir dans quelles conditions les représentations vectorielles sont fiables pour la recherche d'information. Les benchmarks montrent que les modèles d'embeddings doivent être évalués sur des tâches variées, car un bon modèle généraliste peut être insuffisant dans un domaine spécialisé [27, 28]. Cette observation justifie l'adaptation d'un modèle SentenceTransformer au vocabulaire emploi-compétences.

La recherche sémantique résout une partie du problème de vocabulaire, mais elle reste une approximation. Deux textes proches peuvent cacher une incompatibilité : niveau insuffisant, compétence obligatoire absente, localisation incompatible ou métier trop éloigné. La similarité vectorielle doit donc être traitée comme un signal de présélection, non comme une décision finale.

La littérature sur la recherche approximative de plus proches voisins montre par ailleurs que le passage à l'échelle exige des structures d'indexation efficaces. HNSW fournit un cadre robuste pour la recherche vectorielle approximative [29], tandis que FAISS a popularisé la recherche vectorielle à grande échelle [30]. Dans ce mémoire, l'utilisation de PostgreSQL avec pgvector répond à une logique pragmatique : conserver les embeddings dans une base relationnelle exploitable, interrogeable et intégrable au reste du pipeline [2].

1.4.3 Graphes de connaissances et recommandation relationnelle

Hogan et al. montrent que les graphes de connaissances servent à structurer des informations hétérogènes, à rendre explicites des relations sémantiques et à soutenir des tâches de recherche ou de raisonnement [3]. Dans le domaine emploi-compétences, cette représentation complète la recherche vectorielle : elle rend visibles les relations entre candidats, offres, compétences, métiers, formations et niveaux.

La recommandation par graphe a également connu des avancées importantes. Les graph neural networks ont fourni des outils pour propager de l'information dans des graphes [31, 32]. LightGCN a montré que, dans la recommandation collaborative, une architecture simplifiée de propagation sur graphe pouvait être très efficace [33]. Ces travaux établissent l'intérêt de la structure relationnelle pour la recommandation.

Cependant, le présent mémoire ne cherche pas principalement à entraîner un GNN. Le choix méthodologique est différent : utiliser un graphe de connaissances comme support d'explication, de contraintes et de calcul du skill gap. Ce choix est plus adapté au contexte disponible. Les données d'interaction ne sont pas assez riches pour justifier un modèle collaboratif profond, tandis que les relations métier-compétence-formation sont directement interprétables.

1.5 RAG, GraphRAG et orchestration agentique

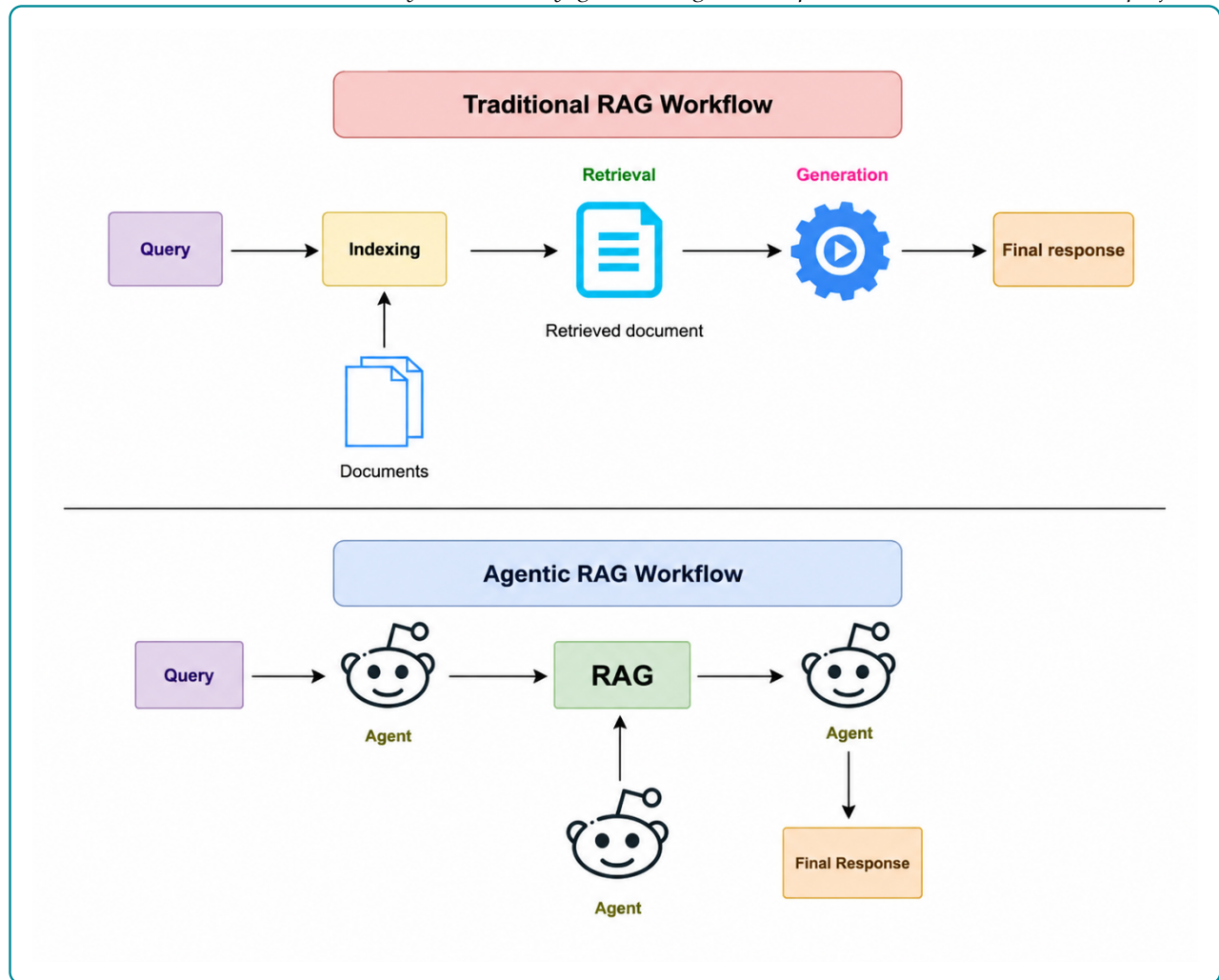
1.5.1 De la RAG au GraphRAG

La génération augmentée par récupération, ou RAG, combine une étape de recherche d'information et une étape de génération. Lewis et al. montrent que l'accès à une mémoire externe améliore les tâches intensives en connaissances, car le modèle ne dépend plus uniquement de ses paramètres internes [4]. Les synthèses récentes sur la RAG insistent sur les mêmes enjeux : qualité de la récupération, actualisation des connaissances, évaluation de la factualité et articulation entre retriever et générateur [34].

Dans une RAG classique, le système récupère des documents proches d'une requête, puis les transmet au LLM. Cette architecture est utile, mais elle reste centrée sur des passages textuels. Pour le matching emploi-compétences, le contexte doit aussi intégrer des relations : compétences possédées, compétences requises, niveau de formation, métier cible, secteur et formation possible. Le GraphRAG étend alors la récupération vers des informations structurées par graphe.

Figure 1.2 – Différence fonctionnelle entre RAG, GraphRAG et Agentic RAG

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.



La figure 1.2 illustre cette évolution. Une RAG simple suit une chaîne relativement fixe : requête, récupération, génération. Une architecture GraphRAG ajoute une couche relationnelle. Une architecture agentic introduit une capacité de planification et de contrôle : le système peut choisir un outil, observer le résultat, élargir le contexte et réviser la réponse.

1.5.2 Agentic RAG, critic et traçabilité

Les travaux ReAct montrent l'intérêt d'alterner raisonnement et action outillée dans les modèles de langage [35]. Self-RAG va plus loin en introduisant une logique d'auto-réflexion pour décider quand récupérer de l'information et comment critiquer la réponse [36]. Pour ce mémoire, ces travaux ne doivent pas être interprétés comme une autorisation à laisser un agent décider librement. Ils justifient plutôt une orchestration contrôlée : classifier l'intention, choisir les sources, récupérer les preuves, construire le contexte, générer, puis vérifier si la réponse est

suffisamment fondée.

Le manque de la littérature est ici opérationnel. Beaucoup de travaux décrivent des architectures générales, mais la fiabilité dépend de détails concrets : qualité des chunks, schéma du graphe, robustesse des requêtes, traçabilité des sources, gestion des cas sans résultat et critères de révision. C'est pourquoi l'architecture retenue ajoute un critic et des traces d'exécution. Le LLM produit la réponse finale, mais les données récupérées et les outils appelés doivent rester observables.

1.5.3 Évaluer la performance et la factualité

L'évaluation d'un système emploi-compétences ne peut pas se limiter à la fluidité de la réponse générée. Elle doit d'abord mesurer la qualité de la récupération d'information : le système retrouve-t-il les offres, compétences ou documents réellement pertinents ? La littérature en recherche d'information utilise pour cela des métriques de classement comme Precision@k, Recall@k, MRR et NDCG [27, 28]. Ces métriques supposent l'existence d'un ensemble de résultats pertinents, noté $Rel(q)$, pour une requête q , et d'une liste classée de résultats retournés par le système.

La **Precision@k** mesure la proportion de résultats pertinents parmi les k premiers résultats retournés. Si $R_k(q)$ désigne l'ensemble des k premiers éléments proposés pour la requête q , alors :

$$Precision@k(q) = \frac{|R_k(q) \cap Rel(q)|}{k}. \quad (1.1)$$

Cette métrique répond à la question suivante : parmi les premières recommandations affichées, combien sont réellement utiles ? Dans un système emploi-compétences, une Precision@5 élevée signifie que les cinq premières offres proposées contiennent peu de résultats non pertinents. Sa limite est qu'elle ne mesure pas si le système a retrouvé tous les bons résultats disponibles.

Le **Recall@k** mesure au contraire la proportion des éléments pertinents retrouvés dans les k premiers résultats :

$$Recall@k(q) = \frac{|R_k(q) \cap Rel(q)|}{|Rel(q)|}. \quad (1.2)$$

Cette métrique répond à la question : parmi toutes les offres ou compétences pertinentes, quelle part le système a-t-il réussi à faire remonter ? Elle est importante pour ne pas éliminer trop tôt des offres utiles. En recommandation emploi, un faible Recall@k peut signifier que le

moteur rate des opportunités pertinentes pour le candidat.

Le **MRR** (*Mean Reciprocal Rank*) évalue la position du premier résultat pertinent. Pour une requête q , si $rank_q$ est le rang du premier résultat pertinent, alors le score associé est :

$$RR(q) = \frac{1}{rank_q}. \quad (1.3)$$

Sur un ensemble de N requêtes, le MRR est :

$$MRR = \frac{1}{N} \sum_{q=1}^N \frac{1}{rank_q}. \quad (1.4)$$

Cette métrique est utile lorsque l'on veut savoir si le système place rapidement au moins une bonne réponse. Dans une interface de recommandation, cela correspond à une logique pratique : l'utilisateur regarde souvent les premiers résultats avant de parcourir toute la liste.

Le **NDCG@k** (*Normalized Discounted Cumulative Gain*) tient compte non seulement de la présence des résultats pertinents, mais aussi de leur ordre et éventuellement de leur degré de pertinence. Si rel_i désigne le niveau de pertinence du résultat placé au rang i , alors :

$$DCG@k = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i + 1)}. \quad (1.5)$$

Le score est ensuite normalisé par le meilleur classement possible, noté $IDCG@k$:

$$NDCG@k = \frac{DCG@k}{IDCG@k}. \quad (1.6)$$

Cette métrique est particulièrement adaptée lorsque tous les résultats pertinents ne se valent pas. Par exemple, une offre parfaitement alignée avec les compétences du candidat devrait être mieux classée qu'une offre seulement partiellement compatible.

Dans une architecture RAG, l'évaluation ne s'arrête pas au classement des résultats. Elle doit aussi mesurer la qualité de la réponse générée à partir du contexte récupéré. RAGAS distingue notamment la fidélité au contexte, la pertinence de la réponse, la qualité de la récupération et l'usage des sources [37]. La **fidélité** vérifie si les affirmations de la réponse sont supportées par les éléments récupérés. La **pertinence de réponse** vérifie si la réponse traite effectivement la question posée. La **couverture des sources** vérifie si le contexte contient suffisamment d'informations pour répondre. Enfin, la **traçabilité** vérifie si les preuves ou citations sont identifiables.

Ces critères sont moins simples à formaliser que Precision@k ou Recall@k, car ils portent

sur du langage naturel. Des approches comme G-Eval utilisent des LLM comme juges pour approximer l'évaluation humaine [38]. Cette pratique doit rester prudente : un juge automatique peut aider à comparer des variantes de système, mais il ne remplace pas une validation métier. Dans ce mémoire, l'évaluation doit donc combiner des métriques de retrieval pour pgvector et Neo4j, des critères de factualité pour la réponse générée, et une interprétation professionnelle de l'utilité des recommandations.

Source de données et Approche méthodologique

2.1 Cadre méthodologique et sources de données

2.1.1 Démarche d'ingénierie appliquée retenue

La présente étude adopte une démarche d'ingénierie appliquée orientée vers la conception, l'intégration et l'évaluation d'un artefact numérique d'aide à la décision. L'objet étudié n'est donc pas uniquement un modèle statistique pris isolément, mais un système complet de recommandation emploi-compétences combinant données structurées, représentations vectorielles, graphe de connaissances, règles métier et RAG. Ce positionnement est cohérent avec la problématique du mémoire : améliorer l'appariement entre candidats, offres d'emploi, compétences, métiers et trajectoires de formation dans un contexte où l'information est hétérogène, incomplète et dispersée.

La logique méthodologique retenue repose sur quatre principes. Le premier est la *traçabilité* : chaque recommandation doit pouvoir être rattachée à des données observables, à un référentiel ou à une relation explicite du graphe. Le deuxième est l'*hybridation* : aucune source d'information ne suffit seule à résoudre le problème. La similarité sémantique capte les proximités de langage, le graphe encode les relations métier, les règles contrôlent les contraintes de niveau et le LLM sert à formuler une réponse intelligible. Le troisième principe est l'*explicabilité opérationnelle* : le système ne doit pas seulement retourner une liste d'offres, mais justifier le rapprochement, signaler les compétences communes, identifier les écarts et proposer une trajectoire de montée en compétences. Le quatrième principe est le *contrôle génératif* : les réponses produites par le LLM doivent être contraintes par le contexte récupéré afin de réduire le risque d'hallucination, conformément à la logique du RAG [4, 37].

Sur le plan de l'orchestration, cette étude retient explicitement *LangGraph* comme cadre méthodologique de pilotage du système agentique [39]. Ce choix signifie que le raisonnement applicatif n'est pas laissé à une boucle conversationnelle informelle. *LangGraph* est donc uti-

lisé pour organiser le routage entre les briques du système, conserver les traces des outils mobilisés, permettre une révision lorsque le contexte est insuffisant et rendre le workflow auditable.

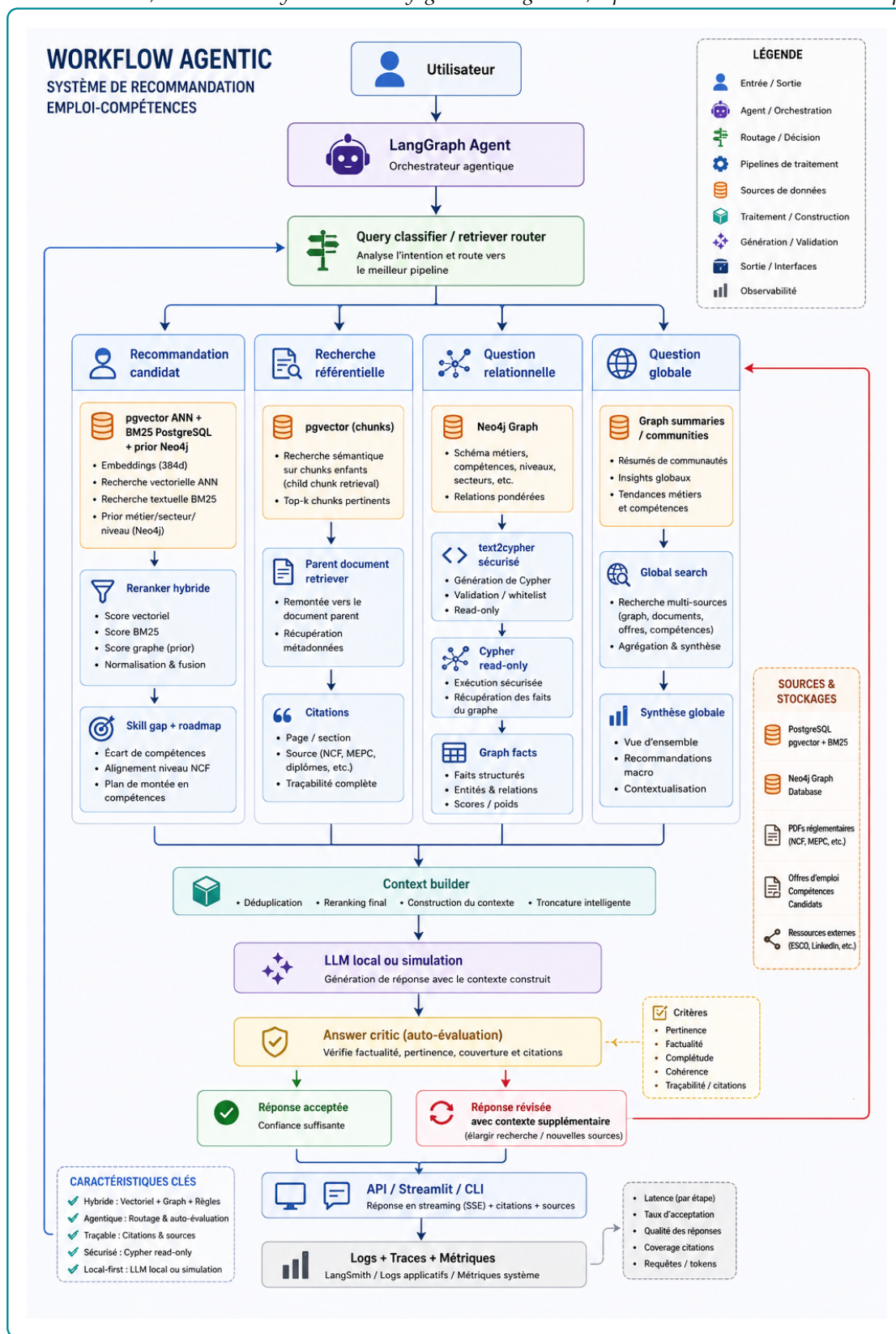
La figure 2.1 présente l'architecture générale retenue pour la solution. Elle doit être lue comme une carte méthodologique du système plutôt que comme un simple schéma technique. Le point d'entrée est la requête de l'utilisateur. Cette requête n'est pas transmise directement au modèle génératif; elle est d'abord interprétée par une couche d'orchestration qui détermine le type de besoin exprimé. Cette étape est centrale, car une demande de recommandation candidat-offre, une question sur un référentiel, une interrogation relationnelle sur les compétences ou une question globale ne mobilisent pas les mêmes sources ni les mêmes traitements.

Le workflow distingue ainsi plusieurs familles de traitements. La recommandation candidat s'appuie sur une combinaison de recherche vectorielle, de recherche textuelle et de signaux issus du graphe afin de produire un classement explicable. La recherche référentielle privilégie les fragments documentaires et les métadonnées associées, car l'enjeu est alors de retrouver une information sourcée. Les questions relationnelles exploitent davantage Neo4j, où les liens entre métiers, compétences, niveaux, secteurs et formations permettent de répondre à des questions que la similarité textuelle seule ne suffit pas à résoudre. Les questions globales mobilisent une synthèse plus large, fondée sur des résumés, des communautés ou des agrégations issues de plusieurs sources.

L'intérêt méthodologique de cette architecture est donc l'hybridation contrôlée. PostgreSQL avec pgvector joue le rôle de mémoire sémantique pour retrouver des objets proches dans l'espace des embeddings. Neo4j joue le rôle de mémoire relationnelle pour vérifier, expliquer et pondérer les liens métier-compétence-formation. Les documents référentiels apportent une base de citation et de traçabilité. Le constructeur de contexte rassemble ensuite ces résultats dans un format exploitable par le LLM, en évitant de lui confier seul la décision de pertinence. La génération intervient donc en aval, après récupération et structuration du contexte.

Enfin, la partie inférieure du schéma rappelle que le système ne s'arrête pas à la production d'une réponse. Un module de critique évalue si la réponse est suffisamment pertinente, factuelle, complète et traçable. Si le contexte est insuffisant, la boucle peut relancer une recherche élargie avant de produire une réponse révisée. Ce choix méthodologique est important pour le mémoire : l'architecture ne cherche pas seulement à générer un texte convaincant, mais à produire une recommandation fondée sur des sources identifiables, des relations vérifiables et des

Figure 2.1 – Architecture méthodologique du workflow agentique de recommandation
 Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.



critères métier explicites. Les sections suivantes détaillent chacune de ces briques : préparation des données, construction des embeddings, stockage vectoriel, graphe de connaissances, moteur hybride, orchestration agentique et protocole d'évaluation.

Table 2.1 – Correspondance entre objectifs spécifiques et choix méthodologiques

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Objectif spécifique	Choix méthodologique retenu
Normaliser les offres, candidats et référentiels	Construction d'une couche de données standardisée : identifiants stables, champs nettoyés, niveaux de formation, secteurs, compétences et textes d'embedding.
Adapter la recherche sémantique au domaine emploi-compétences	Fine-tuning contrastif d'un modèle SentenceTransformer, puis indexation des embeddings dans PostgreSQL avec pgvector pour le retrieval top-K [23, 2].
Représenter les relations entre métiers, compétences et profils	Construction d'un graphe de connaissances Neo4j reliant candidats, offres, compétences, métiers, secteurs, localisations et référentiels [3, 40].
Produire une recommandation explicable	Score hybride combinant similarité vectorielle, couverture de compétences, compatibilité de niveau et alignement sectoriel, complété par une analyse de <i>skill gap</i> .
Orchestrer la réponse utilisateur	Workflow LangGraph routant la requête vers les outils adaptés : pgvector, Neo4j, recherche documentaire, Text2Cypher, critic et génération finale.
Evaluer la qualité du système	Protocole séparant l'évaluation du retrieval, du graphe, du classement et de la réponse générée.

2.1.2 Sources de données et leur rôle dans le système

Le système mobilise deux catégories de données. La première correspond aux données opérationnelles du marché du travail : offres d'emploi et profils de candidats. Ces données décrivent respectivement la demande de travail et l'offre de travail disponible. La seconde catégorie correspond aux référentiels de structuration : ESCO, MEPC et NCF. Ces référentiels permettent d'éviter une approche purement textuelle et de rattacher les intitulés, compétences et niveaux de formation à des nomenclatures interprétables.

Les offres d'emploi apportent les informations relatives aux postes disponibles : intitulé, employeur, secteur, localisation, type de contrat, expérience attendue, niveau d'étude, compétences mentionnées et description détaillée. Les profils candidats décrivent les caractéristiques

individuelles : diplôme, niveau d'étude, spécialité, expérience, compétences, mobilité et objectif professionnel. Ces deux sources sont naturellement bruitées : les intitulés ne sont pas normalisés, les compétences peuvent être formulées librement et certaines variables sont incomplètes. La méthodologie doit donc transformer ces données en objets comparables.

ESCO fournit une nomenclature internationale des métiers et compétences. Son intérêt est de disposer d'un vocabulaire structuré et de relations métier-compétence déjà établies. La MEPC permet d'ancrer le système dans la nomenclature camerounaise des métiers. La NCF fournit une structure nationale des niveaux et domaines de formation. L'intégration conjointe de ces sources permet de relier le langage des annonces, les profils des candidats et les classifications institutionnelles.

Table 2.2 – Sources de données mobilisées et utilisation méthodologique

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Source	Nature	Utilisation dans la méthode
Offres d'emploi	Données opérationnelles collectées par KmerAI	Normalisation, extraction de compétences, génération de <code>text_to_embed</code> , indexation vectorielle, création des noeuds d'offres et calcul des scores de matching.
Profils candidats	Base locale de demandeurs	Normalisation du niveau, du diplôme, de la mobilité et du métier visé ; création des embeddings candidats ; analyse de compatibilité avec les offres.
ESCO	Référentiel métiers-compétences	Source de métiers, compétences, groupes ISCO et relations métier-compétence, utilisée pour structurer le graphe et enrichir les correspondances.
MEPC	Nomenclature camerounaise des métiers	Ancrage national des métiers, structuration hiérarchique et alignement partiel avec les groupes internationaux.
NCF	Nomenclature camerounaise des formations	Codification des niveaux et domaines de formation, utilisée pour la compatibilité candidat-offre et l'analyse de montée en compétences.
PDF réglementaires	Documents officiels fragmentés	Constitution de <code>DocChunk</code> pour la recherche référentielle et la citation des sources.

2.1.3 Préparation et normalisation des données

La préparation des données constitue la première couche méthodologique du système. Son objectif est de produire des tables propres, stables et exploitables par les deux moteurs de recherche : le moteur vectoriel et le moteur graphe. Cette étape ne se limite pas au nettoyage cosmétique des textes. Elle détermine la qualité des relations construites, la pertinence des embeddings et la fiabilité des recommandations.

Pour les offres d'emploi, la normalisation consiste d'abord à réduire les doublons et les variations de forme. Une annonce peut apparaître plusieurs fois avec des différences mineures de titre, d'employeur ou de localisation. Le traitement doit donc stabiliser les identifiants et conserver les informations utiles sans multiplier artificiellement les observations. Les champs textuels sont ensuite harmonisés : suppression des caractères parasites, standardisation des espaces, séparation des champs multi-valeurs et conservation des informations de contexte. Les compétences mentionnées sont transformées en listes exploitables, tandis que les localisations et secteurs sont séparés en valeurs principales et valeurs secondaires.

Pour les candidats, la normalisation porte sur le diplôme, le niveau d'étude, la spécialité, l'expérience, les compétences, le secteur visé et la mobilité géographique. Une attention particulière est portée au niveau de formation. Lorsque l'information disponible le permet, le niveau est rapproché de la NCF. Ce rapprochement est indispensable parce qu'une recommandation emploi-compétences ne peut pas se fonder uniquement sur la proximité textuelle : une offre peut être sémantiquement proche d'un profil tout en exigeant un niveau de qualification supérieur.

Chaque offre et chaque candidat reçoit enfin un champ `text_to_embed`. Ce champ synthétise les éléments utiles pour la recherche sémantique : intitulé, compétences, secteur, niveau, description et objectif. Le choix d'un texte synthétique répond à une contrainte pratique : les modèles d'embeddings travaillent sur des séquences textuelles, mais les données métier sont partiellement structurées. Le champ `text_to_embed` sert donc d'interface entre les variables métiers et l'espace vectoriel.

2.1.4 Alignement des référentiels métiers et formations

L'alignement des référentiels répond à un besoin central : rendre comparables des données issues de sources différentes. Les annonces utilisent des formulations libres, ESCO propose une taxonomie internationale, la MEPC suit une logique nationale de classification des métiers et la NCF structure les formations. La méthode adoptée est prudente : elle vise un alignement

exploitable pour la recommandation, sans prétendre établir une équivalence parfaite entre toutes les nomenclatures.

La MEPC est structurée selon ses niveaux hiérarchiques : grands groupes, sous-groupes et groupes de base. La NCF est organisée autour des niveaux de formation, grands domaines, domaines spécialisés et domaines détaillés. Cette structuration permet de conserver l'information institutionnelle au lieu de la réduire à de simples libellés. Les tables obtenues alimentent ensuite le graphe Neo4j et la base vectorielle.

L'alignement avec ESCO s'appuie sur les codes et les proximités de libellés lorsque ces informations sont disponibles. Cette stratégie est méthodologiquement raisonnable, mais elle doit être interprétée avec prudence. Un même groupe de classification peut regrouper plusieurs réalités professionnelles. L'alignement sert donc à améliorer la couverture et l'interprétabilité, non à produire une vérité définitive sur l'équivalence entre métiers.

2.2 Modélisation sémantique et structuration des connaissances

2.2.1 Choix du modèle d'embeddings et fine-tuning

Le système utilise un modèle SentenceTransformer pour représenter les offres, candidats, métiers, compétences et fragments documentaires dans un espace vectoriel. Ce choix est adapté au problème, car le matching emploi-compétences repose largement sur la similarité sémantique entre textes courts ou semi-structurés : intitulés de postes, compétences, descriptions d'annonces, objectifs professionnels et libellés de référentiels. Les SentenceTransformers ont été conçus pour produire des embeddings directement comparables par similarité cosinus, ce qui les rend adaptés aux tâches de recherche sémantique [23].

Le fine-tuning retenu n'est pas un fine-tuning génératif. Il ne vise pas à apprendre au modèle à rédiger des réponses, mais à adapter l'espace vectoriel au vocabulaire local du domaine emploi-compétences. La logique est contrastive : rapprocher les paires pertinentes et éloigner les paires moins pertinentes. Par exemple, une offre de « data analyst » doit être rapprochée d'un profil mentionnant analyse de données, statistique, Python, SQL ou visualisation, même si les formulations exactes diffèrent. A l'inverse, un profil orienté développement web ne doit pas être considéré comme équivalent à une offre de comptabilité uniquement parce que certaines expressions génériques se ressemblent.

Les paires d'apprentissage sont construites à partir des données préparées. Elles peuvent mobiliser les relations offre-compétence, métier-compétence, candidat-métier et offre-candidat lorsque les signaux disponibles sont suffisamment fiables. Cette étape est critique : un fine-

tuning apprend les régularités de ses exemples. Des paires mal construites peuvent dégrader l'espace vectoriel au lieu de l'améliorer. La méthodologie retient donc une construction contrôlée des exemples, puis une évaluation séparée du modèle de base et du modèle adapté.

2.2.2 Indexation vectorielle dans PostgreSQL et pgvector

Les embeddings produits sont stockés dans PostgreSQL avec l'extension pgvector. Ce choix permet de conserver dans un même environnement des représentations vectorielles et des métadonnées relationnelles. pgvector permet l'indexation et la recherche de voisins proches dans PostgreSQL, ce qui facilite l'intégration avec les autres tables du système [2]. Pour accélérer les recherches top-K, la méthode s'appuie sur une logique d'indexation approximative des voisins proches, cohérente avec les travaux sur HNSW [29].

La base vectorielle joue deux rôles. Le premier est le retrieval sémantique : retrouver les offres, métiers, compétences ou documents proches d'une requête. Le second est le préfiltrage pour la recommandation : produire un ensemble de candidats plausibles avant d'appliquer des contraintes plus structurées. Ce préfiltrage est nécessaire, car le graphe de connaissances apporte de l'explicabilité, mais il n'est pas toujours le meilleur premier filtre lorsque la requête est formulée librement.

Chaque résultat vectoriel doit conserver ses métadonnées : identifiant, type d'entité, libellé, source, score de similarité et texte associé. Sans ces métadonnées, la génération finale serait incapable de citer ses sources et le système perdrait sa traçabilité.

2.2.3 Construction du graphe de connaissances sous Neo4j

Le graphe de connaissances représente la couche relationnelle du système. Son rôle est de modéliser explicitement les relations entre les candidats, les offres, les compétences, les métiers, les secteurs, les localisations et les référentiels. Cette représentation est justifiée par la nature même du problème : le matching emploi-compétences n'est pas seulement une question de proximité textuelle, mais une question de relations. Une offre requiert des compétences, un candidat possède des compétences, un métier est classé dans une nomenclature, un niveau de formation conditionne l'accès à certains postes, et une trajectoire de montée en compétences dépend des écarts identifiés.

La méthodologie retient des noeuds correspondant aux entités du système. Les relations décrivent les liens fonctionnels : une offre **REQUIERT** une compétence, un candidat **POSSEDE** une compétence, une offre est rattachée à un secteur ou à une localisation, un métier est classé dans un groupe, et un fragment documentaire provient d'un document référentiel.

Ce graphe sert à trois opérations méthodologiques. Premièrement, il vérifie les correspondances proposées par la recherche vectorielle. Deuxièmement, il explique les recommandations en identifiant les compétences communes et manquantes. Troisièmement, il permet des requêtes relationnelles que la recherche vectorielle gère mal, par exemple : quelles compétences sont associées à un métier donné ? quelles offres exigent une compétence précise ? quelles compétences manquent à un candidat pour viser une famille de métiers ?

2.2.4 Score hybride et logique de skill gap

Le score de recommandation combine plusieurs signaux complémentaires. La similarité sémantique mesure la proximité entre le texte du profil et celui de l'offre. La couverture de compétences mesure la proportion des compétences requises par l'offre qui sont déjà présentes chez le candidat. La compatibilité de niveau vérifie si le niveau de formation du candidat est cohérent avec le niveau attendu par l'offre. L'alignement sectoriel mesure la cohérence entre le secteur ou le domaine du candidat et celui de l'offre.

La forme générale du score est la suivante :

$$S_{hyb}(c, o) = \alpha S_{sem}(c, o) + \beta S_{graph}(c, o) + \gamma C_{ncf}(c, o) + \delta A_{sect}(c, o),$$

où c désigne un candidat, o une offre, S_{sem} la similarité sémantique, S_{graph} la couverture de compétences issue du graphe, C_{ncf} la compatibilité de niveau de formation et A_{sect} l'alignement sectoriel. Cette formulation traduit un choix méthodologique important : le système ne délègue pas toute la décision au LLM. Le LLM intervient pour synthétiser et expliquer, mais le classement repose sur des signaux calculables et auditable.

Les pondérations ne doivent pas être fixées uniquement par intuition. La méthodologie retient donc une calibration statistique par optimisation bayésienne avec Optuna, en utilisant le *Tree-structured Parzen Estimator* comme stratégie de recherche [41]. Dans l'expérience logicielle retenue, le score calibré est volontairement ramené à deux signaux observables dans les sorties d'évaluation : le score sémantique issu de pgvector et le score Neo4j issu de l'enrichissement graphe. Le problème optimisé est donc :

$$S_{cal}(c, o) = w_{sem} S_{sem}(c, o) + w_{Neo4j} S_{Neo4j}(c, o), \quad w_{sem} + w_{Neo4j} = 1.$$

Le principe d'Optuna est le suivant. A chaque essai, l'algorithme propose une valeur candidate de w_{sem} , en déduit $w_{Neo4j} = 1 - w_{sem}$, recalcule le score des offres candidates, rerange les offres pour chaque candidat, puis évalue ce nouveau classement avec une fonction objectif.

Les essais précédents alimentent le modèle probabiliste du *TPESampler* : les zones de poids ayant produit de bons scores sont explorées plus intensément que les zones peu prometteuses. La méthode ne cherche donc pas un coefficient « vrai » au sens économique ; elle cherche la pondération qui maximise empiriquement une métrique de ranking sur l'échantillon d'évaluation.

Trois fonctions objectif sont testées afin de ne pas confondre un bon classement global avec une bonne première recommandation : le NDCG@10 seul, la moyenne NDCG@10–Recall@10–Precision@10 et la moyenne MRR@10–Precision@10. Le NDCG@10 valorise l'ordre complet du top 10 ; le rappel et la précision évaluent la couverture binaire des offres pertinentes ; le MRR@10 met davantage de pression sur le rang de la première offre pertinente. Cette calibration est volontairement séparée de la génération LLM. Elle concerne seulement le moteur de classement. Elle inclut également un garde-fou : les poids proposés par Optuna sont comparés à des solutions de frontière, par exemple un score purement sémantique ou un score purement fondé sur Neo4j. La calibration est donc utilisée comme un test empirique des pondérations, non comme une justification automatique du caractère hybride du système. Si les labels de pertinence sont des proxys et non des jugements humains, les coefficients obtenus doivent être interprétés comme optimaux dans le protocole expérimental courant, pas comme des paramètres définitifs du marché du travail.

L'analyse de *skill gap* complète le score. Elle distingue les compétences acquises, les compétences requises et les compétences manquantes. Cette distinction permet de produire un verdict opérationnel : candidat immédiatement compatible, candidat compatible après montée en compétence, candidat à conserver en vivier ou profil hors cible. Ce verdict est plus utile qu'un simple score numérique, car il transforme la recommandation en aide à la décision.

2.3 Orchestration agentique et génération contrôlée

2.3.1 Architecture Agentic RAG et orchestration LangGraph

L'orchestration du système repose sur un workflow agentique implémentable avec *LangGraph*. Méthodologiquement, LangGraph est retenu parce qu'il permet de représenter l'agent comme un graphe d'états plutôt que comme une simple succession de prompts [39]. La requête utilisateur est d'abord analysée afin d'identifier son intention : diagnostic, recommandation candidat, skill gap, recherche référentielle, question relationnelle sur le graphe, orientation métier ou recherche générale. Le système sélectionne ensuite les outils adaptés. Cette étape de routage est essentielle : interroger pgvector, Neo4j ou les documents réglementaires ne répond

pas au même besoin.

Le workflow suit une chaîne contrôlée : `analyse_request`, `plan_tools`, `execute_tools`, `build_context`, `generate_final_answer`, puis critique éventuelle de la réponse. Le noeud d'analyse identifie le use-case. Le noeud de planification sélectionne les outils. Le noeud d'exécution appelle les bases et services nécessaires. Le noeud de construction de contexte organise les résultats récupérés. Le noeud de génération produit la réponse finale avec le LLM. Le critic vérifie ensuite si la réponse paraît suffisamment fidèle au contexte ; en cas de faiblesse, le workflow peut élargir le contexte avant de régénérer la réponse.

Ce choix s'inscrit dans la logique de l'Agentique RAG : le système n'est pas un simple prompt envoyé à un modèle, mais une orchestration d'outils spécialisés autour d'un état partagé [35, 36]. Dans le présent travail, l'agent n'est pas conçu comme une entité autonome non contrôlée. Il est plutôt un routeur raisonné, limité à des outils définis, dont les sorties sont tracées.

2.3.2 Text2Cypher et rôle du LLM

Deux usages du LLM sont distingués. Le premier concerne l'interrogation du graphe en langage naturel. Pour les questions de type `graph_query` ou les recherches générales nécessitant un raisonnement relationnel, la question peut être transformée en requête Cypher par un module Text2Cypher. Le modèle retenu pour cette tâche est `neo4j/text2cypher-gemma-2-9b-it-finetuned`. La génération de Cypher est encadrée par le schéma Neo4j : types de noeuds, propriétés et relations autorisées. Cette contrainte est indispensable, car un modèle génératif qui ne connaît pas le schéma réel peut produire des requêtes syntaxiquement plausibles mais inexécutables ou non pertinentes.

Le second usage du LLM concerne la génération de la réponse finale. Le système utilise l'API OpenRouter avec le modèle `openai/gpt-oss-20b:free`. Ce modèle ne doit pas inventer la réponse à partir de ses connaissances générales. Il reçoit un contexte construit par le workflow : résultats pgvector, lignes Neo4j, fragments documentaires et métadonnées de source. Le prompt lui demande de répondre en français, de structurer la réponse et de citer les informations utilisées.

Cette séparation des rôles est méthodologiquement importante. Text2Cypher sert à produire une requête structurée vers le graphe. Le LLM final sert à formuler une synthèse lisible. Dans les deux cas, le modèle est subordonné aux données disponibles et non l'inverse.

2.3.3 Modes d'accès et traçabilité des décisions

Le système prévoit trois modes d'accès : une interface Streamlit pour l'usage conversationnel, une API FastAPI pour l'intégration applicative et une CLI pour les tests de développement. Ces interfaces ne changent pas la logique de recommandation ; elles exposent le même moteur sous des formes différentes. La CLI facilite la vérification rapide des use-cases, Streamlit permet une démonstration interactive et FastAPI prépare l'intégration dans un environnement applicatif comme la plateforme Kmer-AI, les agences de recrutement (FNE et l'ACPE).

La traçabilité est intégrée à plusieurs niveaux. Les traces du workflow indiquent les noeuds traversés et les outils appelés. Les résultats conservent les identifiants des entités, les sources documentaires, les scores et les chemins du graphe. La réponse finale doit citer les éléments utilisés. Cette exigence est décisive dans un système d'aide à l'orientation ou au recrutement : une recommandation sans preuve peut être séduisante, mais elle n'est pas défendable.

2.4 Évaluation prévue, limites et synthèse méthodologique

2.4.1 Protocole d'évaluation prévu

L'évaluation est conçue comme une évaluation en plusieurs étages. Le premier étage porte sur le retrieval vectoriel : le système retrouve-t-il les offres, compétences, métiers ou documents attendus dans les premiers résultats ? Les métriques retenues sont notamment Precision@K, Recall@K, MRR@K, MAP@K et NDCG@K, couramment utilisées pour les tâches de recherche d'information [27, 28].

Le deuxième étage porte sur le graphe : les relations retournées sont-elles correctes, utiles et interprétables ? Les tests doivent couvrir les requêtes métier-compétence, candidat-compétence, offre-compétence, compatibilité de niveau et recherche de chemins explicatifs. Le troisième étage porte sur le score hybride : il s'agit de comparer les classements produits par la recherche vectorielle seule, le graphe seul et la combinaison hybride. Le quatrième étage porte sur la réponse générée : fidélité au contexte, pertinence par rapport à la question, couverture des preuves, citations et utilité opérationnelle.

La méthode distingue donc clairement performance de récupération et qualité de génération. Une réponse bien rédigée ne prouve pas que le système a retrouvé les bonnes informations. Inversement, un bon retrieval ne garantit pas une bonne synthèse. Cette séparation protège l'analyse contre une erreur fréquente dans les systèmes RAG : juger le système uniquement à l'apparence de la réponse finale.

2.4.2 Limites méthodologiques et précautions d'interprétation

Plusieurs limites doivent être prises en compte dès la conception. Premièrement, les données d'offres et de candidats peuvent contenir des omissions, des doublons et des formulations ambiguës. Deuxièmement, l'alignement entre référentiels reste partiel : il améliore la structuration, mais ne garantit pas une équivalence parfaite entre catégories. Troisièmement, le graphe dépend de la qualité des relations extraites ou importées. Une relation manquante peut conduire à sous-estimer une compatibilité réelle. Quatrièmement, les LLM peuvent produire des formulations plausibles mais non soutenues par les données ; c'est pourquoi le contexte, les citations et le critic sont nécessaires.

Une autre limite est matérielle. L'utilisation locale ou semi-locale de modèles spécialisés, notamment pour Text2Cypher, peut être contrainte par la mémoire disponible. Cette contrainte influence les choix techniques : recours à une API pour la génération finale, chargement contrôlé du modèle Text2Cypher et possibilité de fallback vers des templates Cypher sécurisés. Ces choix ne sont pas des faiblesses méthodologiques en soi ; ils traduisent l'adaptation du système aux ressources disponibles.

2.4.3 Ce que l'architecture garantit et ses exigences

La méthodologie proposée construit un système hybride plutôt qu'un modèle isolé. Elle part de données locales hétérogènes, les normalise, les rattache à des référentiels, les encode dans un espace vectoriel, les structure dans un graphe de connaissances, puis orchestre leur interrogation par un workflow agentique. Le rôle du LLM est strictement encadré : transformer certaines questions en requêtes graphe, puis formuler une réponse fondée sur les preuves récupérées.

Cette approche répond aux objectifs du mémoire. Elle permet de dépasser la recherche par mots-clés, d'exploiter les référentiels institutionnels, de produire des recommandations explicables et de relier chaque verdict à des compétences, des niveaux de formation et des sources identifiables. Les chapitres suivants présentent la réalisation opérationnelle de cette méthodologie, puis l'évaluation de ses performances.

Deuxième partie

Présentation des résultats

Construction opérationnelle du système hybride emploi-compétences

Ce chapitre présente la matérialisation opérationnelle de l'architecture méthodologique décrite précédemment. Son objectif est de montrer ce qui a été effectivement construit, alimenté, indexé, relié et rendu exploitable dans le système de recommandation emploi-compétences.

La présentation suit l'ordre réel du workflow : diagnostic des données exploitables, adaptation du modèle d'embeddings, matérialisation de la mémoire vectorielle, construction du graphe Neo4j, calibration du score hybride, puis orchestration agentique et exposition du système. Les résultats sont donc introduits au moment où ils justifient une décision d'implémentation, et non comme une simple succession de graphiques.

3.1 Données exploitables et contraintes de qualité

3.1.1 Volumes disponibles après nettoyage

Le corpus utilisé dans l'implémentation ne doit pas être lu comme une simple base descriptive. Il constitue la matière première du moteur de recommandation : chaque variable conservée doit donc apporter un signal exploitable pour rapprocher une offre, un candidat, une compétence, un métier ou un référentiel. La question d'ingénierie posée ici est directe : le système dispose-t-il d'assez d'information structurée pour recommander correctement, et quelles limites doivent être compensées par l'architecture hybride ? Le tableau suivant synthétise uniquement les volumes qui conditionnent directement la faisabilité technique du système.

Table 3.1 – Volumes réellement exploités par les modules de recommandation

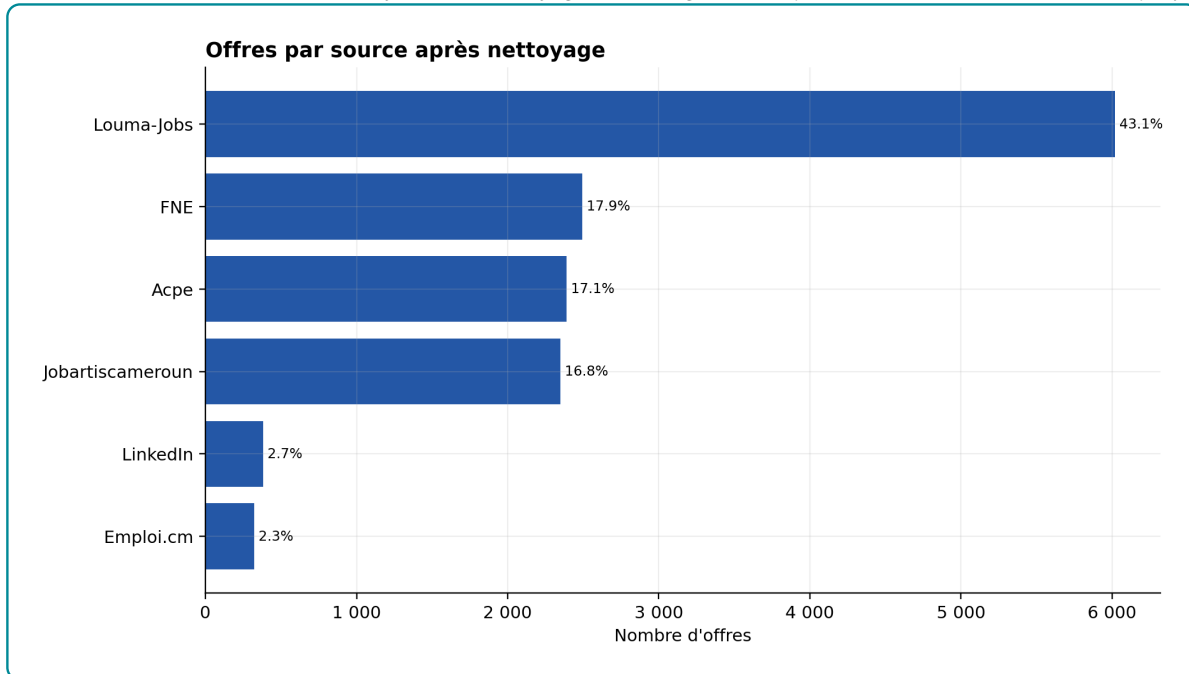
Source : Nos travaux, diagnostics PostgreSQL–pgvector, à partir des embeddings, des métadonnées et des fragments documentaires indexés.

Objet exploité	Volume	Utilité pour le système
Offres normalisées	13 957	Base principale à recommander, indexée dans pgvector et reliée au graphe.
Candidats normalisés	41 298	Vivier à partir duquel sont calculés les rapprochements offre–profil.
Compétences indexées	13 939	Signal de matching, de filtrage et de justification du <i>skill gap</i> .
Secteurs nettoyés	1 039	Signal de contexte pour limiter les rapprochements absurdes entre domaines trop éloignés.
Métiers indexés	3 039	Niveau d’agrégation utile pour stabiliser les recommandations au-delà des intitulés d’offres.
Groupe métiers MEPC	209	Référentiel externe pour structurer les proximités métier.
Domaines détaillés NCF	201	Référentiel de formation utilisé pour contextualiser les niveaux et domaines.
Localisations représentées dans Neo4j	312	Signal géographique pour filtrage souple et explication des résultats.

Le premier résultat d’ingénierie est positif : les volumes sont suffisants pour construire un système hybride, car le corpus ne se limite pas aux offres. Il contient aussi des candidats, des compétences, des secteurs, des métiers, des localisations et des référentiels. En revanche, l’asymétrie entre 13 957 offres et 41 298 candidats impose de contrôler les effets de popularité : un secteur très représenté peut devenir dominant même lorsque la pertinence individuelle est moyenne. Cette limite justifie l’usage d’un reranking hybride plutôt qu’un classement purement vectoriel.

Figure 3.1 – Répartition des offres par source après nettoyage

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.



Les 13 957 offres proviennent de six sources : Louma-Jobs, FNE, ACPE, Jobartis Cameroun, LinkedIn et Emploi.cm. Louma-Jobs concentre 6 021 offres, soit 43,14 % du fichier. Le FNE représente 2 494 offres, ACPE 2 388, Jobartis Cameroun 2 350, LinkedIn 380 et Emploi.cm 324. Cette concentration n'est pas neutre : le modèle observe davantage le vocabulaire, les formats et les secteurs présents sur Louma-Jobs que ceux des autres plateformes. Le corpus est donc robuste pour construire un prototype opérationnel, mais il ne doit pas être présenté comme une photographie exhaustive du marché camerounais de l'emploi. La bonne lecture est la suivante : les sources sont assez nombreuses pour éviter une dépendance totale à une seule plateforme, mais assez déséquilibrées pour imposer une analyse par source dans l'évaluation.

3.1.2 Qualité des variables utiles au matching emploi-compétences

La qualité d'un système de recommandation dépend moins du nombre brut de lignes que de la disponibilité des variables qui portent la décision : description textuelle, compétences, localisation, secteur, niveau et source. Le diagnostic est donc séparé en deux lectures complémentaires. La première vérifie si la localisation peut réellement servir au filtrage et à l'explication. La seconde examine les compétences et les descriptions, c'est-à-dire les deux signaux qui alimentent directement les embeddings et le graphe.

3.1.2.1 Localisation exploitable pour filtrer sans exclure

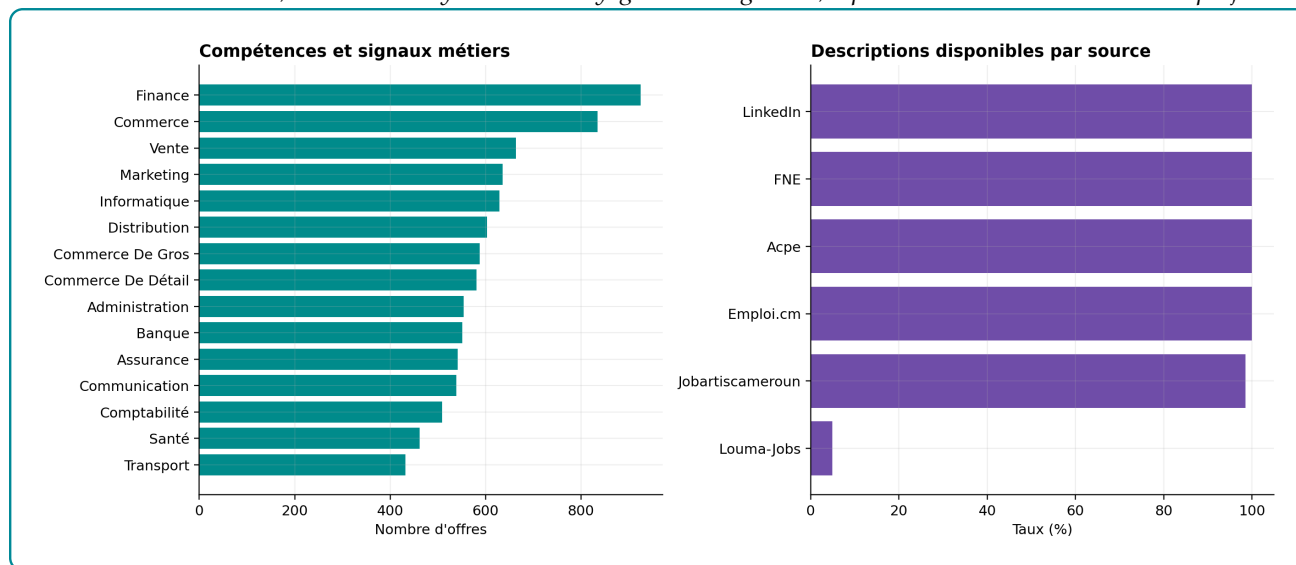
Le diagnostic détaillé de localisation est renvoyé en annexe (figure A.1). Dans le corps du chapitre, on retient seulement que la localisation est suffisamment renseignée pour filtrer et expliquer les recommandations, mais pas assez homogène pour être utilisée comme contrainte dure.

La localisation confirme que le système dispose d'un signal géographique exploitable : 79,88 % des offres sont rattachées au Cameroun et 20,12 % à l'international. Les localités dominantes sont notamment Yaoundé, Douala, Bafoussam, Kribi, Maroua, Garoua, Bamenda et Buéa, avec aussi des localités plus fines comme Ngousso ou Yaoundé-Ekoumdoum. Lorsque la ville n'est pas indiquée mais que le pays est renseigné, l'offre n'est plus classée comme localisation globalement inconnue : elle conserve une étiquette du type « Cameroun (ville non précisée) » ou « États-Unis (ville non précisée) ». Le nettoyage a donc privilégié une stratégie hiérarchique : exploiter d'abord la ville, sinon le pays, et ne pas inventer une ville absente du fichier brut.

3.1.2.2 Compétences et descriptions : signal utile mais hétérogène

Figure 3.2 – Compétences fréquentes et disponibilité descriptive

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.



Les signaux les plus fréquents sont Finance, Commerce, Vente, Marketing, Informatique, Distribution, Banque, Assurance, Communication, Comptabilité, Santé et Transport. Cette structure est cohérente avec un système emploi-compétences, mais elle révèle un risque méthodologique : plusieurs libellés sont des domaines d'activité plutôt que des compétences observables. Dans l'implémentation, ces variables sont donc utilisées comme signaux d'ap-

pariement et de contexte, non comme preuve définitive d'une compétence maîtrisée par un candidat.

La disponibilité descriptive est le principal point faible du corpus. Les sources FNE, ACPE, LinkedIn et Emploi.cm disposent d'une description pour toutes leurs offres dans le fichier nettoyé ; Jobartis Cameroun atteint 98,55 %. La limite vient de Louma-Jobs : seulement 295 descriptions disponibles sur 6 021 offres, soit 4,90 %. Cette hétérogénéité explique un choix central de l'ETL : la représentation envoyée au modèle d'embeddings ne peut pas être uniquement le détail textuel de l'offre ; elle doit combiner métadonnées, compétences, secteur, localisation et description lorsque celle-ci existe.

3.1.3 Nettoyage réalisé et limites conservées

Le nettoyage appliqué au corpus répond à un principe conservateur : corriger les incohérences techniques sans créer une information qui n'existe pas. Les libellés ont été harmonisés pour limiter les effets de casse et de formats d'écriture. Pour la localisation, la règle retenue est hiérarchique : si la ville est disponible, elle est utilisée ; si la ville est absente mais que le pays est connu, le pays est conservé ; si aucune information géographique exploitable n'existe, l'observation reste non localisable. Cette règle évite de supprimer silencieusement les offres concernées, tout en signalant au moteur que le filtrage par ville doit être moins contraignant que le filtrage par pays.

La déduplication des offres suit également une règle stricte : une offre n'est considérée comme doublon que lorsque toute la ligne normalisée est identique sur l'ensemble des variables. Ce choix est rigoureux pour un corpus multi-sources, car deux offres peuvent partager un intitulé ou un employeur tout en différant par la ville, la source, la date, le niveau, le secteur ou la description. Une déduplication floue aurait risqué de supprimer des opportunités réellement distinctes.

Le tableau détaillé des règles de nettoyage est placé en annexe (tableau A.4). Le chapitre conserve uniquement le principe d'ingénierie : corriger les formats sans inventer une information absente des données brutes.

Les graphiques secondaires détaillant la structure sectorielle, les villes détaillées, les niveaux NCF, l'expérience, la structure des candidats, les longueurs de textes et les référentiels MEPC-NCF sont renvoyés en annexes. Dans ce chapitre, seuls les résultats qui changent une décision d'architecture sont conservés : enrichir les embeddings, utiliser pgvector pour le rappel rapide, exploiter Neo4j pour les contraintes métier, et garder un critic pour contrôler les réponses générées.

3.2 Adaptation du modèle d'embeddings au domaine emploi-compétences

3.2.1 Encodeur retenu et paires contrastives

Le modèle de base retenu est `sentence-transformers/all-MiniLM-L6-v2`. Ce choix est défendable pour une raison d'ingénierie : il produit des embeddings de 384 dimensions, donc moins coûteux à stocker et à rechercher que des modèles à 768 dimensions, tout en restant compatible avec une indexation HNSW dans `pgvector`. Dans ce projet, la contrainte est concrète : le même encodeur sert ensuite à vectoriser les candidats, les offres, les compétences, les métiers et les référentiels. Un modèle plus lourd peut améliorer certains cas sémantiques, mais il augmente la taille de stockage, la latence d'encodage et le coût de réindexation.

Ce modèle est acceptable comme point de départ parce qu'il fournit déjà une représentation dense générale des phrases. Il n'est toutefois pas suffisant comme modèle final. Un encodeur généraliste rapproche des textes proches en langage courant, mais il ne connaît pas nécessairement les proximités utiles au domaine emploi-compétences camerounais : intitulés locaux, secteurs, niveaux de formation, familles métiers, descriptions incomplètes et compétences parfois formulées comme domaines. Le fine-tuning vise donc à spécialiser l'espace vectoriel sans changer l'architecture de production.

Table 3.2 – Protocole de construction des paires de fine-tuning

Source : Nos travaux, traitements Python et SentenceTransformer, à partir des paires emploi-compétences et des entités nettoyées du projet.

Élément	Implémentation observée
Modèle de départ	sentence-transformers/all-MiniLM-L6-v2, embeddings denses de 384 dimensions.
Rôle de sentence1	Représentation structurée de l'offre : titre, secteur, contrat, niveau d'études, expérience, ville et compétences.
Rôle de sentence2	Détails nettoyés de l'annonce uniquement, c'est-à-dire le texte descriptif que le modèle doit rapprocher de la représentation structurée.
Filtrage informatif	Conservation des paires avec compétences disponibles, sentence1 d'au moins 40 caractères et sentence2 d'au moins 120 caractères; le champ ft_eligible est appliqué lorsqu'il existe.
Nombre total de paires	6 476 paires offre-description.
Split	4 542 paires en train, 981 en validation et 953 en test, avec stratification par secteur principal.

La structure sentence1/sentence2 est volontairement asymétrique. sentence1 représente la requête métier structurée que le système manipulera souvent en production : métadonnées, compétences et contexte de l'offre. sentence2 représente le texte descriptif à retrouver. Ce choix entraîne le modèle à rapprocher une description naturelle d'une formulation structurée, ce qui correspond mieux au besoin du système qu'un entraînement sur deux phrases descriptives ordinaires.

Le filtrage des offres informatives est nécessaire. Entraîner le modèle sur des paires dont la description est vide, trop courte ou sans compétences introduirait du bruit : le modèle apprendrait à rapprocher des textes pauvres au lieu de structurer le domaine. Le seuil de 120 caractères pour sentence2 et de 40 caractères pour sentence1 est donc un garde-fou d'apprentissage. Il réduit le volume utilisé pour le fine-tuning, mais améliore la qualité du signal.

3.2.2 Fine-tuning : contraintes de calcul et preuve de gain

3.2.2.1 Paramétrage contraint et dynamique d'apprentissage

Table 3.3 – Hyperparamètres du fine-tuning SentenceTransformer

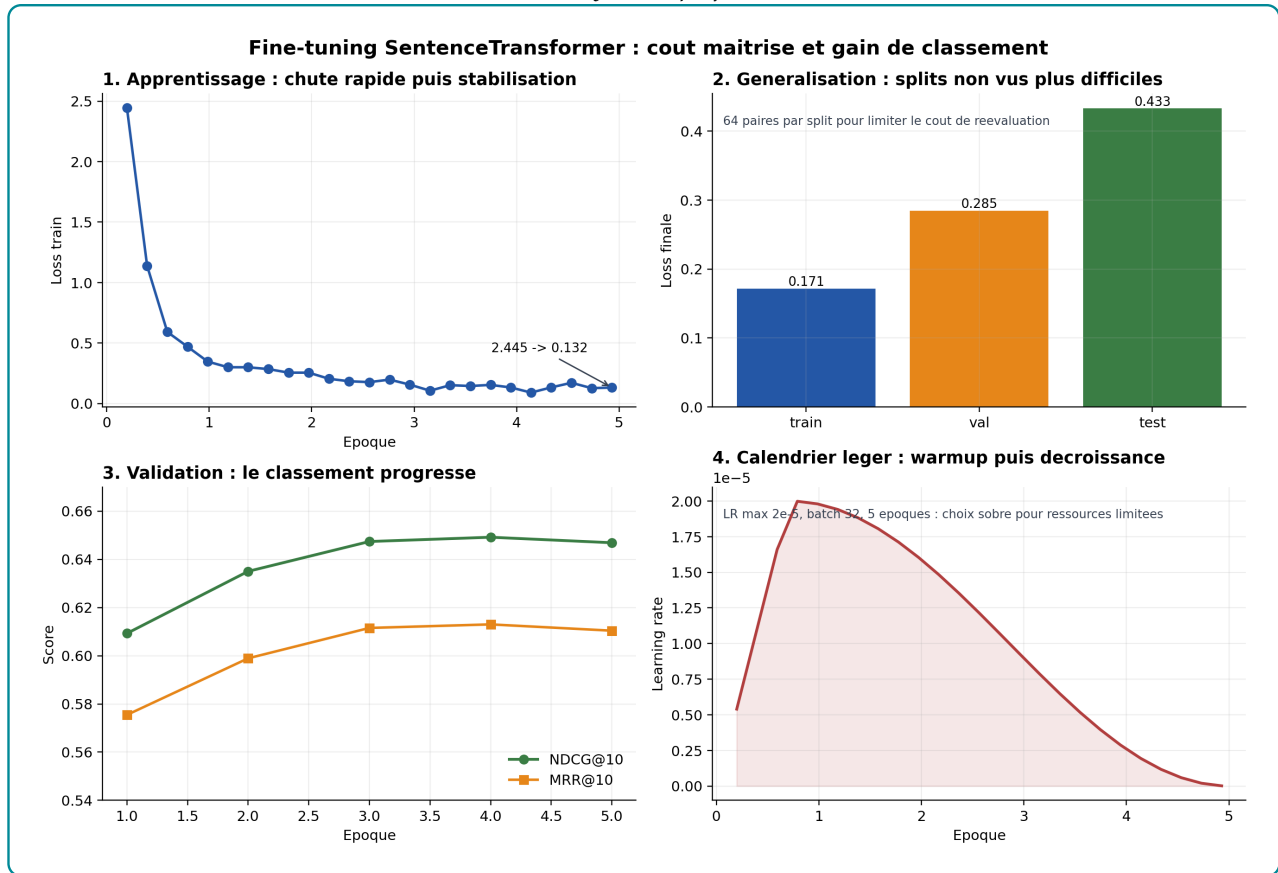
Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Hyperparamètre	Valeur
Nombre d'époques	5
Batch size	32
Négatifs implicites par exemple	31
Learning rate maximal	2×10^{-5}
Scheduler	Cosine avec warmup
Warmup	100 étapes configurées, limitées par les étapes disponibles de l'époque
Perte	<i>MultipleNegativesRankingLoss</i> avec similarité cosinus
Temps d'entraînement observé	15 160 s
Checkpoint retenu	Meilleur checkpoint sur validation autour de l'époque 4

Ces hyperparamètres ne sont pas des valeurs arbitraires. Ils traduisent un compromis imposé par les ressources disponibles : le fine-tuning devait rester exécutable localement, sans infrastructure GPU lourde ni entraînement long. Le modèle `all-MiniLM-L6-v2` a donc été conservé parce qu'il produit des vecteurs de 384 dimensions, moins coûteux à encoder, stocker et rechercher dans `pgvector`. Le batch de 32 limite la consommation mémoire tout en conservant assez d'exemples négatifs implicites pour la *MultipleNegativesRankingLoss*. Le nombre de cinq époques évite un entraînement excessif sur un corpus de paires encore imparfait. Le *learning rate* de 2×10^{-5} , avec *warmup* puis décroissance, permet une adaptation progressive du modèle sans déstabiliser les représentations déjà apprises.

Figure 3.3 – Diagnostic du fine-tuning : perte, généralisation, métriques de validation et calendrier du learning rate

Source : Nos travaux, traitements Python et SentenceTransformer, à partir des paires emploi-compétences et des entités nettoyées du projet.



La figure 3.3 se lit en quatre blocs. Le premier montre que la perte d'entraînement chute fortement au début, de 2,445 à 0,132 en fin d'entraînement : le modèle apprend rapidement à rapprocher les paires positives du domaine. Le deuxième bloc compare la perte finale sur un échantillon fixe de 64 paires par split : 0,171 sur train, 0,285 sur validation et 0,433 sur test. Cette hausse est normale : les jeux non vus sont plus difficiles que les exemples utilisés pendant l'apprentissage. Le troisième bloc montre que les métriques de validation progressent jusqu'aux dernières époques, avec un léger tassement final ; il n'y a donc pas de signal évident d'effondrement. Le quatrième bloc rappelle le choix de ressources : un *learning rate* maximal de 2×10^{-5} , 100 étapes de *warmup*, un batch de 32 et seulement cinq époques. Le protocole cherche donc un gain utile de classement, pas une optimisation coûteuse et difficile à reproduire.

Ces valeurs ne remplacent pas les métriques de ranking, car la *MultipleNegativesRankingLoss* dépend de la composition des batches. Elles servent surtout de contrôle de cohérence : si la perte train baissait mais que la validation stagnait ou chutait, le fine-tuning serait suspect. Ici,

la baisse de perte et la progression du NDCG@10/MRR@10 indiquent que l'adaptation du modèle améliore bien le retrieval dans le domaine emploi-compétences.

La validation atteint un NDCG@10 maximal de 0,6492 à l'époque 4, avant un léger recul à 0,6469 à l'époque 5. Ce profil est cohérent avec un apprentissage rapide suivi d'un plateau. Il serait donc fragile de prolonger l'entraînement sans contrôle : au-delà d'un certain point, le gain marginal devient faible et le risque de sur-adaptation augmente.

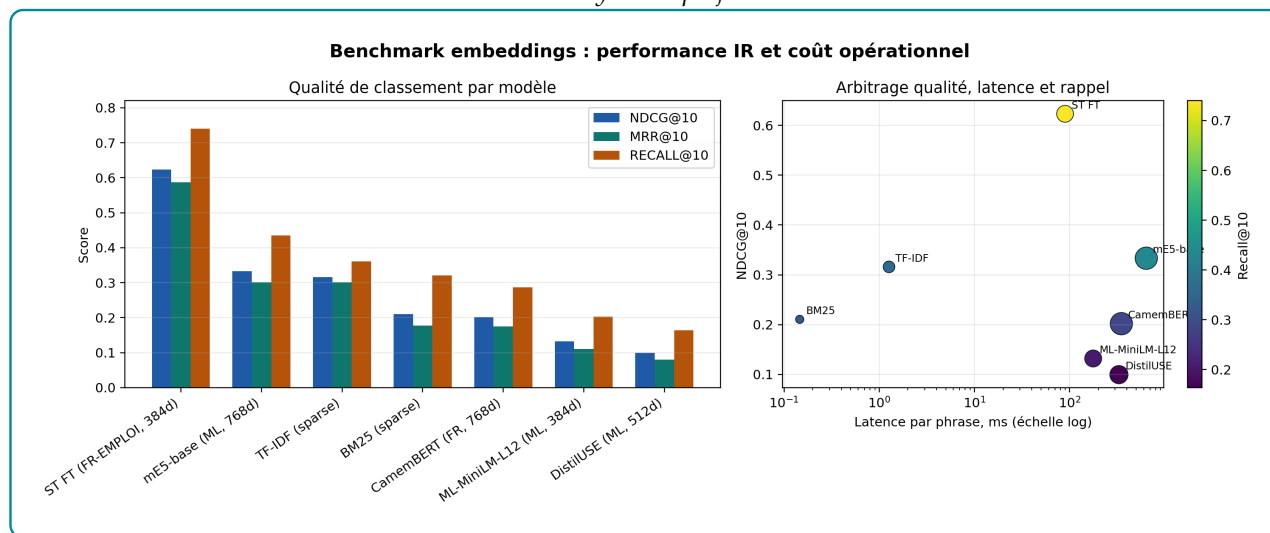
3.2.2.2 Preuve de gain sur le ranking sémantique

La comparaison numérique directe entre baseline et modèle fine-tuné est reportée en annexe (tableau A.5), tandis que l'évaluation complète du retrieval est reprise au chapitre 4.

Cette comparaison a été régénérée sur 953 paires de test après la mise à jour du jeu de paires. Elle est la preuve la plus directe de l'intérêt du fine-tuning : le modèle de base ne structure pas correctement le domaine dans ce protocole, alors que le modèle adapté récupère beaucoup plus souvent la description pertinente dans les premiers rangs. Le NDCG@10 est multiplié par 3,7 et le MRR@10 par 4,2. Le gain de recall@10 est particulièrement important, car l'application ne dépend pas d'une seule réponse immédiate ; elle récupère d'abord un ensemble court de candidats pertinents avant reranking hybride.

Figure 3.4 – Comparaison des modèles d'embeddings : qualité de classement, rappel et latence

Source : Nos travaux, traitements Python et SentenceTransformer, à partir des paires emploi-compétences et des entités nettoyées du projet.



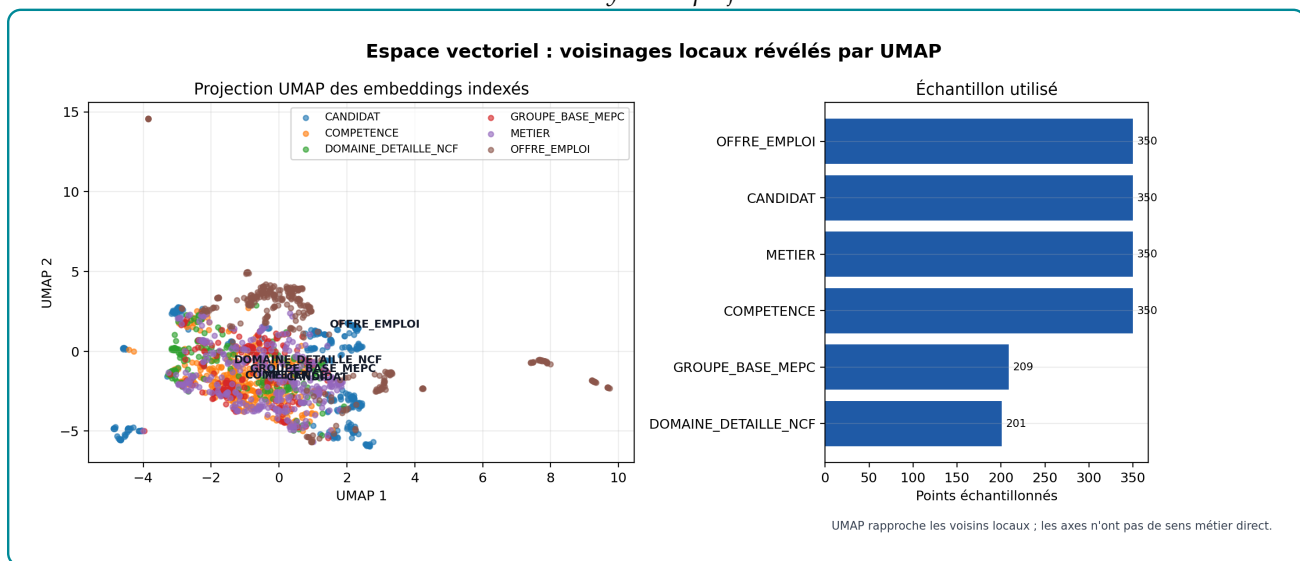
Le benchmark multi-modèles confirme le résultat. Le modèle fine-tuné atteint un NDCG@10 de 0,6232, un MRR@10 de 0,5874 et un recall@10 de 0,7398. Le meilleur modèle généraliste dense testé, mE5-base, atteint seulement 0,3333 en NDCG@10 et 0,4355 en recall@10, avec une latence nettement plus élevée. TF-IDF reste très rapide et compétitif sur certains signaux lexi-

caux, mais son recall@10 reste inférieur à celui du modèle fine-tuné. L'intérêt du fine-tuning est donc empirique, pas seulement théorique. La bonne architecture n'oppose pas « deep learning » et lexical ; elle utilise le modèle fine-tuné comme signal sémantique principal et conserve les signaux lexicaux comme complément.

3.2.3 Structure de l'espace vectoriel après fine-tuning

Figure 3.5 – Projection UMAP des embeddings indexés

Source : Nos travaux, traitements Python et SentenceTransformer, à partir des paires emploi-compétences et des entités nettoyées du projet.



La figure 3.5 utilise UMAP pour projeter en deux dimensions un échantillon de 1810 embeddings indexés : 350 candidats, 350 offres, 350 compétences, 350 métiers, 209 groupes de base MEPC et 201 domaines détaillés NCF. UMAP est ici un outil de lecture visuelle. La première lecture porte sur la séparation partielle des familles d'objets. Les offres d'emploi apparaissent plus excentrées, avec un centroïde situé dans une zone différente de celle des autres types d'entités. C'est cohérent avec leur contenu : une offre combine titre, secteur, contrat, localisation, niveau et description, donc un vocabulaire plus contextuel. Les candidats, compétences, métiers, groupes MEPC et domaines NCF sont davantage rapprochés dans la partie centrale du nuage. Cette proximité n'est pas un défaut. Le système a précisément besoin que ces objets partagent un espace commun : un candidat doit pouvoir être rapproché de compétences, de métiers et d'offres, sinon pgvector ne pourrait pas servir de premier étage de retrieval.

La deuxième lecture concerne le recouvrement entre catégories. Le nuage ne montre pas des blocs parfaitement séparés, et il ne faut pas chercher cette séparation. Dans un système

emploi-compétences, une compétence, un métier et un profil candidat peuvent partager les mêmes termes : « comptabilité », « vente », « marketing », « informatique ». Le recouvrement visuel signifie donc que le modèle place dans une même région des objets qui parlent du même domaine, même s'ils n'ont pas le même type technique dans la base. C'est utile pour récupérer des voisins sémantiques ; c'est aussi la raison pour laquelle Neo4j reste nécessaire ensuite, car pgvector rapproche les textes mais ne vérifie pas à lui seul les relations POSSEDE, REQUIERT ou NECESSITE.

La visualisation doit enfin être lue avec discipline. UMAP ne prouve pas que les recommandations sont bonnes et ne remplace pas le benchmark. Il sert à vérifier que l'espace vectoriel n'est pas incohérent : les offres ne sont pas mélangées au hasard, les objets référentiels restent raccordés aux profils et aux compétences, et les zones mixtes rendent possible la recherche sémantique. La preuve quantitative reste le ranking présenté en section précédente, notamment le NDCG@10, le MRR@10 et le recall@10.

3.3 Mémoire vectorielle pgvector pour le retrieval

La base pgvector n'est pas utilisée comme un simple entrepôt de vecteurs. Elle constitue le premier étage du retrieval de l'Agentic RAG : transformer une requête en vecteur, récupérer un top-K d'entités proches, puis transmettre ces résultats au moteur hybride et au graphe Neo4j. La preuve attendue est donc statistique et opérationnelle : volume indexé, couverture des entités, dimension des vecteurs, index effectivement présents et latence de recherche.

Table 3.4 – Distribution des embeddings par type d'entité

Source : Nos travaux, diagnostics PostgreSQL–pgvector, à partir des embeddings, des métadonnées et des fragments documentaires indexés.

Type d'entité	Vecteurs	Part	Couverture Neo4j
Candidats	41 298	56,85 %	100 %
Offres d'emploi	13 957	19,22 %	100 %
Compétences	13 939	19,19 %	100 %
Métiers	3 039	4,18 %	100 %
Groupes de base MEPC	209	0,29 %	100 %
Domaines détaillés NCF	201	0,28 %	100 %
Total entités structurées	72 643	100 %	100 %
Chunks documentaires	142	–	100 %

Cette distribution montre que pgvector couvre les deux côtés du matching. Les candidats représentent 56,85 % des embeddings et les offres 19,22 %. Les compétences et métiers forment le vocabulaire pivot qui permet de passer d'une similarité textuelle à une logique emploi-compétences. La couverture Neo4j de 100 % est déterminante : chaque résultat vectoriel peut être replacé dans le graphe pour récupérer secteur, niveau, localisation, compétences reliées ou chemin explicatif.

Table 3.5 – Structure opérationnelle des tables pgvector

Source : Nos travaux, diagnostics PostgreSQL–pgvector, à partir des embeddings, des métadonnées et des fragments documentaires indexés.

Table	Rôle	Champs et usages clés
embeddings	Recherche sémantique sur les entités structurées.	entity_kind, entity_id, model_id, embedding, label_fr, text_to_embed, neo4j_node_id.
doc_chunks	Recherche documentaire sur les référentiels.	chunk_id, source, section_title, subsection_title, chunk_text, chunk_strategy, ncf_code, mepc_code, embedding, neo4j_node_id.

Le modèle d'encodage utilisé est all-MiniLM-L6-v2-ft-offres-cm. Les vecteurs ont une dimension de 384, ce qui garde un compromis raisonnable entre précision de retrieval et coût

d'indexation. Au total, la brique vectorielle contient 72 643 embeddings d'entités structurées et 142 embeddings documentaires, soit 72 785 vecteurs exploitables par le retrieval.

3.3.1 Stratégie de chunking des référentiels

Les chunks documentaires ne sont pas produits par un découpage arbitraire du texte. La stratégie d'ingestion est structurelle par défaut : elle détecte les titres de section à partir de la mise en forme et des motifs numérotés, puis chaque titre ouvre un chunk dont le contenu s'étend jusqu'au titre suivant. Les fragments trop courts, inférieurs à 80 caractères, sont fusionnés avec le fragment précédent afin d'éviter d'indexer des passages sans contexte. Une limite supérieure de 3 000 caractères contrôle la taille maximale du passage envoyé à l'encodeur. Cette stratégie est cohérente avec les documents exploités, car la NCF, le MEPC et le référentiel des diplômes sont des documents hiérarchisés : niveaux, domaines, groupes, sous-groupes, métiers et descripteurs.

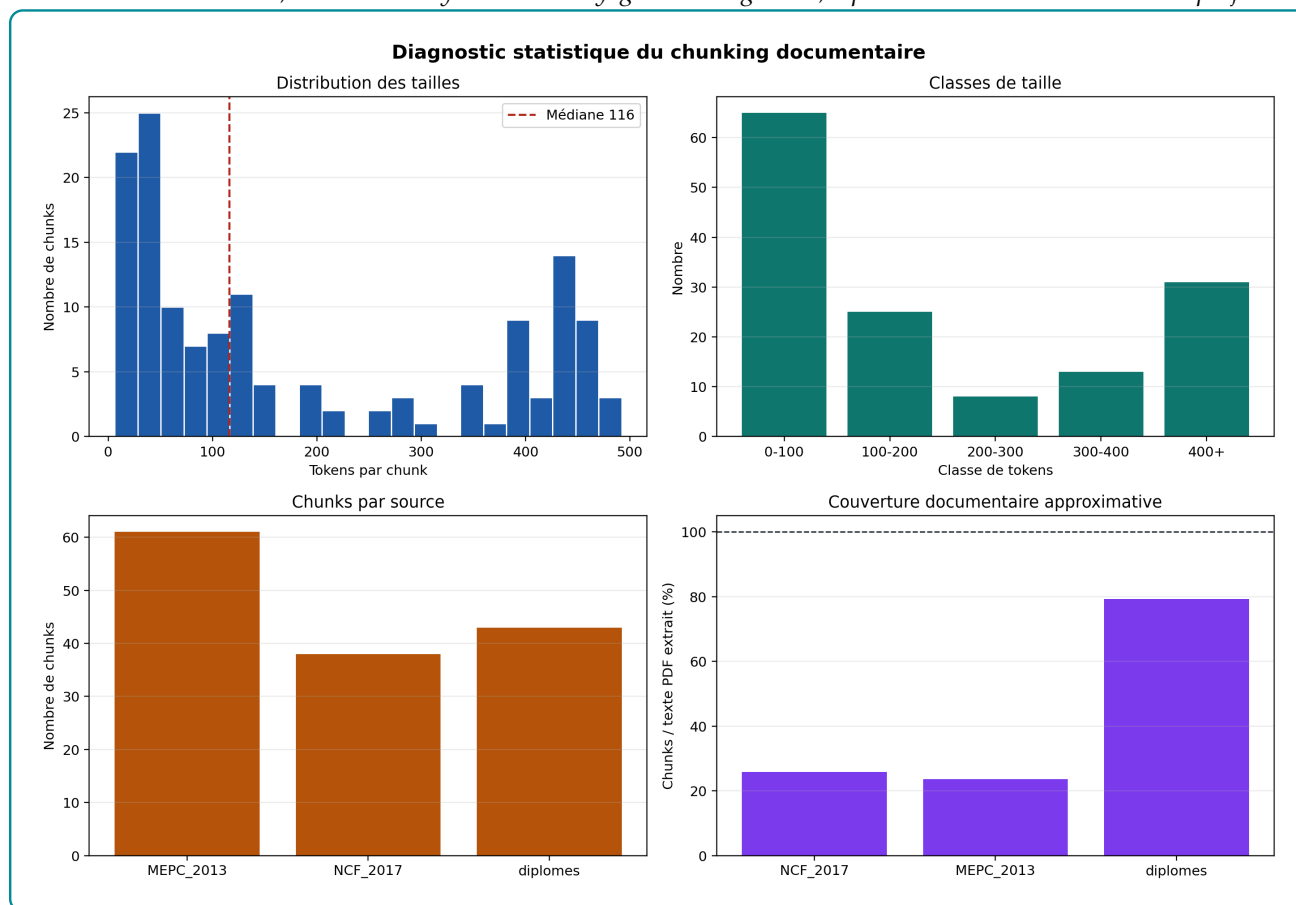
Le pipeline prévoit aussi une stratégie sémantique de secours : si la structure n'est pas détectée, le texte est découpé en fenêtres glissantes de 800 caractères avec un recouvrement de 120 caractères. Cette option n'a pas été nécessaire dans l'index courant : les 142 chunks effectivement présents dans `doc_chunks` ont tous été générés par la stratégie structurelle. Le taux de recouvrement observé est donc nul dans la base actuelle. C'est une information importante pour l'évaluation, car le retrieval documentaire ne récupère pas seulement des morceaux de taille fixe ; il récupère des unités qui correspondent autant que possible à des sections réglementaires interprétables.

Les statistiques détaillées du chunking documentaire sont placées en annexe (tableau A.6). Le chapitre conserve la logique : découper les référentiels selon leur structure afin d'éviter des fragments sans contexte.

Ce diagnostic est plus nuancé qu'une simple description de paramètres. La taille médiane de 116 tokens produit des unités courtes, adaptées au retrieval, mais le coefficient de variation de 0,91 indique une hétérogénéité importante. Certains chunks sont très courts, ce qui peut créer des passages peu informatifs ; d'autres approchent 500 tokens, ce qui peut diluer le signal dans l'embedding. Cette variabilité n'est pas automatiquement mauvaise : elle reflète la structure inégale des référentiels. Elle impose cependant de contrôler la qualité des chunks récupérés dans les réponses RAG.

Figure 3.6 – Distribution des tailles, sources et couverture du chunking documentaire

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.



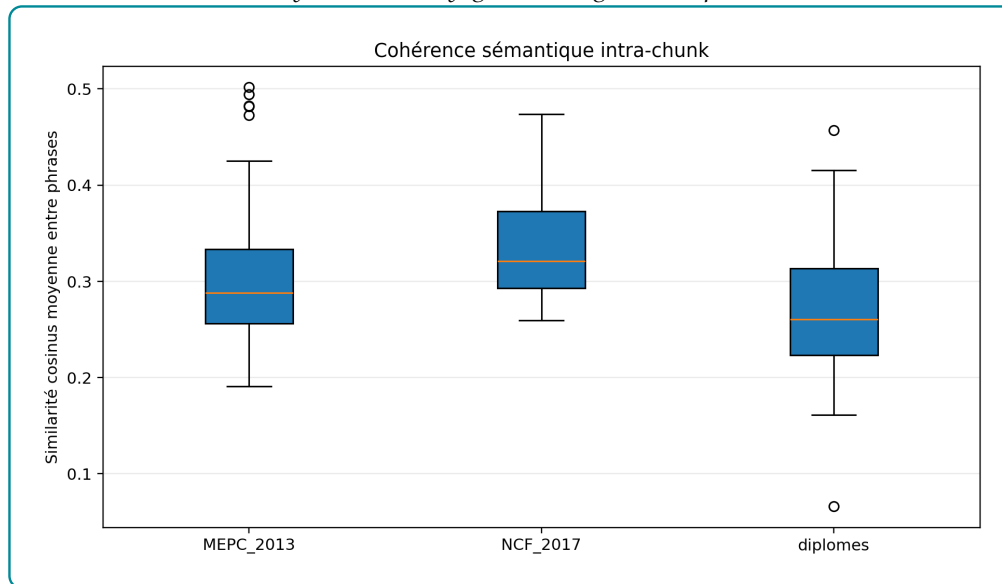
La couverture documentaire et la duplication des chunks sont détaillées en annexe (tableau A.7).

La couverture documentaire est calculée en rapportant le nombre de tokens indexés dans les chunks au nombre de tokens extraits des PDF. Elle doit être interprétée avec prudence, car les PDF contiennent aussi des pages de garde, tables, sommaires et contenus peu utiles au retrieval. Malgré cette limite, le résultat est un signal de vigilance : l'index actuel couvre 25,9 % de la NCF et 23,6 % du MEPC, contre 79,3 % pour le référentiel des diplômes. Le chunking structurel a donc privilégié les sections détectées comme exploitables, mais il ne doit pas être présenté comme une ingestion exhaustive des PDF réglementaires. A l'inverse, le taux de duplication quasi exacte est nul ; la base ne gaspille donc pas le stockage sur des chunks presque identiques.

La mesure de cohérence intra-chunk est reportée en annexe (tableau A.8).

Figure 3.7 – Distribution de la cohérence sémantique intra-chunk par source

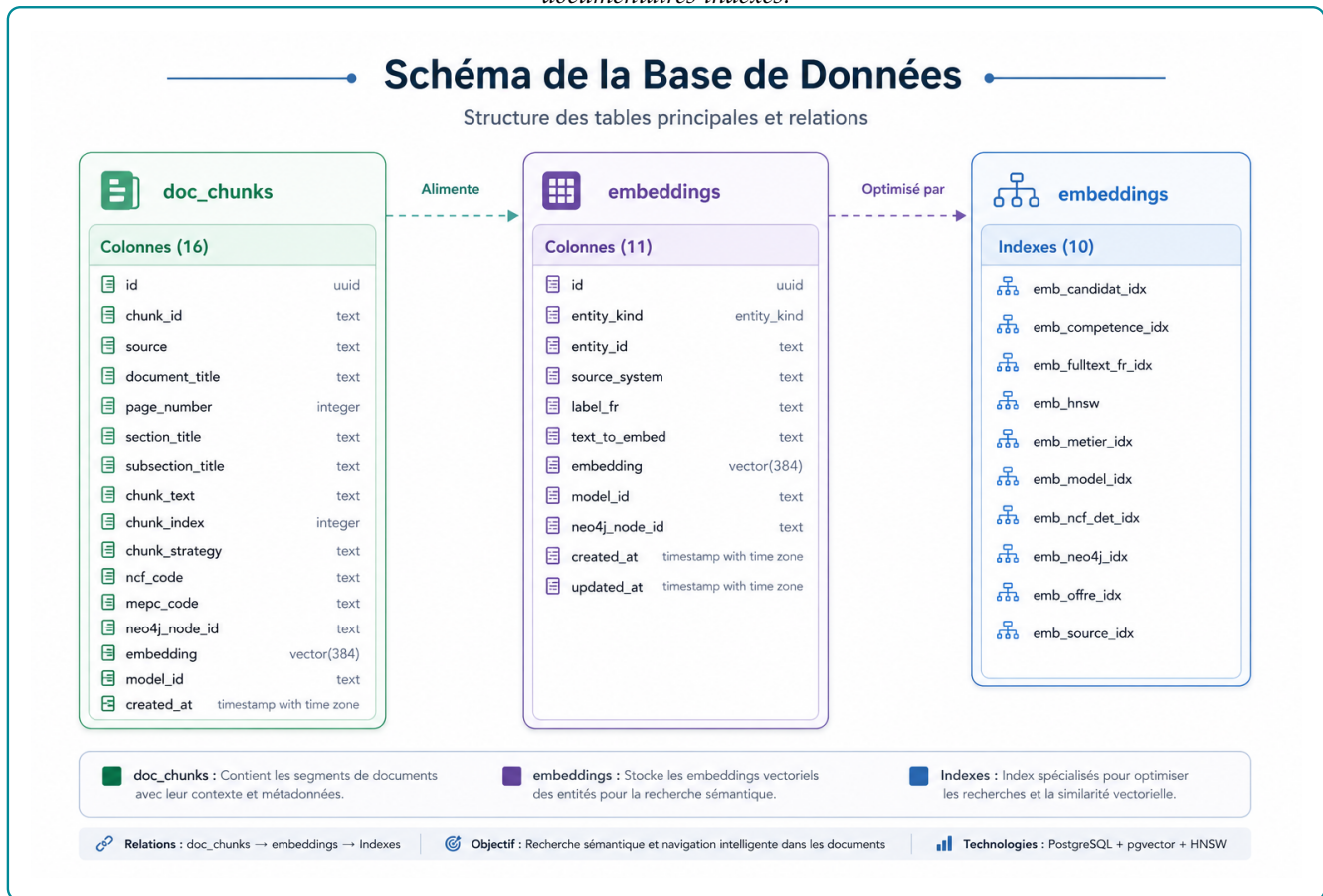
Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.



La cohérence sémantique moyenne vaut 0,303. Ce niveau est modéré : les chunks ne sont pas incohérents, mais ils ne forment pas non plus des blocs fortement homogènes. C'est cohérent avec des référentiels administratifs qui mêlent intitulés, codes, descriptions et listes. Pour un RAG de production, cette mesure suggère deux améliorations : fusionner certains fragments trop courts et tester une variante de chunking sémantique contrôlée par la cohérence intra-chunk. L'objectif ne serait pas de maximiser artificiellement la cohérence, mais de trouver le meilleur compromis entre contexte, taille et performance de retrieval.

Figure 3.8 – Schéma physique de la base pgvector : tables documentaires, embeddings et index de recherche

Source : Nos travaux, diagnostics PostgreSQL-pgvector, à partir des embeddings, des métadonnées et des fragments documentaires indexés.



La figure 3.8 montre que la mémoire vectorielle repose sur deux tables complémentaires. La table `doc_chunks` conserve les fragments documentaires et leurs métadonnées; la table `embeddings` stocke les représentations vectorielles des entités métier. Les index spécialisés accélèrent la recherche par type d'entité, par modèle, par identifiant Neo4j et par similarité vectorielle. Cette organisation permet au système de passer d'un fragment ou d'une entité retrouvée sémantiquement vers le graphe Neo4j correspondant, sans confondre stockage documentaire, retrieval vectoriel et raisonnement relationnel.

Les indicateurs techniques pgvector sont reportés en annexe (tableau A.9).

La mesure de latence est suffisamment basse pour servir un retrieval interactif : une médiane de 30,89 ms pour un top-10 HNSW laisse de la marge pour les étapes suivantes, notamment le reranking graphe et la construction du contexte. Le maximum observé, 160,40 ms, rappelle toutefois qu'il faut raisonner en distribution et non seulement en moyenne. Pour un Agentic RAG, la stabilité du p90 est plus importante que le meilleur cas.

Le détail des index pgvector et textuels est placé en annexe (tableau A.10).

Les documents référentiels ajoutent 142 chunks indexés : 43 pour les diplômes, 61 pour le MEPC 2013 et 38 pour la NCF 2017. Tous les chunks disposent d'un embedding et d'un identifiant Neo4j. Cette double indexation est importante pour le GraphRAG : pgvector retrouve le passage documentaire pertinent, puis Neo4j permet de le relier à une formation, un métier, un niveau ou un domaine.

Les recherches supportées sont de trois types. La première est la recherche sémantique top-K, fondée sur l'opérateur cosinus de pgvector. La deuxième est la recherche lexicale française, via les index GIN et to_tsvector. La troisième est la recherche hybride : présélection vectorielle, filtre métier par type d'entité, puis enrichissement par Neo4j.

Ces requêtes montrent que pgvector est prêt pour le retrieval de l'Agentic RAG : il peut produire un top-K sémantique, filtrer par nature d'entité, mobiliser un score lexical complémentaire et renvoyer des identifiants directement raccordables au graphe.

La conclusion technique est nette : pgvector remplit les conditions minimales d'une brique de retrieval. Il indexe toutes les entités utiles, conserve les liens Neo4j, dispose d'index vectoriels et lexicaux, et présente des latences compatibles avec une application interactive. Le point à surveiller pour l'évaluation n'est donc plus la disponibilité du stockage, mais la qualité du ranking produit par le retrieval et son amélioration par le graphe.

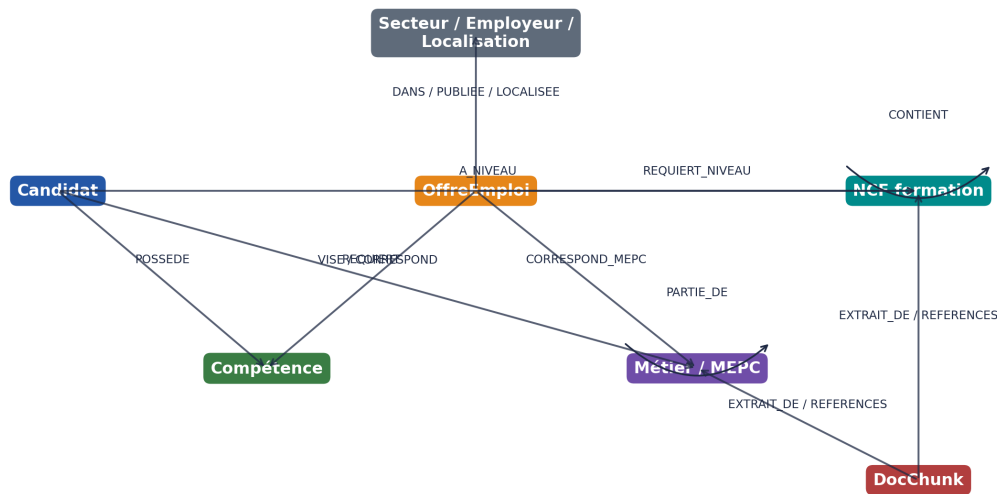
3.4 Neo4j : mémoire relationnelle du matching

3.4.1 Du voisinage vectoriel au chemin explicable

Figure 3.9 – Schéma logique du graphe de connaissances

Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.

Schéma logique du graphe de connaissances Neo4j

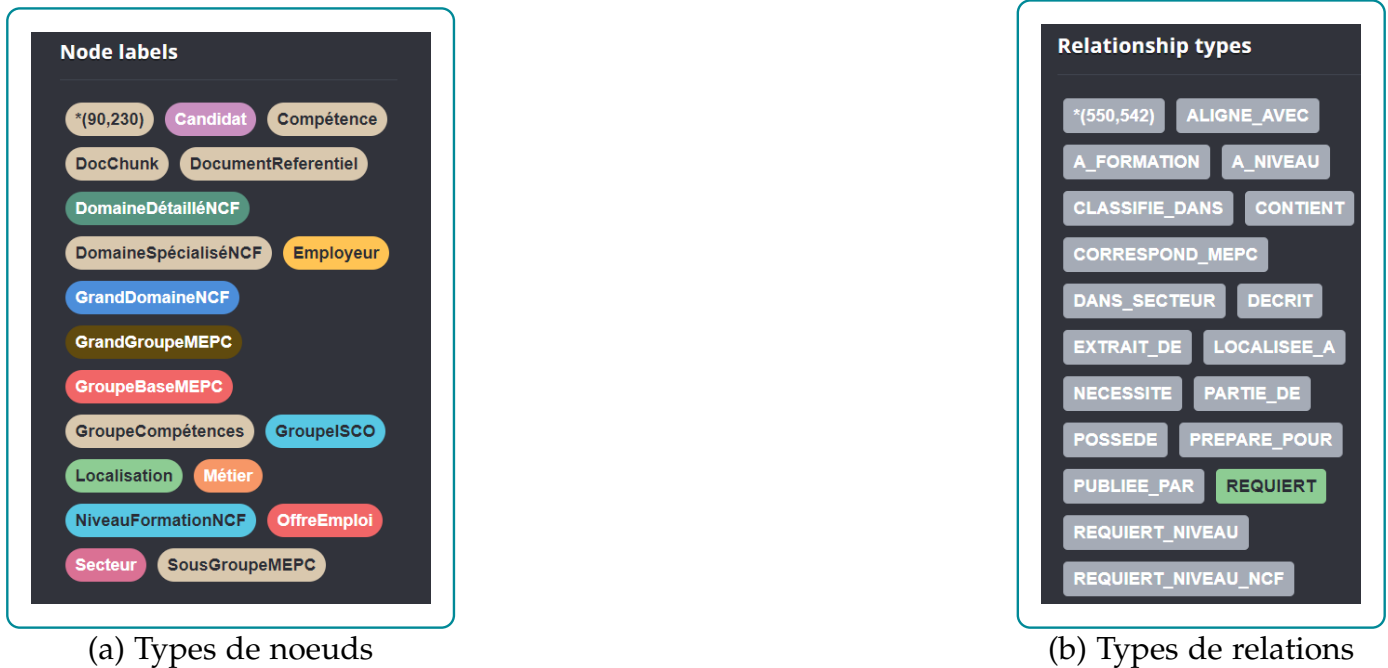


Le graphe Neo4j doit être lu comme la mémoire relationnelle du système. Il relie les candidats, offres, compétences, métiers, référentiels MEPC, niveaux et domaines NCF, secteurs, employeurs, localisations et chunks documentaires. Son rôle n'est pas de remplacer les embeddings : pgvector retrouve d'abord des objets proches dans l'espace sémantique, tandis que Neo4j vérifie ensuite comment ces objets sont reliés. Une offre peut donc être proche du profil dans pgvector, puis être acceptée, rétrogradée ou expliquée à partir de ses compétences requises, de son secteur, de son niveau NCF ou de son rattachement métier. La section ne présente donc pas un graphe au sens général ; elle montre comment le graphe construit sert à transformer un voisinage vectoriel en chemin de décision lisible.

La lecture correcte est la suivante : le graphe transforme une proximité textuelle en raisonnement contrôlable. Sans Neo4j, le système peut seulement dire qu'un profil et une offre se ressemblent dans un espace vectoriel. Avec Neo4j, il peut préciser par quelles relations cette proximité devient crédible : compétences possédées par le candidat, compétences exigées par l'offre, niveau de formation compatible ou non, secteur cohérent, et éventuel chemin vers une formation ou un métier de référence.

Figure 3.10 – Lecture conjointe des noeuds et relations Neo4j mobilisés par le matching

Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.



La figure 3.10 doit être lue en deux temps. Le panneau des noeuds indique les objets que le système manipule : Candidat, OffreEmploi, Compétence, Métier, référentiels MEPC, niveaux et domaines NCF, secteurs, employeurs, localisations et fragments documentaires. Le panneau des relations indique ce que le système sait vérifier entre ces objets : un candidat POSSEDE des compétences, une offre REQUIERT des compétences, un métier NECESSITE des compétences de référence, une offre est rattachée à un secteur, à un niveau, à un employeur et à une localisation. L'information utile n'est donc pas le noeud isolé, mais le couple « objet + relation ».

Cette représentation rend l'interprétation du graphe plus simple : Neo4j ne dit pas seulement qu'une offre, un candidat ou une compétence existe dans la base ; il montre comment ces éléments sont reliés. Une recommandation devient défendable lorsque le chemin est lisible : candidat → compétences possédées → compétences requises par l'offre, puis éventuellement métier, niveau NCF, secteur ou formation de référence. À l'inverse, si le chemin est absent ou porté seulement par une compétence trop générale, le graphe doit servir à rétrograder l'offre plutôt qu'à la justifier artificiellement.

Les noeuds représentent les objets métier manipulés par le système. Les noeuds Candidat et OffreEmploi portent les deux pôles du matching : un profil disponible et une opportunité à classer. Les noeuds Compétence jouent le rôle de pivot explicatif, car ils permettent de comparer ce qu'un candidat possède avec ce qu'une offre requiert. Les noeuds Métier, GroupeBaseMEPC, GroupeISCO, NiveauFormationNCF, DomaineDétailéNCF, Secteur et Localisation ajoutent les

dimensions de contrôle : métier de référence, famille professionnelle, niveau de qualification, domaine de formation, secteur d'activité et lieu. Les noeuds DocChunk et DocumentReferentiel relient le graphe aux sources documentaires utilisées par le GraphRAG.

Cette organisation évite une erreur fréquente : interpréter une offre et un candidat comme deux textes isolés. Dans le graphe, ils deviennent deux ensembles de relations. Un candidat n'est pas seulement une phrase de profil ; il est relié à des compétences, un niveau, une filière et un secteur. Une offre n'est pas seulement un intitulé ; elle est reliée à des compétences exigées, un employeur, une localisation, un secteur et un niveau requis. C'est cette représentation relationnelle qui rend l'explication possible.

Les relations décrivent les contraintes et les preuves utilisées par la recommandation. POSSEDE relie un candidat aux compétences déclarées ou inférées à partir de son profil. REQUIERT relie une offre aux compétences attendues. NECESSITE relie un métier aux compétences de référence du référentiel. A_NIVEAU, REQUIERT_NIVEAU et REQUIERT_NIVEAU_NCF structurent la compatibilité de formation. DANS_SECTEUR, LOCALISEE_A et PUBLIEE_PAR contextualisent l'offre. Enfin, les relations de référentiel, comme PARTIE_DE, CLASSIFIE_DANS ou EXTRAIT_DE, permettent de passer d'un résultat opérationnel à une explication institutionnelle.

Ces relations ne doivent pas être lues comme des vérités absolues. Dans l'implémentation, les liens REQUIERT et POSSEDE vers les compétences ESCO sont matérialisés par correspondance lexicale et portent des métadonnées d'audit comme la méthode de rapprochement et le niveau de confiance. Leur fonction est donc opérationnelle : fournir un signal exploitable et vérifiable pour le matching. Une interprétation rigoureuse doit conserver cette limite, car une compétence mal extraite ou trop générique peut améliorer artificiellement le score d'une offre.

3.4.2 Structure du graphe et risques de hubs

3.4.2.1 Taille réelle du graphe et couverture de la projection matching

Le tableau de bord statistique complet du graphe est renvoyé en annexe (figure A.2).

Le graphe complet contient 90 230 noeuds et 550 542 relations, répartis en 18 labels et 18 types de relations. Sa densité dirigée globale est très faible, environ $6,76 \times 10^{-5}$, ce qui est normal pour un graphe hétérogène de connaissances : toutes les entités n'ont pas vocation à être reliées entre elles. Cette faible densité ne signifie donc pas que le graphe est pauvre ; elle signifie qu'il encode des relations spécialisées plutôt qu'un réseau où tout serait connecté à tout.

Pour analyser la partie réellement mobilisée par le matching, une projection non orientée a été construite à partir des relations POSSEDE, REQUIERT, NECESSITE, CORRESPOND_MEPC, A_NIVEAU,

REQUIERT_NIVEAU, DANS_SECTEUR et LOCALISEE_A. Cette projection contient 72 516 noeuds et 499 080 arêtes ; elle forme une seule composante connexe. Ce résultat est important : les entités utiles au matching ne sont pas dispersées en sous-graphes indépendants, elles appartiennent à un même réseau de propagation candidat-offre-compétence. En pratique, cela permet au GraphRAG de construire des chemins courts entre un profil, une offre, une compétence, un métier ou un niveau de formation. La composition détaillée des noeuds et relations est reportée en annexe (figure A.3).

Les principaux labels sont Candidat avec 41 298 noeuds, GroupeCompétences avec 14 579 noeuds, OffreEmploi avec 13 957 noeuds, Compétence avec 13 939 noeuds et Métier avec 3 039 noeuds. Les cinq premiers labels concentrent 96,21 % des noeuds. Cette structure est cohérente avec l'objectif métier : le graphe est d'abord un graphe d'appariement, non un graphe encyclopédique. Le risque statistique est clair : si le score de recommandation n'est pas normalisé, les familles très représentées peuvent dominer la recherche au détriment de signaux plus spécifiques comme le métier, le niveau ou le secteur.

Cette structure impose toutefois une lecture prudente. Comme les relations de compétences dominant fortement, le score final dépend beaucoup de leur qualité. Si une offre n'a pas de relation REQUIERT, Neo4j ne peut pas calculer correctement la couverture de compétences. Si une compétence générique devient trop connectée, elle peut créer un hub peu discriminant. L'apport du graphe existe donc à condition de contrôler la complétude et la précision des relations.

3.4.2.2 Noeuds très connectés : signal utile ou bruit de popularité

La distribution complète des degrés est placée en annexe (figure A.4).

La distribution des degrés est fortement asymétrique. L'estimation exploratoire d'une loi de puissance sur la queue de distribution donne un exposant compris entre 1,40 et 2,51 selon le seuil $x_{\min}$ retenu. Ce n'est pas une preuve formelle de loi de puissance, mais un diagnostic utile : le graphe comporte bien des hubs. Dans un système de recommandation, ces hubs doivent être traités avec prudence. Une compétence très connectée n'est pas automatiquement discriminante ; elle peut simplement être générique, très fréquente ou issue d'un référentiel large.

Le tableau d'hétérogénéité des degrés par famille de noeuds est reporté en annexe (tableau A.11).

Les degrés moyens montrent une forte hétérogénéité. Les noeuds NiveauFormationNCF ont un degré moyen de 5 812,56, parce que neuf niveaux seulement absorbent une grande partie

des relations candidat–niveau et offre–niveau. Les GroupeBaseMEPC, les Secteur, les Métier, les Localisation et les Compétence jouent aussi un rôle de concentration. Cette hétérogénéité est une force pour l’explication, mais une faiblesse pour le scoring si elle n’est pas normalisée : le graphe peut favoriser des noeuds populaires au lieu de noeuds réellement informatifs.

3.4.2.3 Compétences pivots selon degré, PageRank et intermédiarité

Le graphique des centralités de compétences est renvoyé en annexe (figure A.5).

Les centralités donnent trois lectures complémentaires. La centralité de degré et le PageRank identifient surtout les compétences les plus connectées, par exemple « coordonner le transport », « respecter le planning de travail dans les transports », « gérer le transport de fonds » et « comptabilité ». L’intermédiarité approximée, calculée sur le sous-graphe des noeuds de plus fort degré, fait ressortir d’autres compétences : « utiliser différents moyens de communication », « gérer les budgets », « former le personnel », « normes de qualité » et « assurer la gestion de projets ». Ces dernières sont plus intéressantes pour la recommandation, car elles jouent un rôle de passerelle entre plusieurs familles de métiers.

Les exemples de compétences centrales selon les métriques de graphe sont placés en annexe (tableau A.12).

La distinction entre degré, PageRank et intermédiarité est importante. Un noeud peut être très populaire sans être une bonne passerelle de recommandation. Pour le *skill gap*, les compétences d’intermédiarité sont particulièrement utiles : elles indiquent des compétences transversales qui peuvent rapprocher plusieurs trajectoires professionnelles. Dans le moteur hybride, cela justifie de ne pas traiter toutes les compétences manquantes de la même manière.

3.4.2.4 Communautés Louvain et robustesse aux hubs

Les communautés Louvain et distances sont détaillées en annexe (figure A.6).

La détection de communautés a été réalisée avec Louvain sur un échantillon pondéré de la composante géante, enrichi par les noeuds de plus fort degré afin de conserver l’ossature du réseau. Elle identifie 100 communautés dans la projection analysée. Ce résultat ne doit pas être vendu comme une typologie définitive des métiers ; il sert à vérifier que le graphe contient des regroupements relationnels exploitables. La distance moyenne échantillonnée dans la composante géante vaut 4,10 sauts, avec une médiane de 4. En pratique, cela signifie qu’un candidat, une offre, une compétence ou un métier peut être relié au reste du réseau par quelques relations seulement, ce qui rend le GraphRAG opérationnel pour produire des chemins explicatifs courts.

Le test de robustesse après retrait des hubs est reporté en annexe (figure A.7).

Le test de robustesse retire successivement les 1 %, 5 % et 10 % de noeuds les plus centraux selon le degré, puis mesure la taille de la composante géante restante. Ce test est sévère, car il cible les hubs au lieu de supprimer des noeuds au hasard. Son intérêt est de mesurer la dépendance du graphe à quelques entités très connectées. Si la composante géante s'effondre fortement après retrait des hubs, cela signifie que le système dépend excessivement de compétences ou niveaux génériques. Dans ce cas, le reranking doit pénaliser les hubs trop peu discriminants et valoriser les compétences spécifiques.

3.4.2.5 Skill gap observé entre profils et offres

Les compétences moyennes par objet métier sont visualisées en annexe (figure A.8).

L'analyse métier confirme le rôle pivot des compétences. Une offre exige en moyenne 4,19 compétences, avec une médiane de 6. Un candidat possède en moyenne 5,36 compétences, avec une médiane de 6. Les métiers sont beaucoup plus riches : 41,48 compétences en moyenne et 37 en médiane, parce qu'ils agrègent des exigences de référentiel. Cette différence est saine : l'offre et le candidat portent un signal opérationnel court ; le métier porte un contexte plus large qui sert à l'enrichissement, à l'explication et à la montée en compétences.

La distribution détaillée du skill gap candidat-offre est renvoyée en annexe (figure A.9).

Le *skill gap* a été mesuré sur un échantillon de couples candidat-offre en comparant les compétences REQUIERT de l'offre aux compétences POSSEDE du candidat. Le déficit moyen est de 5,32 compétences, avec une médiane de 6 ; seulement 12,33 % des couples échantillonnés ont un déficit inférieur ou égal à trois compétences. Ce résultat est sévère, mais utile : le graphe ne dit pas que la majorité des candidats sont immédiatement compatibles avec une offre prise au hasard. Il montre au contraire que le matching doit sélectionner, filtrer et reranker. C'est précisément la raison d'être du moteur hybride : utiliser pgvector pour présélectionner des couples plausibles, puis Neo4j pour mesurer finement les compétences acquises, manquantes et passerelles.

L'interprétation opérationnelle est directe. Lorsque le déficit est faible, l'offre peut être considérée comme accessible ou proche. Lorsque le déficit est moyen, la recommandation doit être accompagnée d'un plan de montée en compétences. Lorsque le déficit est élevé, le système doit éviter de présenter l'offre comme immédiatement réaliste. Le graphe apporte donc une information que la similarité vectorielle seule ne fournit pas : il distingue la ressemblance textuelle d'une compatibilité professionnelle argumentée.

La conclusion à retenir est donc précise. Le graphe est riche parce qu'il relie les objets utiles au recrutement ; il est déséquilibré parce que certaines compétences et certains métiers

concentrent beaucoup de relations ; il est décisionnel parce qu’il modifie le classement produit par pgvector ; il reste imparfait parce que les liens de compétences dépendent de la qualité de l’extraction et du rattachement lexical. Cette nuance est importante : Neo4j améliore l’explicabilité et le reranking, mais il ne doit pas être présenté comme une garantie automatique de vérité métier.

3.5 Score hybride et reranking des recommandations

Le moteur hybride constitue le coeur décisionnel du système. Sans lui, l’application ne serait qu’un chatbot branché sur une recherche vectorielle. L’implémentation suit une logique en trois temps : pgvector présélectionne un ensemble d’offres candidates, Neo4j enrichit chaque couple candidat-offre avec des signaux relationnels, puis un score hybride réordonne les offres avant la génération. Le classement final n’est donc pas directement le score de similarité textuelle ; c’est un arbitrage pondéré entre le score pgvector et le score Neo4j, calibré ensuite par Optuna.

3.5.1 Formalisation du score hybride

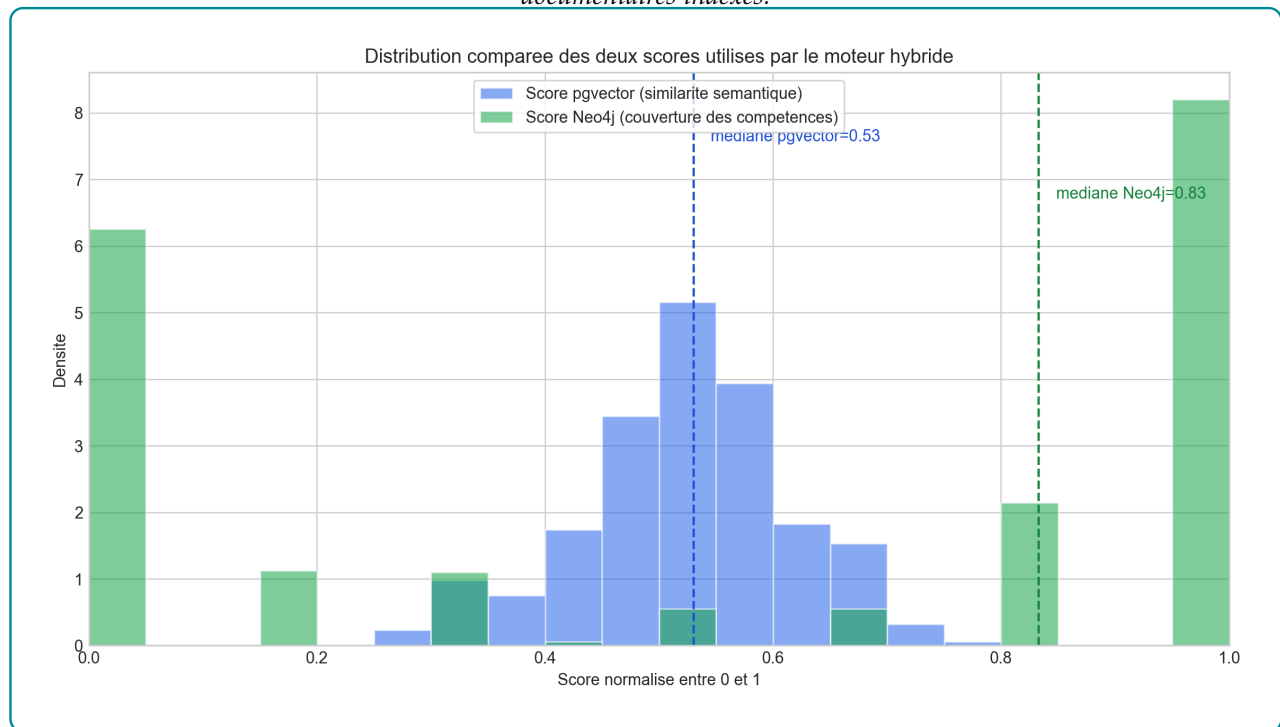
Le score hybride utilisé pour le reranking ne doit pas être présenté comme une addition de nombreux critères indépendants. Dans la version évaluée ici, il combine deux signaux seulement. Le premier vient de pgvector : il mesure la proximité sémantique entre le profil ou la requête et l’offre candidate. Le second vient de Neo4j : il mesure la compatibilité relationnelle, principalement à travers la couverture des compétences requises par l’offre et possédées par le candidat. La formulation retenue est donc :

$$S_{\text{hybride}} = \alpha S_{\text{pgvector}} + \beta S_{\text{Neo4j}}, \quad \alpha + \beta = 1$$

où S_{pgvector} correspond à la colonne `score_sem` et S_{Neo4j} à la colonne `taux_match`. Cette écriture est volontairement simple : pgvector répond à la question « les textes se ressemblent-ils ? », alors que Neo4j répond à la question « les compétences exigées sont-elles couvertes ? ». Les autres informations du graphe, comme le niveau NCF, le secteur ou la localisation, restent utiles pour l’explication et les verdicts, mais elles ne sont pas ajoutées comme poids autonomes dans cette calibration. Sinon, le score deviendrait difficile à défendre statistiquement et risquerait de compter plusieurs fois le même signal métier.

Figure 3.11 – Distribution comparée des scores pgvector et Neo4j utilisés dans le reranking

Source : Nos travaux, diagnostics PostgreSQL–pgvector, à partir des embeddings, des métadonnées et des fragments documentaires indexés.



La figure 3.11 montre que les deux scores n'ont pas le même comportement. Le score pgvector est continu et relativement concentré : sa moyenne vaut 0,5225 et sa médiane 0,5304 sur les 690 couples candidat–offre évalués. Le score Neo4j est plus discontinu : sa moyenne vaut 0,5606, sa médiane 0,8330, mais 31,30 % des couples ont un score nul. Cette différence est normale. pgvector renvoie presque toujours une proximité textuelle, même moyenne ; Neo4j est plus sévère, car une offre peut être textuellement proche tout en n'ayant aucune compétence requise couverte par le candidat.

Table 3.6 – Résultats de calibration des poids du score hybride

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Objectif optimisé	Jeu retenu	α pgvector	β Neo4j	NDCG@10
NDCG@10	Neo4j seul	0,0000	1,0000	0,9697
(NDCG@10 + Recall@10 + Precision@10) / 3	Neo4j seul	0,0000	1,0000	0,9697
(MRR@10 + Precision@10) / 2	Mixte Optuna	0,6615	0,3385	0,9674

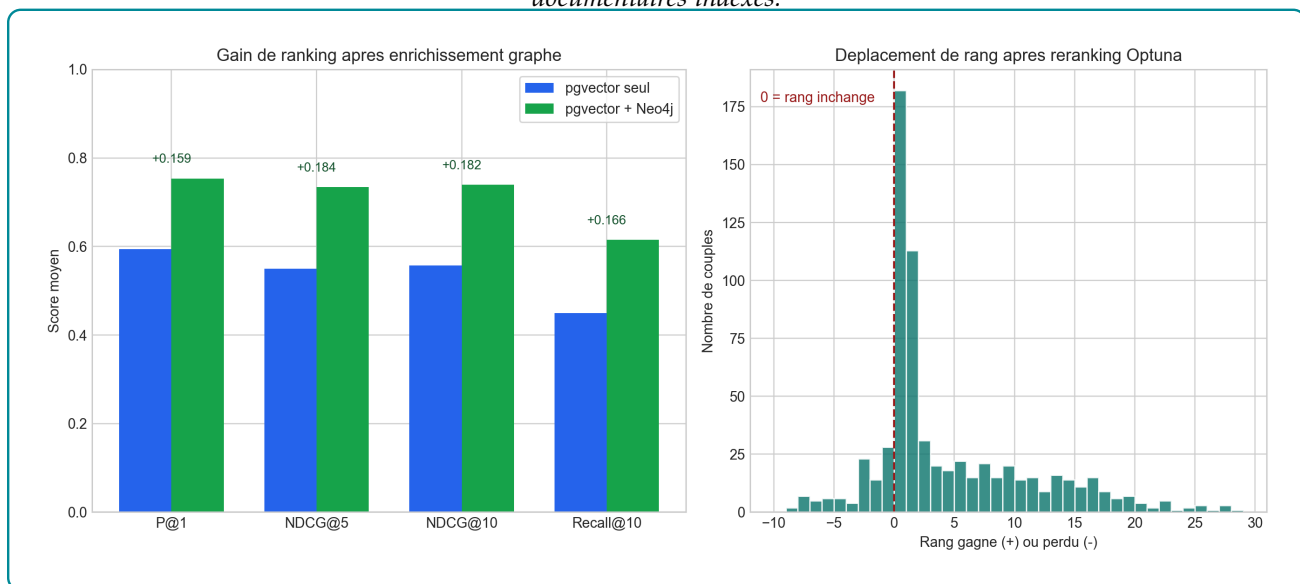
La calibration Optuna a été réalisée avec un TPESampler, 500 essais et une graine fixée à 42. Lorsque l'objectif est de maximiser le NDCG@10, le meilleur jeu retenu donne tout le poids au score Neo4j : $\alpha = 0$ et $\beta = 1$. Ce résultat ne signifie pas que pgvector est inutile. Il signifie que, sur le sous-ensemble déjà récupéré par pgvector, le reclassement est mieux ordonné lorsque la compatibilité graphe domine. La bonne lecture est donc séquentielle : pgvector sert à constituer le vivier de candidats plausibles ; Neo4j sert ensuite à départager ces candidats par couverture de compétences. Pour un objectif davantage centré sur le rang exact de la première bonne offre, le critère $(\text{MRR}@10 + \text{Precision}@10)/2$ retient au contraire un mélange, avec 0,6615 pour pgvector et 0,3385 pour Neo4j. Le choix final dépend donc du besoin applicatif : maximiser la qualité du top 10 ou privilégier la première recommandation.

3.5.2 Effet du reranking hybride

L'effet du reranking se mesure à deux niveaux. Le premier niveau regarde les rangs des couples candidat-offre déjà présents dans le vivier récupéré par pgvector. Le second niveau regarde les métriques de ranking lorsque l'on compare le système pgvector_only au système pgvector_plus_graph. Cette distinction est importante : Neo4j ne remplace pas le retrieval vectoriel, il intervient après lui pour réordonner les offres candidates.

Figure 3.12 – Effet du reranking Neo4j sur les métriques et les déplacements de rang

Source : Nos travaux, diagnostics PostgreSQL–pgvector, à partir des embeddings, des métadonnées et des fragments documentaires indexés.



La figure 3.12 montre un effet net du graphe sur l'ordre des résultats. Sur les 690 couples évalués, 59,86 % gagnent des positions après reranking, 13,77 % perdent des positions et 26,38 % restent au même rang. Le gain moyen est de 4,01 rangs, avec une médiane de 1 rang et un

maximum de 28 rangs. Cette moyenne ne doit pas être interprétée comme une amélioration automatique de chaque offre : elle indique que le score Neo4j change réellement l'ordre produit par pgvector, en remontant surtout les offres dont les compétences requises sont mieux couvertes.

Le résumé détaillé des déplacements de rang après reranking est placé en annexe (tableau A.13).

L'ablation confirme ensuite que ces déplacements ne sont pas seulement cosmétiques. Sur 69 requêtes évaluées, l'ajout du graphe augmente la précision@1 de 0,5942 à 0,7536, le NDCG@5 de 0,5495 à 0,7339, le NDCG@10 de 0,5575 à 0,7392 et le recall@10 de 0,4491 à 0,6147. Le hit rate@10 reste identique à 0,9420 : cela signifie que les éléments pertinents sont souvent déjà présents dans le vivier pgvector, mais que Neo4j les ordonne mieux. C'est exactement le rôle attendu du reranking : ne pas refaire la recherche depuis zéro, mais transformer une liste plausible en liste mieux priorisée.

La conclusion doit rester stricte. Le reranking hybride améliore le classement dans ce protocole, mais la pertinence utilisée est un proxy construit à partir de métadonnées normalisées et non une annotation humaine indépendante. Le résultat est donc une preuve opérationnelle de cohérence interne, pas encore une preuve définitive de satisfaction recruteur ou candidat. Pour défendre le système en production, il faudra compléter cette évaluation par des jugements humains ou des retours d'usage.

3.6 Orchestration agentique, critic et exposition du système

Évaluation de la performance du système

4.1 Cadre d'évaluation et protocole expérimental

4.1.1 Questions évaluées et logique de séparation

Ce chapitre évalue empiriquement le système implémenté au chapitre précédent. L'objectif n'est pas seulement de produire des scores globaux, mais de répondre à une question centrale du mémoire : l'ajout d'un graphe de connaissances Neo4j apporte-t-il une amélioration mesurable par rapport à une recherche vectorielle fondée uniquement sur pgvector ? Cette question est décisive, car le graphe ajoute une complexité technique réelle : modélisation des nœuds et relations, chargement Neo4j, requêtes Cypher, enrichissement des résultats, maintenance d'une base supplémentaire. Son intérêt doit donc être justifié par un gain observable en qualité de classement, en explicabilité ou en contrôle métier.

L'évaluation a été conduite en cinq niveaux. Le premier niveau porte sur le modèle d'embeddings, c'est-à-dire la capacité du SentenceTransformer fine-tuné à retrouver les paires offre-profil pertinentes dans un corpus de test. Le deuxième niveau porte sur l'ablation du moteur de recommandation : une configuration *pgvector seul* est comparée à une configuration *pgvector + graphe*. Le troisième niveau vérifie fonctionnellement le graphe : recherche métier-compétences, compétence-offres, skill gap et compatibilité NCF. Le quatrième niveau calibre les pondérations du score hybride avec Optuna. Le cinquième niveau évalue le contrôle de génération du workflow Agentic GraphRAG, en particulier le seuil `accept/revise` du critic.

Les résultats présentés dans ce chapitre proviennent d'artefacts reproductibles du pipeline local. Le benchmark des embeddings est produit par le module d'évaluation SentenceTransformer, l'ablation compare directement pgvector et Neo4j sur les mêmes requêtes, la calibration des pondérations est conduite avec Optuna, et le seuil du critic est estimé par un protocole contrastif. Cette traçabilité est importante : les nombres présentés ici ne sont pas des valeurs théoriques, mais des sorties calculées à partir des paires candidat-offre, des contextes récupérés et des réponses générées par le système.

4.1.2 Données, métriques et limites du protocole

Le système dispose de 13 957 offres normalisées et de 41 298 profils candidats. Le benchmark d’embeddings utilise le jeu de test issu de la construction des paires contrastives offre–profil, soit 473 requêtes de test. Chaque requête correspond à une description structurée d’offre ou de profil, et le corpus contient les textes enrichis utilisés pour l’apprentissage contrastif. Les métriques calculées sont celles de la recherche d’information : Precision@K, Recall@K, NDCG@K et MRR@K.

Pour l’ablation *pgvector* contre *pgvector + graphe*, le protocole est différent. Un échantillon de 80 candidats est tiré aléatoirement avec la graine 42 ; 69 candidats ont été effectivement évalués après contrôle d’exécution. Pour chaque candidat, *pgvector* retourne un pool initial de 30 offres candidates. Deux classements sont alors comparés :

- **pgvector seul** : l’ordre des offres est conservé tel qu’il est retourné par la recherche sémantique et lexicale dans PostgreSQL/*pgvector* ;
- **pgvector + graphe** : le même pool initial est enrichi par Neo4j, puis rerangé par le score hybride combinant similarité sémantique, compatibilité de compétences, compatibilité de niveau NCF et alignement sectoriel.

Cette comparaison est volontairement réalisée sans génération LLM. Le modèle génératif OpenRouter intervient dans le chatbot pour formuler la réponse finale, mais il n’est pas mobilisé dans l’ablation. Cette séparation évite de confondre deux phénomènes : la qualité du classement des offres et la qualité rédactionnelle de la réponse.

La pertinence utilisée pour calculer les métriques d’ablation est un proxy construit à partir des métadonnées normalisées : chevauchement lexical entre le profil et l’offre, proximité sectorielle et compatibilité de niveau NCF. Cette approximation est acceptable pour comparer deux variantes internes sur le même échantillon, mais elle ne remplace pas une annotation humaine. Les résultats doivent donc être interprétés comme une mesure comparative de l’effet du graphe, non comme une mesure définitive de satisfaction utilisateur.

4.2 Retrieval sémantique et apport mesuré du graphe

4.2.1 Performance du modèle d’embeddings fine-tuné

Le premier résultat concerne l’intérêt du fine-tuning du SentenceTransformer. Le tableau 4.1 présente les performances du modèle fine-tuné par rapport à plusieurs modèles généralistes ou multilingues. Le modèle spécialisé domine nettement les alternatives sur les mé-

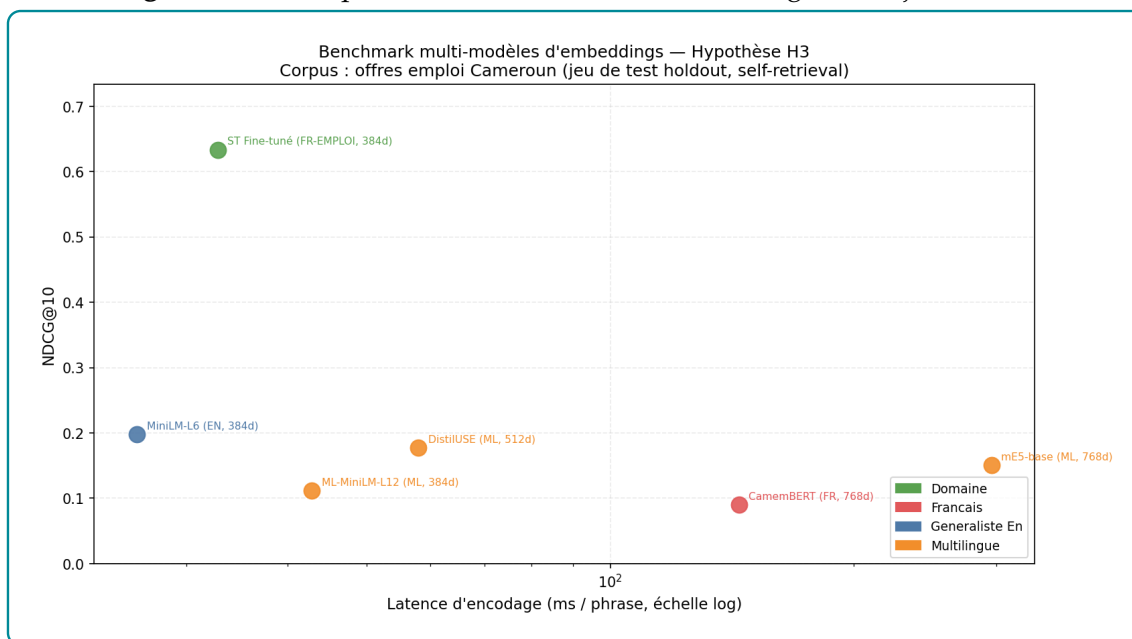
triques de retrieval.

Table 4.1 – Benchmark des modèles d’embeddings sur 473 requêtes de test

Modèle	NDCG@10	MRR@10	Recall@10	Latence ms/phrased
ST fine-tuné emploi-compétences	0.6336	0.5653	0.8520	32.77
MiniLM-L6 généraliste	0.1980	0.1603	0.3192	26.03
DistilUSE multilingue	0.1773	0.1434	0.2896	57.96
mE5-base multilingue	0.1505	0.1274	0.2262	295.76
ML-MiniLM-L12	0.1115	0.0922	0.1734	42.82
CamemBERT sentence	0.0904	0.0771	0.1332	144.15

Le modèle fine-tuné atteint un Recall@10 de 0.8520, contre 0.3192 pour MiniLM-L6 généraliste. L’écart est substantiel : le modèle spécialisé retrouve beaucoup plus souvent le document pertinent dans les dix premiers résultats. Le NDCG@10 passe également de 0.1980 à 0.6336, ce qui indique non seulement une meilleure récupération, mais aussi un meilleur ordre des résultats pertinents. La latence moyenne du modèle spécialisé reste contenue à 32.77 ms par phrase, ce qui est compatible avec une indexation ou une recherche interactive locale.

Figure 4.1 – Comparaison des modèles d’embeddings sur le jeu de test



Ce résultat valide le choix d’un encodeur adapté au domaine. Il montre aussi pourquoi une base vectorielle généraliste ne suffit pas : le vocabulaire emploi-compétences contient des formulations longues, des intitulés hybrides, des niveaux de formation et des compétences

implicites. Le fine-tuning contrastif rapproche ces formulations dans l'espace vectoriel et améliore directement le retrieval.

4.2.2 Ablation pgvector seul contre pgvector enrichi par Neo4j

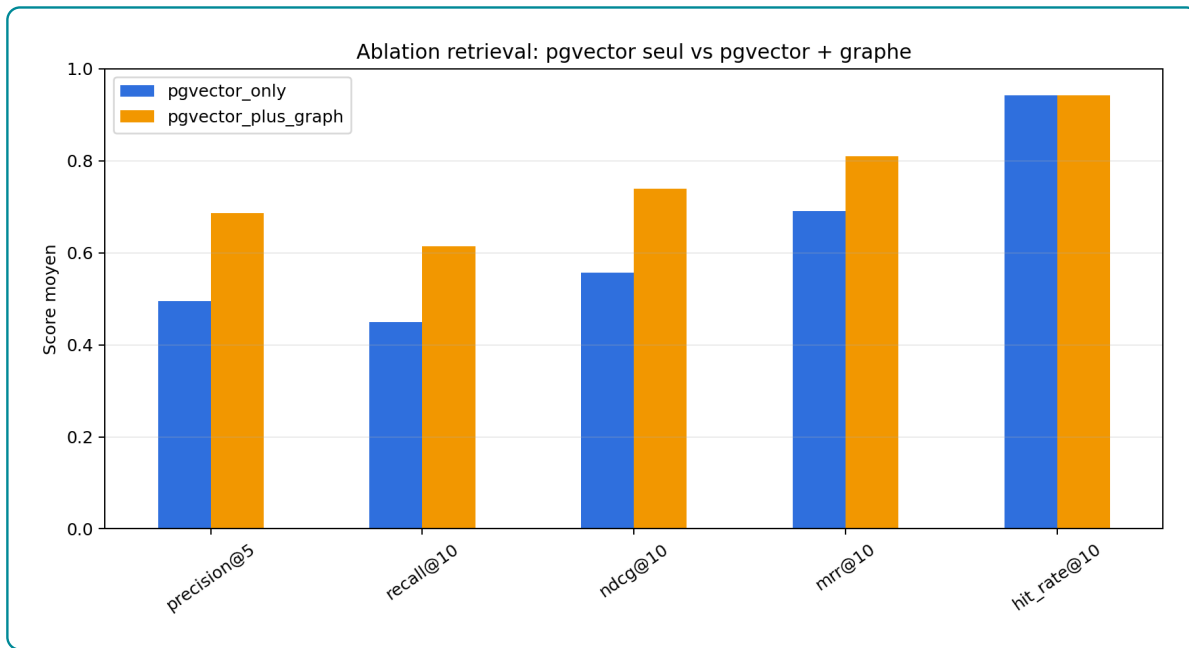
Le coeur de l'évaluation concerne l'apport de Neo4j. Le script d'ablation a demandé l'évaluation de 80 candidats. Sur cet échantillon, 69 requêtes ont été évaluées avec succès et 11 ont été écartées par les contrôles d'exécution. Les résultats doivent donc être lus comme une comparaison sur les requêtes valides, non comme une mesure exhaustive de tout le corpus. L'intérêt de cette ablation est néanmoins fort : le même pool initial de 30 offres est utilisé dans les deux configurations, ce qui isole l'effet du reranking graphe.

Table 4.2 – Comparaison retrieval : pgvector seul versus pgvector + graphe

Configuration	Precision@5	Recall@10	NDCG@10	MRR@10	HitRate@10
pgvector seul	0.4957	0.4491	0.5575	0.6908	0.9420
pgvector + graphe	0.6870	0.6147	0.7392	0.8093	0.9420
Différence	+0.1913	+0.1656	+0.1817	+0.1185	0.0000

Les résultats sont nettement favorables au graphe après correction de l'alignement des identifiants et du calcul exact du `taux_match`. L'ajout de Neo4j améliore Precision@5, Recall@10, NDCG@10 et MRR@10. Autrement dit, lorsque l'on considère les cinq ou dix premiers résultats, l'enrichissement graphe remonte davantage d'offres jugées pertinentes par le proxy métier et les place dans un meilleur ordre. Le Recall@10 progresse de 0.4491 à 0.6147, soit un gain de 0.1656. Le NDCG@10 augmente de 0.5575 à 0.7392, ce qui traduit une amélioration substantielle de l'organisation du top 10.

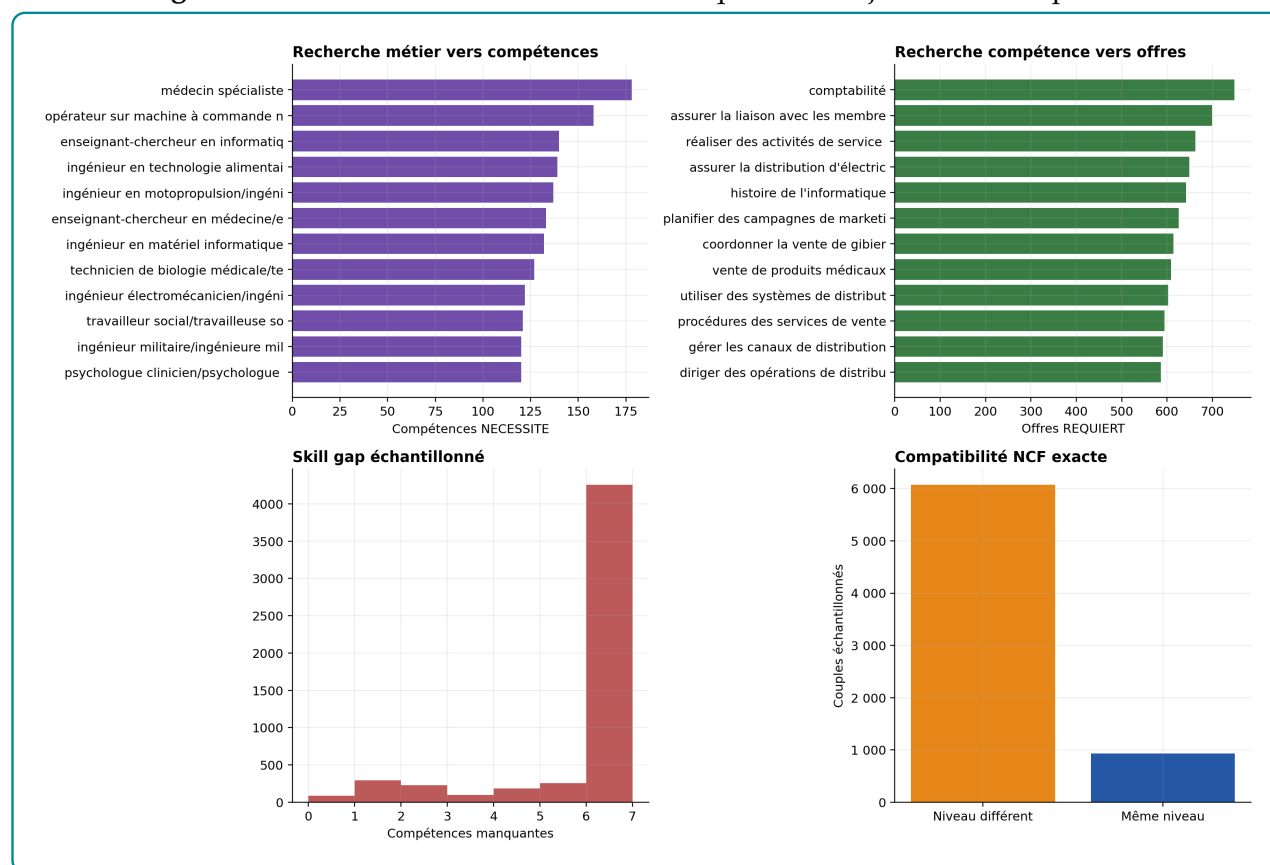
La Precision@1 progresse également, de 0.5942 à 0.7536. Ce point est important, car il corrige l'ancien diagnostic : le graphe ne sert pas seulement à élargir la liste ; lorsqu'il est correctement raccordé aux relations `POSSEDE` et `REQUIERT`, il améliore aussi la première recommandation. Le HitRate@10 reste stable à 0.9420, ce qui signifie que les deux systèmes retrouvent au moins une offre pertinente dans le top 10 dans une proportion comparable, mais que Neo4j classe mieux ces offres.

Figure 4.2 – Effet de l’enrichissement graphe sur les métriques de retrieval

La figure 4.2 confirme cette lecture. La configuration *pgvector* + *graphe* domine la configuration vectorielle seule sur les métriques principales de classement. Cela ne signifie pas que *pgvector* devient secondaire : *pgvector* reste le premier étage, chargé de produire rapidement le pool candidat. Le résultat indique plutôt que, dans ce pool, le graphe apporte une information métier suffisamment discriminante pour améliorer le reranking.

4.3 Lecture fonctionnelle du graphe Neo4j et du GraphRAG

L’ablation précédente mesure l’effet du graphe sur le classement. Elle ne suffit pas à vérifier que Neo4j joue correctement son rôle fonctionnel. Une évaluation complémentaire a donc été réalisée avec un module dédié au diagnostic fonctionnel du graphe. Elle teste cinq scénarios directement liés aux usages du système : recherche métier vers compétences, recherche compétence vers offres, calcul de *skill gap*, compatibilité NCF et comparaison RAG contre GraphRAG.

Figure 4.3 – Évaluation fonctionnelle des requêtes Neo4j utiles au GraphRAG

4.3.1 Recherche métier vers compétences

Le premier test vérifie que le graphe peut partir d'un métier et récupérer les compétences associées par la relation *NECESSITE*. Ce scénario correspond à une question de type : « quelles compétences faut-il pour exercer ce métier ? ». Les métiers les plus riches du graphe contiennent un nombre important de compétences : par exemple « médecin spécialiste » est relié à 178 compétences, « opérateur sur machine à commande numérique » à 158, et « enseignant-chercheur en informatique » à 140. Sur les 25 métiers les plus riches, la moyenne atteint 124,4 compétences.

Cette richesse est utile pour l'orientation et la montée en compétences, mais elle impose une précaution : le métier porte un contexte large, pas une liste minimale d'exigences pour une offre précise. Dans le GraphRAG, les compétences métier doivent donc enrichir l'explication, tandis que les compétences *REQUIERT* d'une offre doivent rester prioritaires pour le matching opérationnel.

4.3.2 Recherche compétence vers offres

Le deuxième test part d'une compétence et récupère les offres qui la requièrent. C'est le scénario inverse : « quelles offres demandent cette compétence ? ». La compétence « comptabilité » est reliée à 748 offres, « réaliser des activités de service après-vente » à 662 offres et « planifier des campagnes de marketing sur les médias sociaux » à 626 offres. Sur les 25 compétences les plus présentes dans les offres, la moyenne atteint 590,16 offres.

Ce résultat prouve que Neo4j peut servir de moteur de recherche relationnelle. Il montre aussi une limite importante : certaines compétences très fréquentes sont peu discriminantes ou relèvent davantage d'un domaine large que d'une compétence opérationnelle fine. Dans un système de recommandation, ces hubs ne doivent pas être récompensés mécaniquement. Ils doivent être pondérés avec le score sémantique, le niveau NCF, le secteur et les compétences plus spécifiques.

4.3.3 Skill gap

Le *skill gap* est calculé en comparant les compétences **REQUIERT** par l'offre et les compétences **POSSEDE** par le candidat. Sur l'échantillon fonctionnel, le déficit moyen est de 5,30 compétences et la médiane vaut 6. Ce résultat est cohérent avec l'objectif du système : il ne s'agit pas de supposer que tous les candidats sont proches de toutes les offres, mais d'identifier les couples où l'écart est acceptable et de transformer les autres cas en trajectoire de montée en compétences.

Cette mesure est plus utile qu'un score opaque. Elle permet d'expliquer pourquoi une offre est classée : compétences déjà couvertes, compétences manquantes, et éventuellement compétences passerelles. Elle permet aussi de produire des verdicts différenciés : compatible, montée en compétence, vivier ou hors cible.

4.3.4 Compatibilité NCF

Le test de compatibilité NCF vérifie si le niveau du candidat correspond au niveau requis par l'offre. Dans l'échantillon de couples candidat-offre évalué, 13,29 % des couples partagent exactement le même niveau. Ce taux relativement faible confirme que le niveau de formation est un filtre fort. Une recommandation purement textuelle peut rapprocher un candidat et une offre, mais elle ne garantit pas la cohérence institutionnelle du niveau attendu.

La compatibilité NCF ne doit cependant pas être utilisée comme un couperet mécanique. Un écart d'un niveau peut être acceptable selon l'expérience, les compétences et la nature de l'offre. C'est pourquoi le moteur hybride l'utilise comme composante pondérée plutôt que

comme filtre absolu.

4.3.5 Comparaison RAG vs GraphRAG

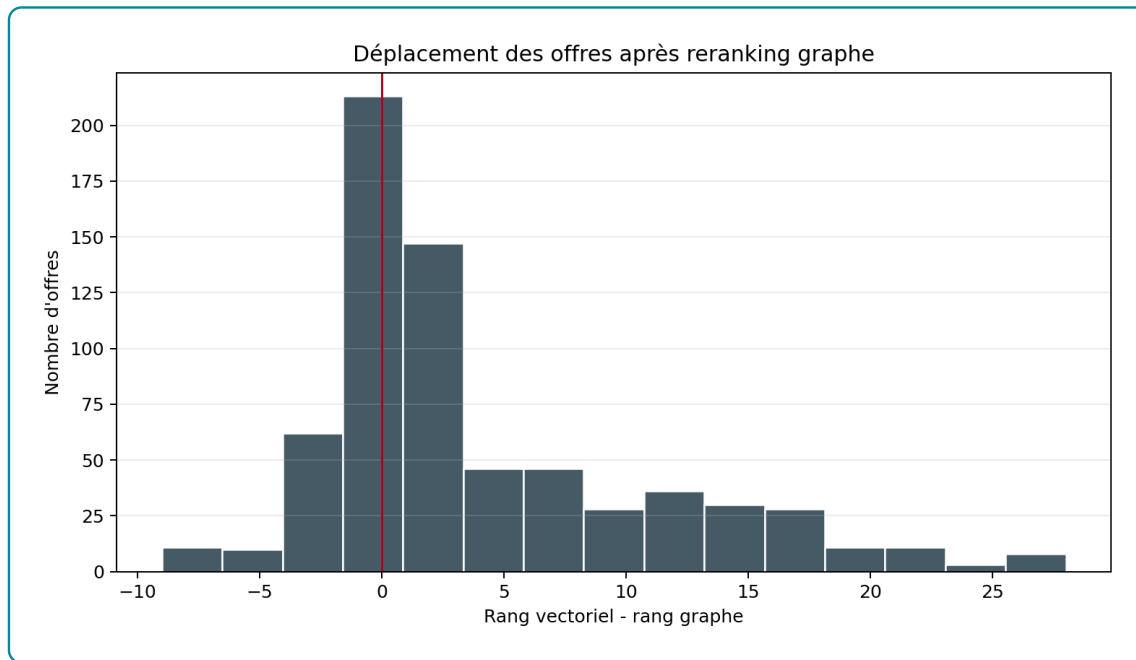
La comparaison RAG contre GraphRAG est matérialisée par l’ablation `pgvector` seul contre `pgvector` + `graphe`. Dans ce protocole, le RAG vectoriel constitue un pool initial de 30 offres par candidat. Le GraphRAG conserve le même pool, mais ajoute les relations Neo4j pour recalculer le score et reranger les offres. La différence observée ne vient donc pas d’un corpus différent, mais de l’information relationnelle ajoutée au classement.

Les résultats sont nets : le NDCG@10 passe de 0,5575 à 0,7392, le Recall@10 de 0,4491 à 0,6147, et le MRR@10 de 0,6908 à 0,8093. Le GraphRAG améliore donc l’ordre des recommandations et pas seulement leur explication. La conclusion correcte est précise : `pgvector` est efficace comme outil de rappel initial, tandis que Neo4j apporte un signal de décision plus discriminant lorsque les relations `POSSEDE`, `REQUIERT`, `A_NIVEAU` et `REQUIERT_NIVEAU` sont disponibles.

4.4 Reranking hybride et calibration des poids

4.4.1 Déplacements de rang après reranking par le graphe

Le reranking graphe déplace les offres dans le classement initial. Une offre qui était par exemple au rang 8 dans `pgvector` peut être remontée au rang 3 si elle présente une meilleure compatibilité NCF, un meilleur alignement sectoriel ou un meilleur score de compétences. Inversement, une offre très proche sémantiquement peut être rétrogradée si le niveau requis ou les signaux métier sont moins compatibles.

Figure 4.4 – Distribution des déplacements de rang après reranking par le graphe

Cette redistribution a deux effets. Le premier est positif : le système ne se limite plus à une proximité textuelle. Il peut corriger des cas où une offre ressemble lexicalement au profil mais ne correspond pas au niveau de formation ou au secteur recherché. Le second est plus problématique : si les relations du graphe sont incomplètes, le score structurel peut pénaliser de bonnes offres ou favoriser des offres seulement parce que leurs métadonnées sont mieux renseignées. Le graphe ajoute donc de l'information, mais il ajoute aussi une source d'erreur potentielle.

Ce résultat appelle une conclusion nuancée. Le graphe n'est pas une garantie automatique d'amélioration. Il améliore le système si les relations sont correctes, si les poids du score hybride sont bien calibrés et si l'objectif est de produire une liste explicable plutôt qu'un unique meilleur résultat. Dans le système actuel, l'apport est visible sur Recall@10 et NDCG@10, mais il reste insuffisamment stabilisé sur le rang 1.

4.4.2 Pondérations optimales du score hybride avec Optuna

Afin de ne pas conserver des pondérations arbitraires dans le moteur hybride, une étape d'optimisation a été intégrée au protocole d'évaluation. Elle exploite les sorties détaillées de l'ablation, optimise les pondérations du score hybride avec Optuna et compare trois fonctions objectif : NDCG@10, moyenne NDCG@10–Recall@10–Precision@10, et moyenne MRR@10–Precision@10. Les résultats retenus sont ensuite synthétisés sous forme de tableaux et de figures pour comparer les pondérations obtenues selon l'objectif optimisé.

L'expérience a été conduite sur 690 couples candidat-offre correspondant à 69 candidats évalués. La pertinence graduée utilisée est `proxy_relevance`, c'est-à-dire le proxy métier construit à partir des métadonnées normalisées. Cette décision est préférable à l'utilisation directe des verdicts du moteur, car elle limite la circularité : on ne calibre pas les poids uniquement sur une étiquette déjà produite par le score hybride. Cependant, cette pertinence reste un label faible ; elle ne remplace pas une annotation humaine.

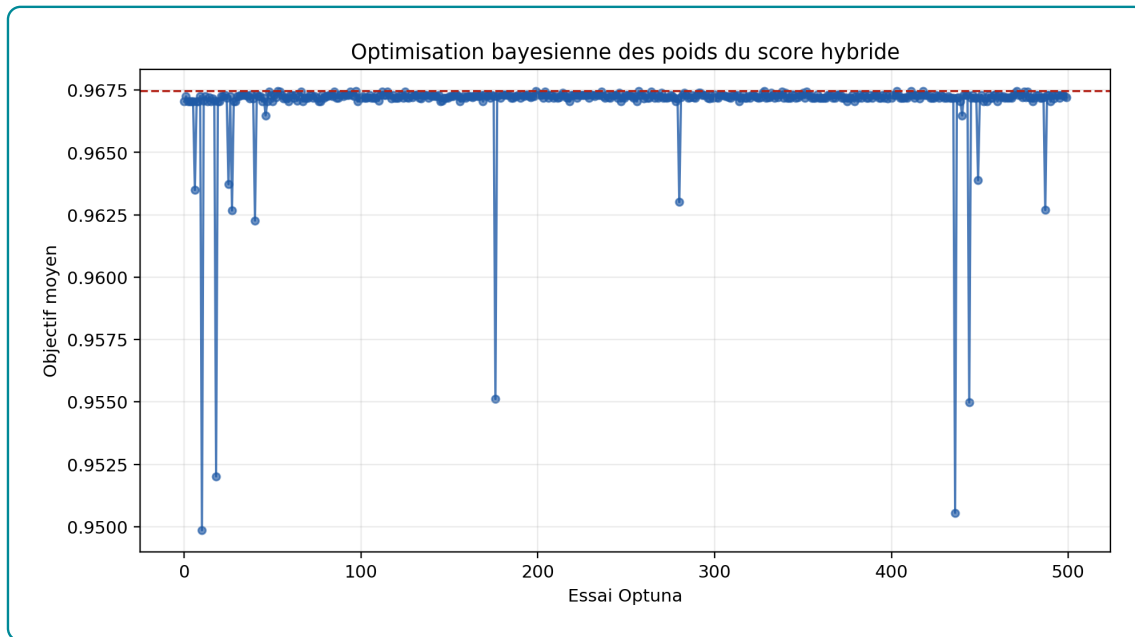
Table 4.3 – Pondérations retenues selon la fonction objectif du score hybride

Fonction objectif	w_{sem}	w_{Neo4j}	NDCG@10	Precision@10	Recall@10	MRR@10
NDCG@10	0.0000	1.0000	0.9697	0.5000	1.0000	0.8109
NDCG@10 + Recall@10 + Precision@10	0.0000	1.0000	0.9697	0.5000	1.0000	0.8109
MRR@10 + Precision@10	0.6615	0.3385	0.9674	0.5000	1.0000	0.8183

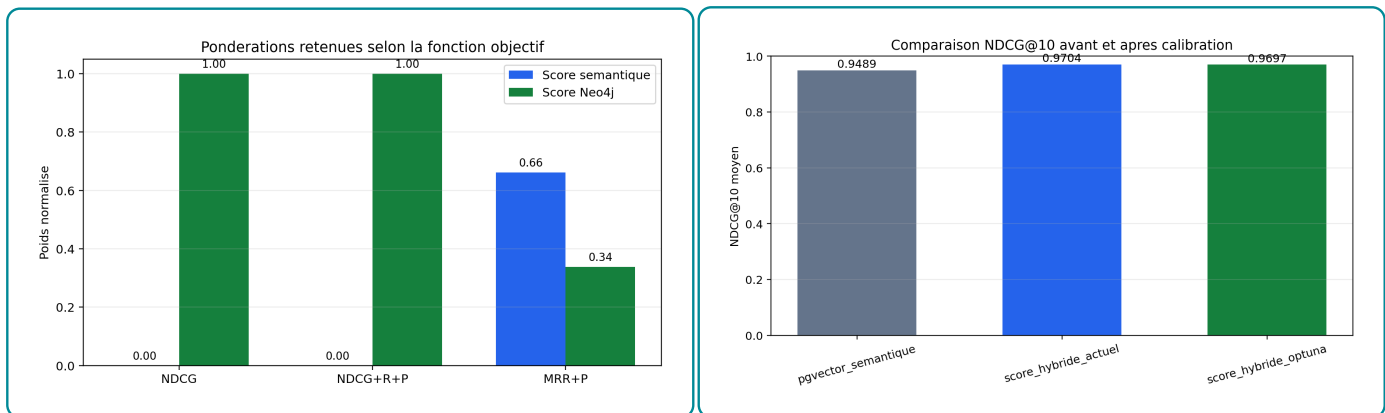
Le résultat est sévère pour une lecture naïve du score hybride. Lorsque la fonction objectif privilégie le classement global du top 10, Optuna retient le score Neo4j pur : $w_{sem} = 0$ et $w_{Neo4j} = 1$. Autrement dit, une fois que pgvector a construit le pool initial des offres candidates, le meilleur reranking selon NDCG@10 est obtenu en donnant tout le poids au taux de couverture des compétences calculé dans Neo4j. Ce résultat ne signifie pas que pgvector est inutile. Il signifie que pgvector joue surtout le rôle de retriever rapide, tandis que Neo4j devient le signal de décision le plus discriminant à l'intérieur du pool.

La troisième fonction objectif nuance cette conclusion. Lorsque le critère donne plus de poids au rang de la première offre pertinente, à travers MRR@10, le meilleur compromis devient hybride : $w_{sem} = 0,6615$ et $w_{Neo4j} = 0,3385$. Ce mélange réduit très légèrement le NDCG@10, de 0.9697 à 0.9674, mais améliore le MRR@10, de 0.8109 à 0.8183. Le choix des coefficients dépend donc de l'usage. Pour une liste de recommandations explicables, le signal graphe domine. Pour pousser rapidement une première offre très pertinente, une part sémantique plus forte reste utile.

Un point statistique doit être souligné : Precision@10 et Recall@10 restent constants dans cette expérience, respectivement 0.5000 et 1.0000 pour les trois solutions retenues. Ces deux métriques ne discriminent donc pas les pondérations sur cet échantillon. La différence réelle se joue sur l'ordre des offres, mesuré par NDCG@10 et MRR@10. Cette stabilité est informative mais limitée : elle suggère que le pool de dix offres contient déjà les éléments jugés pertinents par le proxy, et que l'enjeu principal devient leur rang.

Figure 4.5 – Évolution de l’objectif NDCG@10 au cours des essais Optuna

La figure 4.5 montre que les meilleurs essais se concentrent près de la frontière où le poids Neo4j domine. Cette concentration confirme que le problème n’est pas seulement un mauvais choix initial de pondérations : dans l’échantillon disponible, le signal graphe agrégé par `taux_match` apporte une amélioration marginale au NDCG@10 lorsqu’il est utilisé pour reranger un pool déjà construit par pgvector.

Figure 4.6 – Comparaison des pondérations retenues selon la fonction objectif

La figure 4.6 confirme que le score Neo4j seul atteint un NDCG@10 de 0.9697, contre 0.9489 pour le score sémantique seul. Le score hybride actuel atteint 0.9704, légèrement au-dessus de la calibration à deux composantes, parce qu’il mobilise aussi des signaux de niveau NCF et d’alignement sectoriel. La conclusion d’ingénierie est donc précise : le système doit conserver pgvector pour le rappel initial, mais le classement final doit donner une priorité forte au

score graphe lorsque les relations `POSSEDE` et `REQUIERT` sont disponibles. Si l'objectif applicatif devient la meilleure première recommandation, la pondération retenue par `MRR@10`–`Precision@10` fournit une alternative défendable avec environ deux tiers de signal sémantique et un tiers de signal graphe. La prochaine étape méthodologique consiste à calibrer simultanément toutes les composantes métier sur un jeu annoté humainement, au lieu de se limiter au couple `score_sem/taux_match`.

4.5 Contrôle de génération par critic et décision accept/revise

4.5.1 Calibration contrastive du seuil de fidélité

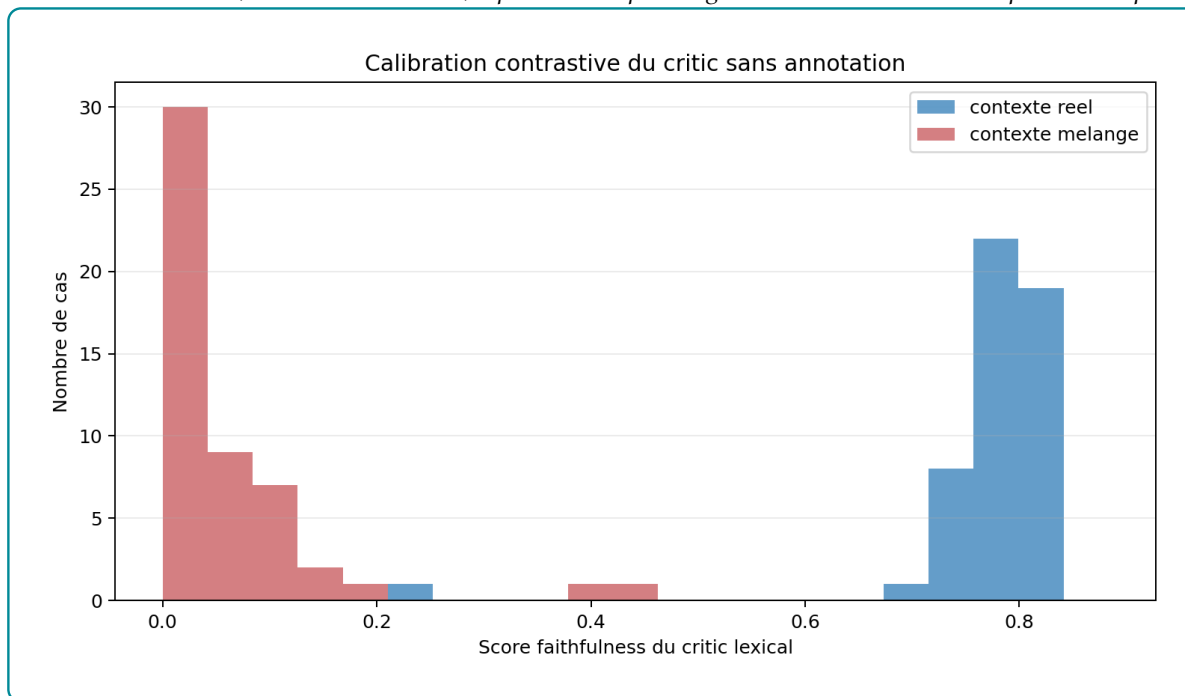
Le workflow Agentic GraphRAG ne s'arrête pas au classement des offres. Après le retrieval et l'enrichissement graphe, le LLM OpenRouter formule une réponse finale, puis un critic local vérifie si cette réponse reste suffisamment appuyée par le contexte récupéré. Ce critic calcule un score de fidélité lexicale : il compare les mots significatifs de la réponse aux mots présents dans le contexte. La décision `accept/revise` dépend ensuite d'un seuil appliqué à ce score.

Comme aucun jeu d'annotations humaines n'est disponible à ce stade, le seuil n'a pas été calibré par vérité terrain. Une calibration contrastive non supervisée a donc été réalisée. Le principe est simple : une réponse construite à partir d'un contexte réel doit obtenir un score plus élevé que la même réponse comparée à un contexte mélangé provenant d'une autre requête. Le protocole ne mesure donc pas la satisfaction d'un recruteur ; il teste une propriété minimale du critic : rejeter les réponses lorsqu'elles ne sont plus reliées aux preuves qui les ont produites.

L'expérience utilise 51 cas de base issus des sorties fonctionnelles du graphe : recherches métier–compétences, recherches compétence–offres et indicateurs de couverture du graphe. Chaque cas est évalué deux fois : une fois avec son contexte réel, une fois avec un contexte mélangé. Le jeu contrastif contient donc 102 évaluations. Les scores moyens séparent nettement les deux situations : le score de fidélité vaut 0,7725 en moyenne avec le contexte réel, contre 0,0532 avec le contexte mélangé. La médiane passe respectivement de 0,7857 à 0,0000.

Figure 4.7 – Distribution des scores du critic avec contexte réel et contexte mélangé

Source : Nos travaux, évaluation du critic, à partir des réponses générées et des contextes de preuve récupérés.



La figure 4.7 confirme que le critic distingue correctement les réponses appuyées par leur contexte des réponses placées face à des preuves non pertinentes. Cette séparation ne valide pas la qualité métier de chaque réponse, mais elle valide le rôle de garde-fou lexical : une réponse déconnectée de ses preuves obtient généralement un score très faible.

Table 4.4 – Seuil critique retenu après calibration contrastive

Source : Nos travaux, évaluation du critic, à partir des réponses générées et des contextes de preuve récupérés.

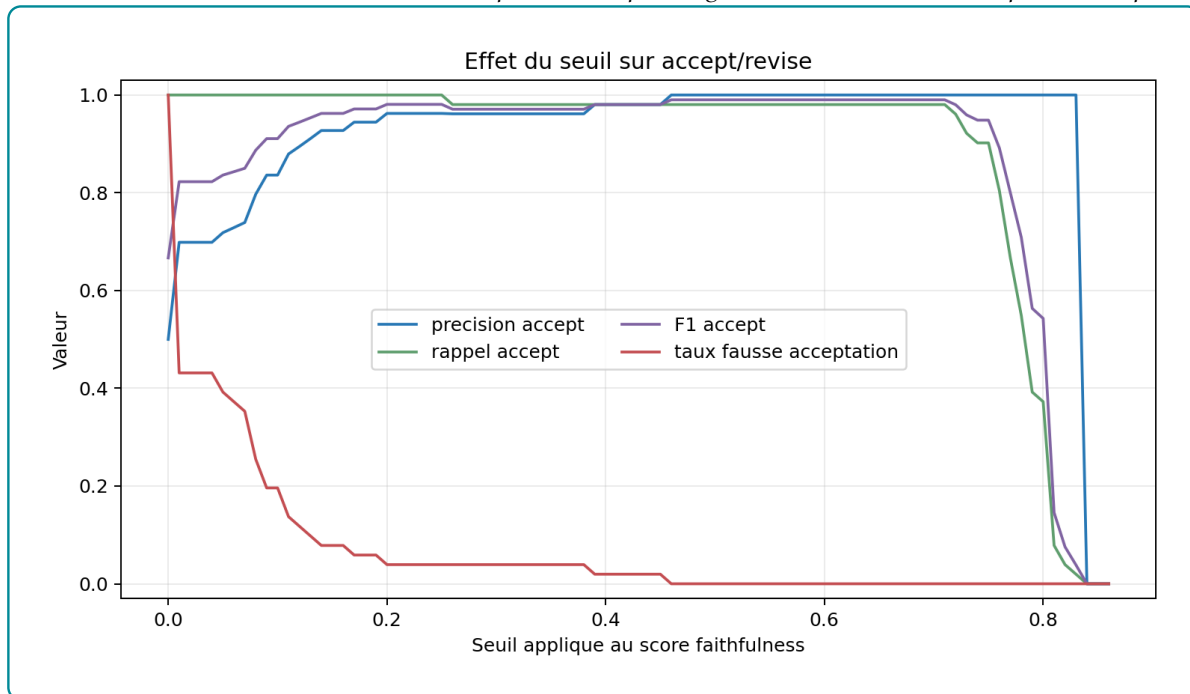
Seuil	F1	Fausse accept.	Fausse révisions	Lecture
0,46 retenu	0,9901	0	1	Seuil prudent : il bloque les contextes mélangés dans ce test contrastif, au prix d'une révision inutile.

4.5.2 Lecture opérationnelle des décisions accept/revise

Le résultat le plus important est le suivant : le seuil opérationnel retenu est 0,46. À ce niveau, aucune réponse associée à un contexte mélangé n'est acceptée dans le test contrastif, tandis que 50 réponses sur 51 restent acceptées avec leur vrai contexte. La règle retenue est volontairement sévère : minimiser d'abord les fausses acceptations, puis maximiser le F1 des décisions *accept/revise*.

Figure 4.8 – Effet du seuil du critic sur les décisions accept/revise

Source : Nos travaux, évaluation du critic, à partir des réponses générées et des contextes de preuve récupérés.



Cette calibration ne transforme pas le critic en juge parfait. Elle ne dit pas qu’une réponse acceptée est correcte au sens métier ; elle dit seulement qu’elle est suffisamment ancrée lexicalement dans les preuves récupérées. La décision correcte pour l’étude est donc de présenter 0,46 comme seuil opérationnel plus prudent pour les réponses générées, tout en conservant la nécessité d’une validation humaine future pour mesurer la qualité réelle des conseils, des citations et des trajectoires de formation.

Discussion et recommandations

Conclusion générale

Bibliographie

- [1] Francesco RICCI et al., éd. *Recommender Systems Handbook*. Springer, 2011. DOI : [10.1007/978-0-387-85820-3](https://doi.org/10.1007/978-0-387-85820-3).
- [2] PGVECTOR CONTRIBUTORS. *pgvector : Open-source Vector Similarity Search for Postgres*. 2024. URL : <https://github.com/pgvector/pgvector> (visité le 04/05/2026).
- [3] Aidan HOGAN et al. "Knowledge Graphs". In : *ACM Computing Surveys* 54.4 (2021), p. 1-37. DOI : [10.1145/3447772](https://doi.org/10.1145/3447772).
- [4] Patrick LEWIS et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In : *Advances in Neural Information Processing Systems* 33 (2020), p. 9459-9474. URL : <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [5] Dale T. MORTENSEN et Christopher A. PISSARIDES. "Job Creation and Job Destruction in the Theory of Unemployment". In : *The Review of Economic Studies* 61.3 (1994), p. 397-415. DOI : [10.2307/2297896](https://doi.org/10.2307/2297896).
- [6] Christopher A. PISSARIDES. *Equilibrium Unemployment Theory*. 2^e éd. Cambridge, MA : MIT Press, 2000.
- [7] George A. AKERLOF. "The Market for "Lemons" : Quality Uncertainty and the Market Mechanism". In : *The Quarterly Journal of Economics* 84.3 (1970), p. 488-500. DOI : [10.2307/1879431](https://doi.org/10.2307/1879431).
- [8] Michael SPENCE. "Job Market Signaling". In : *The Quarterly Journal of Economics* 87.3 (1973), p. 355-374. DOI : [10.2307/1882010](https://doi.org/10.2307/1882010).
- [9] Gary S. BECKER. *Human Capital : A Theoretical and Empirical Analysis, with Special Reference to Education*. New York : Columbia University Press, 1964.
- [10] David H. AUTOR. "The "Task Approach" to Labor Markets : An Overview". In : *Journal for Labour Market Research* 46.3 (2013), p. 185-199. DOI : [10.1007/s12651-013-0128-z](https://doi.org/10.1007/s12651-013-0128-z).

- [11] Gediminas ADOMAVICIUS et Alexander TUZHILIN. "Toward the Next Generation of Recommender Systems : A Survey of the State-of-the-Art and Possible Extensions". In : *IEEE Transactions on Knowledge and Data Engineering* 17.6 (2005), p. 734-749. DOI : [10.1109/TKDE.2005.99](https://doi.org/10.1109/TKDE.2005.99).
- [12] Michael J. PAZZANI et Daniel BILLSUS. "Content-Based Recommendation Systems". In : *The Adaptive Web* (2007), p. 325-341. DOI : [10.1007/978-3-540-72079-9_10](https://doi.org/10.1007/978-3-540-72079-9_10).
- [13] Yehuda KOREN, Robert BELL et Chris VOLINSKY. "Matrix Factorization Techniques for Recommender Systems". In : *Computer* 42.8 (2009), p. 30-37. DOI : [10.1109/MC.2009.263](https://doi.org/10.1109/MC.2009.263).
- [14] Xiangnan HE et al. "Neural Collaborative Filtering". In : *Proceedings of the 26th International Conference on World Wide Web*. 2017, p. 173-182.
- [15] Robin BURKE. "Hybrid Recommender Systems : Survey and Experiments". In : *User Modeling and User-Adapted Interaction* 12 (2002), p. 331-370. DOI : [10.1023/A:1021240730564](https://doi.org/10.1023/A:1021240730564).
- [16] Chen YANG et al. "Modeling Two-way Selection Preference for Person-Job Fit". In : *Proceedings of the 16th ACM Conference on Recommender Systems*. 2022, p. 102-112.
- [17] A. T. WASI. "HRGraph : Leveraging LLMs for HR Data Knowledge Graphs with Information Propagation-based Job Recommendation". In : *Proceedings of the 1st Workshop on Knowledge Graphs and Large Language Models*. 2024, p. 56-62.
- [18] B. YANG et Z. SHEN. "Knowledge Graph Construction and Talent Competency Prediction for Human Resource Management". In : *Alexandria Engineering Journal* 121 (2025), p. 223-235.
- [19] Ashish VASWANI et al. "Attention Is All You Need". In : *Advances in Neural Information Processing Systems*. 2017, p. 5998-6008.
- [20] Tomas MIKOLOV et al. "Distributed Representations of Words and Phrases and their Compositionality". In : *Advances in Neural Information Processing Systems*. T. 26. 2013. URL : <https://proceedings.neurips.cc/paper/5021-distributed-representations-of-words-and-phrases-and-their-compositionality>.
- [21] Dzmitry BAHDANAU, Kyunghyun CHO et Yoshua BENGIO. "Neural Machine Translation by Jointly Learning to Align and Translate". In : *International Conference on Learning Representations*. 2015. URL : <https://arxiv.org/abs/1409.0473>.

- [22] Jacob DEVLIN et al. "BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding". In : *Proceedings of NAACL-HLT*. 2019, p. 4171-4186. URL : <https://aclanthology.org/N19-1423/>.
- [23] Nils REIMERS et Iryna GUREVYCH. "Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks". In : *Proceedings of EMNLP-IJCNLP*. 2019, p. 3982-3992. URL : <https://aclanthology.org/D19-1410/>.
- [24] Tom B. BROWN et al. "Language Models are Few-Shot Learners". In : *Advances in Neural Information Processing Systems* 33 (2020), p. 1877-1901. URL : <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>.
- [25] Colin RAFFEL et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". In : *Journal of Machine Learning Research* 21.140 (2020), p. 1-67. URL : <https://www.jmlr.org/papers/v21/20-074.html>.
- [26] Long OUYANG et al. "Training Language Models to Follow Instructions with Human Feedback". In : *Advances in Neural Information Processing Systems* 35 (2022), p. 27730-27744. URL : https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html.
- [27] Nandan THAKUR et al. "BEIR : A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models". In : *Advances in Neural Information Processing Systems*. T. 34. 2021, p. 28974-28989. URL : <https://arxiv.org/abs/2104.08663>.
- [28] Niklas MUENNIGHOFF et al. "MTEB : Massive Text Embedding Benchmark". In : *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*. 2023, p. 2014-2037. URL : <https://aclanthology.org/2023.eacl-main.148/>.
- [29] Yu A. MALKOV et D. A. YASHUNIN. "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs". In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020), p. 824-836. DOI : [10.1109/TPAMI.2018.2889473](https://doi.org/10.1109/TPAMI.2018.2889473).
- [30] Jeff JOHNSON, Matthijs DOUZE et Hervé JÉGOU. "Billion-scale Similarity Search with GPUs". In : *IEEE Transactions on Big Data* 7.3 (2019), p. 535-547. DOI : [10.1109/TBDATA.2019.2921572](https://doi.org/10.1109/TBDATA.2019.2921572).
- [31] Thomas N. KIPF et Max WELLING. "Semi-Supervised Classification with Graph Convolutional Networks". In : *International Conference on Learning Representations*. 2017.

- [32] Petar VELIČKOVIĆ et al. "Graph Attention Networks". In : *International Conference on Learning Representations*. 2018.
- [33] Xiangnan HE et al. "LightGCN : Simplifying and Powering Graph Convolution Network for Recommendation". In : *Proceedings of SIGIR*. 2020, p. 639-648.
- [34] Yunfan GAO et al. *Retrieval-Augmented Generation for Large Language Models : A Survey*. 2023. arXiv : 2312.10997 [cs.CL]. URL : <https://arxiv.org/abs/2312.10997>.
- [35] Shunyu YAO et al. "ReAct : Synergizing Reasoning and Acting in Language Models". In : *International Conference on Learning Representations*. 2023. URL : https://openreview.net/forum?id=WE_vluYUL-X.
- [36] Akari ASAI et al. "Self-RAG : Learning to Retrieve, Generate, and Critique through Self-Reflection". In : *International Conference on Learning Representations*. 2024. URL : <https://openreview.net/forum?id=hSyW5go0v8>.
- [37] Shahul Es et al. "RAGAS : Automated Evaluation of Retrieval Augmented Generation". In : *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics : System Demonstrations*. 2024, p. 150-158. URL : <https://aclanthology.org/2024.eacl-demo.16/>.
- [38] Yang LIU et al. "G-Eval : NLG Evaluation using GPT-4 with Better Human Alignment". In : *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023, p. 2511-2522. URL : <https://aclanthology.org/2023.emnlp-main.153/>.
- [39] LANGCHAIN. *LangGraph Documentation*. Documentation officielle du framework LangGraph. 2026. URL : <https://langchain-ai.github.io/langgraph/> (visité le 06/06/2026).
- [40] William L. HAMILTON. *Graph Representation Learning*. Morgan & Claypool Publishers, 2020. DOI : 10.2200/S01045ED1V01Y202009AIM046.
- [41] Takuya AKIBA et al. "Optuna : A Next-generation Hyperparameter Optimization Framework". In : *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2019, p. 2623-2631. DOI : 10.1145/3292500.3330701.

Annexes

Table A.1 – Labels de nœuds et relations structurantes du graphe

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Bloc	Labels de nœuds	Relations principales et interprétation
Données d'emploi	OffreEmploi, Candidat, Employeur, Secteur, Localisation	PUBLIEE_PAR relie une offre à son employeur; DANS_SECTEUR la rattache à un secteur; LOCALISEE_A indique la ville ou zone d'exercice; A_NIVEAU relie le candidat à son niveau de formation.
Compétences et métiers	Compétence, Métier, GroupeCompétences, GroupeISCO	NECESSITE relie un métier aux compétences attendues; PARTIE_DE rattache une compétence à son groupe; CLASSIFIE_DANS relie un métier à un groupe ISCO; PLUS_LARGE_QUE représente les relations hiérarchiques entre compétences.
Référentiel MEPC	GrandGroupeMEPC, SousGroupeMEPC, GroupeBaseMEPC	CONTIENT représente la hiérarchie interne de la nomenclature; ALIGNE_AVEC rapproche les groupes MEPC des groupes ISCO; CORRESPOND_MEPC rattache un métier ESCO à un groupe de base MEPC.
Référentiel NCF	NiveauFormationNCF, GrandDomaineNCF, DomaineSpécialiséNCF, DomaineDétailléNCF	CONTIENT représente la hiérarchie formation; REQUIERT_NIVEAU_NCF relie une offre au niveau demandé; A_FORMATION rattache un candidat à son domaine de formation; PREPARE_POUR relie un domaine de formation aux métiers visés.
Recommandation	OffreEmploi, Candidat, Compétence, Métier, DomaineDétailléNCF	Les chemins candidat-compétence-offre permettent d'identifier les compétences acquises et manquantes; les chemins offre-niveau-formation servent à contrôler la compatibilité de qualification; les chemins formation-métier soutiennent la proposition de montée en compétences.

Table A.2 – Volume d’entités indexées dans la base vectorielle pgvector

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Type d’entité	Nombre	Source principale
OFFRE_EMPLOI	15 722	Scraping KmerAI
COMPETENCE	13 939	Référentiel esco v1.1
METIER	3 039	Référentiel esco v1.1
CANDIDAT	1 105	Base locale de profils demandeurs
GROUPE_BASE_MEPC	209	INS Cameroun - MEPC 2013
DOMAINE_DETAILLE_NCF	201	INS Cameroun - NCF 2017
Total	34 215	

Table A.3 – Pipeline général implémenté : étapes, entrées et sorties

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Ordre	Étape	Entrées principales	Sorties concrètes
1	ETL	Données brutes d’offres, candidats et référentiels	Fichiers normalisés, champs <code>text_to_embed</code> , paires <code>pairs_train/val/test.jsonl</code> .
2	Fine-tuning	Paires offre–description et modèle <code>all-MiniLM-L6-v2</code>	Modèle sauvegardé dans <code>models/st_finetuned/final</code> et métriques d’entraînement.
3	Indexation vectorielle	Entités nettoyées et encodeur fine-tuné	Table <code>embeddings</code> , table <code>doc_chunks</code> , index HNSW et index textuels PostgreSQL.
4	Graphe de connaissances	Offres, candidats, compétences, métiers, niveaux et référentiels	Noeuds Neo4j, relations métier-compétence-offre-candidat, contraintes et chemins explicatifs.
5	GraphRAG hybride	Résultats pgvector, chemins Neo4j et règles métier	Contexte candidat–offre, score hybride, analyse de <i>skill gap</i> , critic et Text2Cypher.
6	Orchestration LangGraph	Question utilisateur et registre d’outils	État partagé, routage, exécution des outils, génération finale et traces du workflow.
7	Exposition applicative	Workflow agentique et services locaux	API FastAPI, CLI de test et interface Streamlit.

A.1 Compléments techniques du chapitre 3

Les éléments suivants documentent les diagnostics détaillés de l'implémentation. Ils sont conservés en annexe afin que le chapitre 3 reste centré sur les décisions d'architecture et non sur l'inventaire complet des sorties graphiques.

Figure A.1 – Localisation nettoyée des offres

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

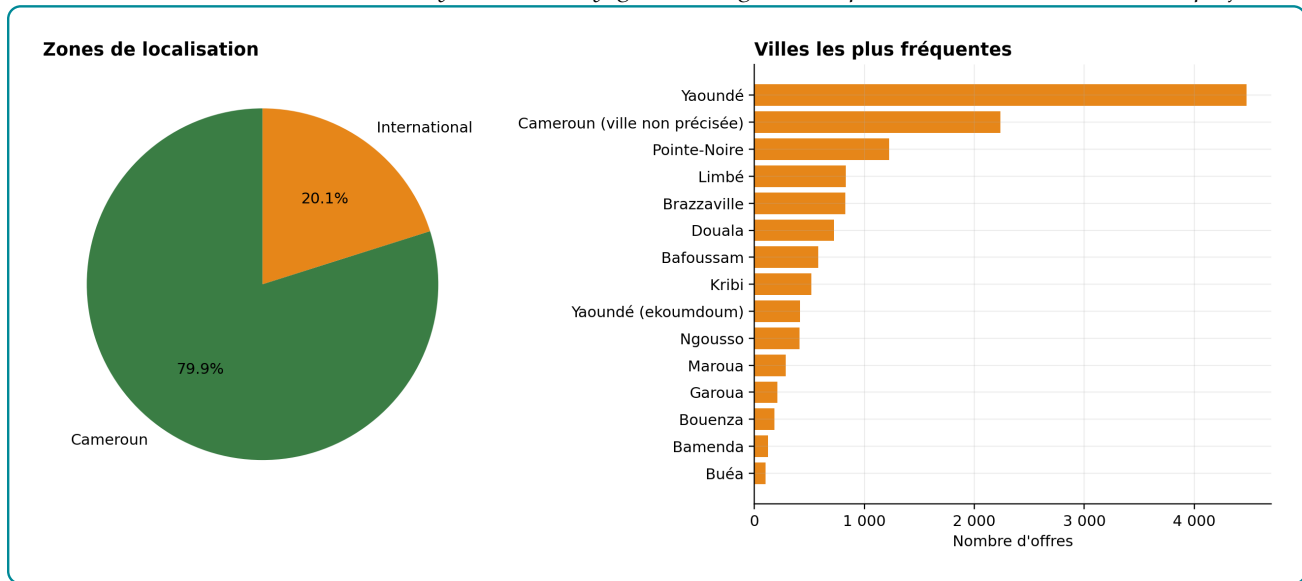


Table A.4 – Synthèse des traitements de nettoyage utiles au moteur de recommandation

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Traitement	Décision implémentée	Effet attendu
Casse et formats	Harmonisation des libellés pour réduire les variantes d'écriture.	Moins de fragmentation dans les compétences, secteurs, villes et sources.
Doublons	Validation uniquement si toute la ligne est identique sur toutes les variables.	Suppression prudente des répétitions exactes, sans effacer des offres proches mais distinctes.
Localisation	Séparation Cameroun, international et non précisée; conservation des localités fines disponibles.	Filtrage géographique possible sans perte des offres incomplètes.
Champs vides	Modalités explicites lorsque le champ reste exploitable comme absence d'information.	Le système peut pénaliser ou contextualiser le manque d'information au lieu d'ignorer l'offre.
Descriptions faibles	Enrichissement de la représentation par métadonnées et compétences.	Robustesse accrue pour les sources où le détail de l'offre est absent ou court.

Table A.5 – Comparaison directe entre le baseline et le modèle fine-tuné

Source : Nos travaux, traitements Python et SentenceTransformer, à partir des paires emploi-compétences et des entités nettoyées du projet.

Métrique test	Baseline	Fine-tuné	Gain
NDCG@10	0,1681	0,6232	+0,4552
MRR@10	0,1410	0,5874	+0,4463
Recall@1	0,0986	0,5268	+0,4281
Recall@5	0,1983	0,6632	+0,4648
Recall@10	0,2560	0,7398	+0,4837
Precision@1	0,0986	0,5268	+0,4281

Table A.6 – Diagnostic statistique du chunking documentaire

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Indicateur	Valeur observée
Documents sources indexés	3
Chunks documentaires indexés	142
Stratégie réellement utilisée	100 % structurelle
Taille moyenne	187,1 tokens
Taille médiane	116,0 tokens
Écart-type	169,6 tokens
Minimum / maximum	6 / 492 tokens
Coefficient de variation	0,91
Chunks avec embedding	142 / 142
Chunks synchronisés avec Neo4j	142 / 142

Table A.7 – Couverture documentaire et duplication des chunks

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Source	Couverture approximative	Chunks
NCF 2017	25,9 %	38
MEPC 2013	23,6 %	61
Référentiel des diplômes	79,3 %	43
Taux de duplication cosinus > 0,95	0,0 %	142
Similarité maximale hors diagonale	0,921	–

Table A.8 – Cohérence sémantique intra-chunk

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Source	Moyenne	Médiane	Intervalle observé
MEPC 2013	0,307	0,288	0,191–0,502
NCF 2017	0,336	0,321	0,259–0,474
Référentiel des diplômes	0,268	0,260	0,066–0,457

Table A.9 – Indicateurs techniques pgvector

Source : Nos travaux, diagnostics PostgreSQL–pgvector, à partir des embeddings, des métadonnées et des fragments documentaires indexés.

Indicateur	Valeur observée
Embeddings d'entités structurées	72 643
Chunks documentaires indexés	142
Dimension des vecteurs	384
Modèle d'encodage	all-MiniLM-L6-v2-ft-offres-cm
Index PostgreSQL sur embeddings et doc_chunks	19
Taille totale de la table embeddings	259,05 Mo
Taille de l'index emb_hnsw	81,30 Mo
Taille totale de doc_chunks	1,51 Mo
Latence moyenne top-10 HNSW sur 25 requêtes	34,31 ms
Latence médiane top-10 HNSW sur 25 requêtes	30,89 ms
Latence p90 top-10 HNSW sur 25 requêtes	64,41 ms

Table A.10 – Index pgvector et index complémentaires

Source : Nos travaux, diagnostics PostgreSQL–pgvector, à partir des embeddings, des métadonnées et des fragments documentaires indexés.

Famille d'index	Index observés	Usage dans le retrieval
Vectoriel HNSW	emb_hnsw, doc_chunks_hnsw vector_cosine_ops.	avec Recherche approximative des plus proches voisins en similarité cosinus.
B-tree métier	emb_offre_idx, emb_candidat_idx, emb_competence_idx, emb_metier_idx.	Filtrage par type d'entité et accès rapide aux objets métiers.
Synchronisation graphe	emb_neo4j_idx.	Passage d'un résultat vectoriel vers le noeud Neo4j correspondant.
Modèle et unicité	emb_model_idx, contrainte unique entity_kind, entity_id, model_id.	Éviter les doublons d'embeddings et gérer les futures versions de modèles.
Recherche lexicale	emb_fulltext_fr_idx, doc_chunks_fulltext_fr_idx	Complément lexical français pour requêtes par mots exacts, acronymes ou intitulés rares.
Référentiels	doc_chunks_ncf_idx, doc_chunks_mepc_idx, doc_chunks_source_idx.	Recherche ciblée dans NCF, MEPC et référentiel des diplômes.

Figure A.2 – Tableau de bord statistique du graphe Neo4j

Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.

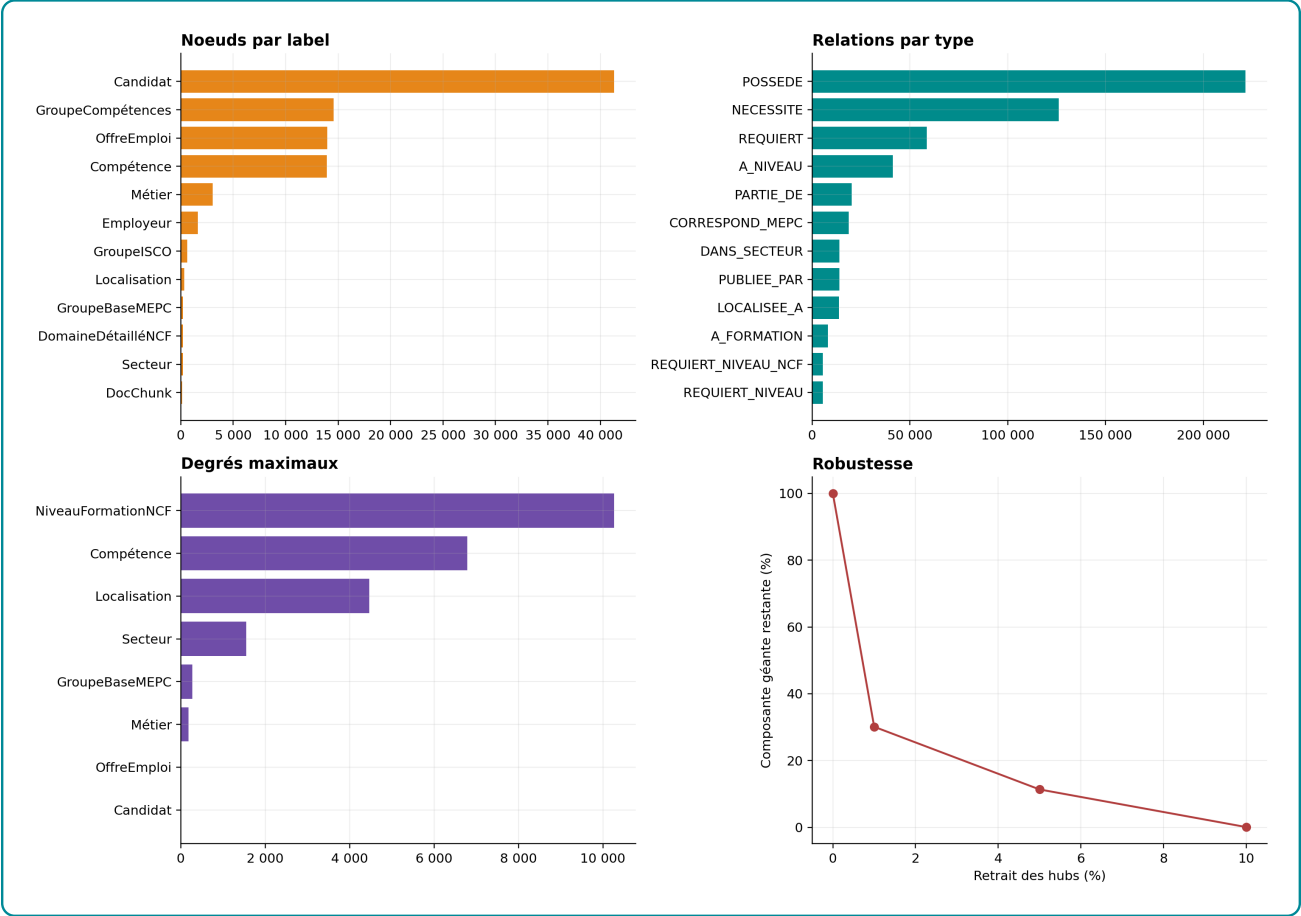


Figure A.3 – Composition du graphe par types de noeuds et relations

Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.

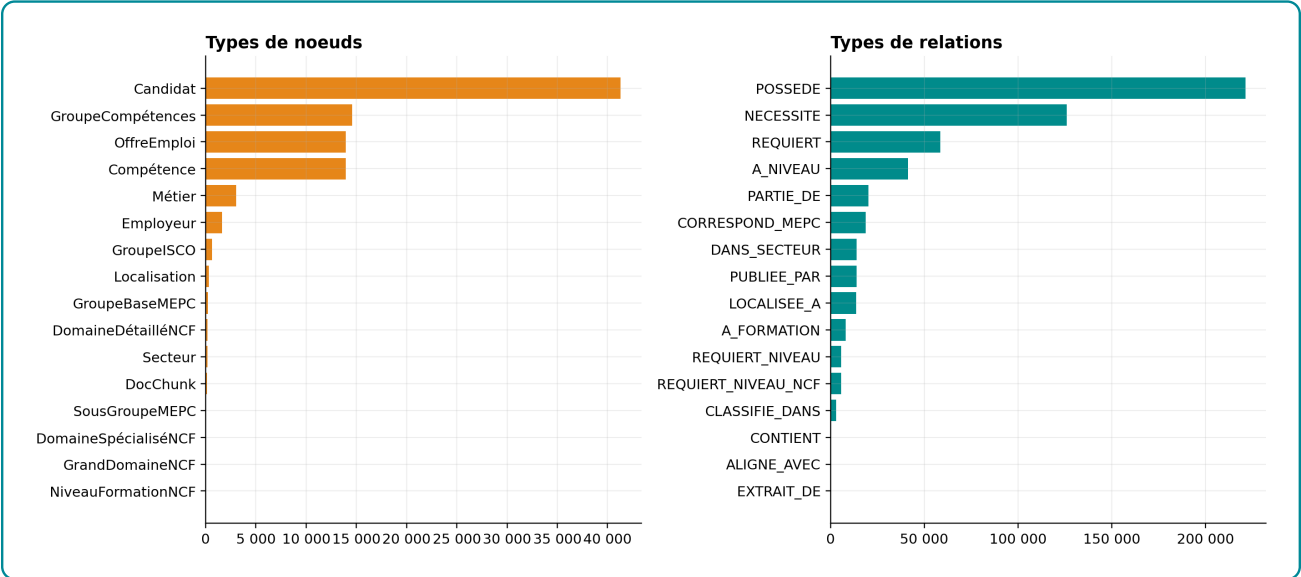
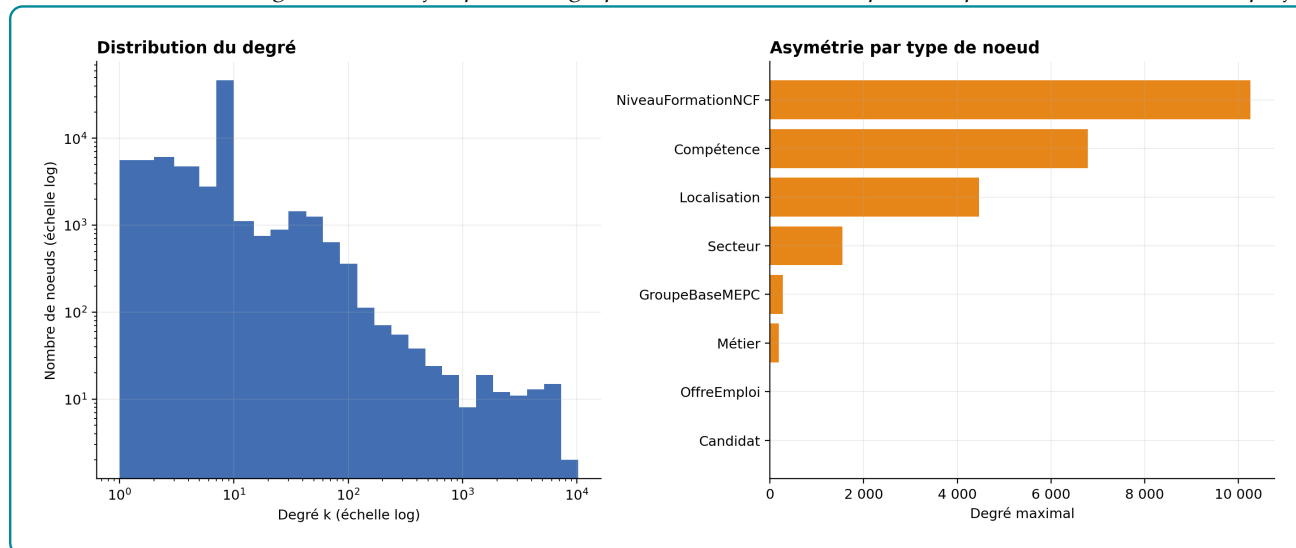


Figure A.4 – Distribution des degrés dans la projection matching

Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.

**Table A.11 – Hétérogénéité des degrés par famille de noeuds**

Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.

Label	Moyenne	Médiane	p90	Max
NiveauFormationNCF	5 812,56	7 073	10 672,4	11 050
GroupeBaseMEPC	90,22	73	201	276
Secteur	72,69	5,5	151,6	1 548
Métier	48,61	44	77	186
Localisation	43,96	1	8	4 467
Compétence	30,57	5	28	6 789
OffreEmploi	7,97	9	11	11
Candidat	6,56	7	8	8

Figure A.5 – Centralités des compétences : degré, PageRank et intermédiarité approximée

Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.

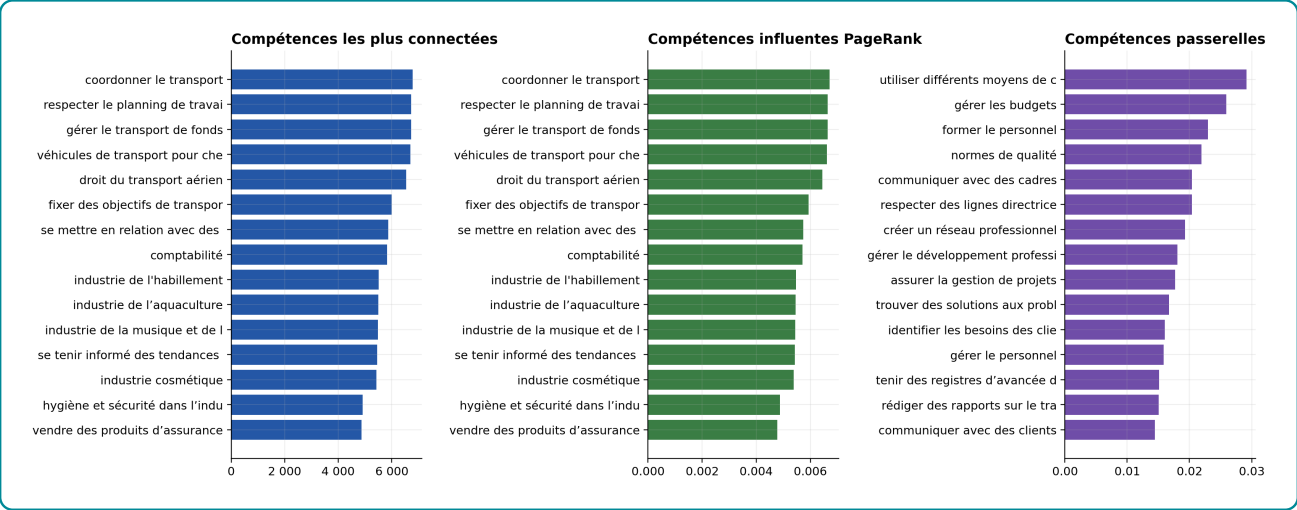


Table A.12 – Exemples de compétences centrales selon trois métriques

Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.

Degré	Valeur	PageRank	Valeur	Intermédiarité	Valeur
Coordonner le trans- port	6 787	Coordonner le trans- port	0,00672	Utiliser différents moyens de communica- tion	0,02919
Respecter le planning de travail dans les transports	6 726	Respecter le planning de travail dans les transports	0,00664	Gérer les budgets	0,02592
Gérer le transport de fonds	6 718	Gérer le transport de fonds	0,00664	Former le personnel	0,02297
Comptabilité	5 822	Comptabilité	0,00571	Normes de qualité	0,02193

Figure A.6 – Communautés Louvain, distances et composantes connexes

Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.

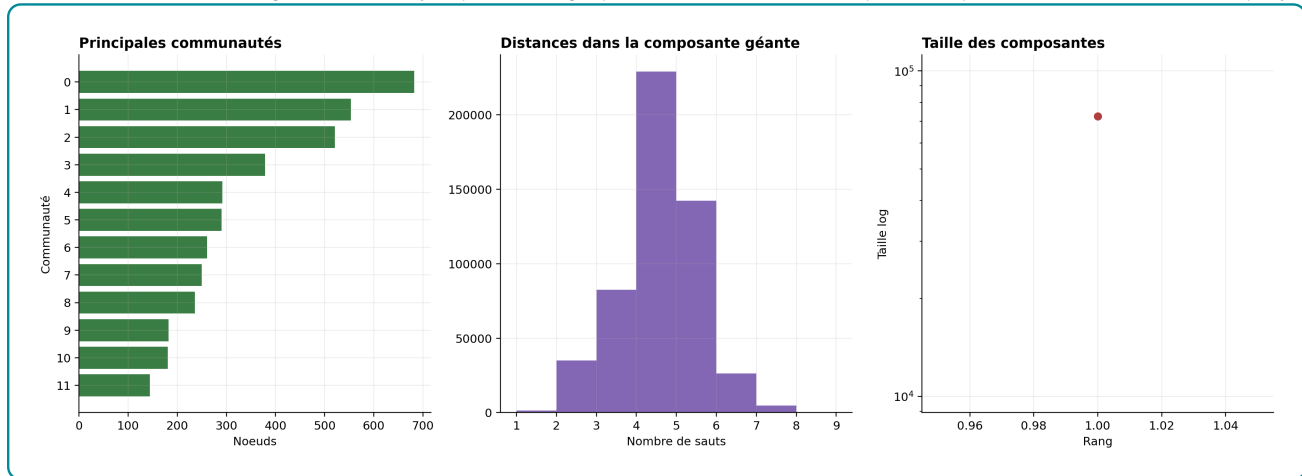


Figure A.7 – Robustesse de la projection matching après retrait des hubs

Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.

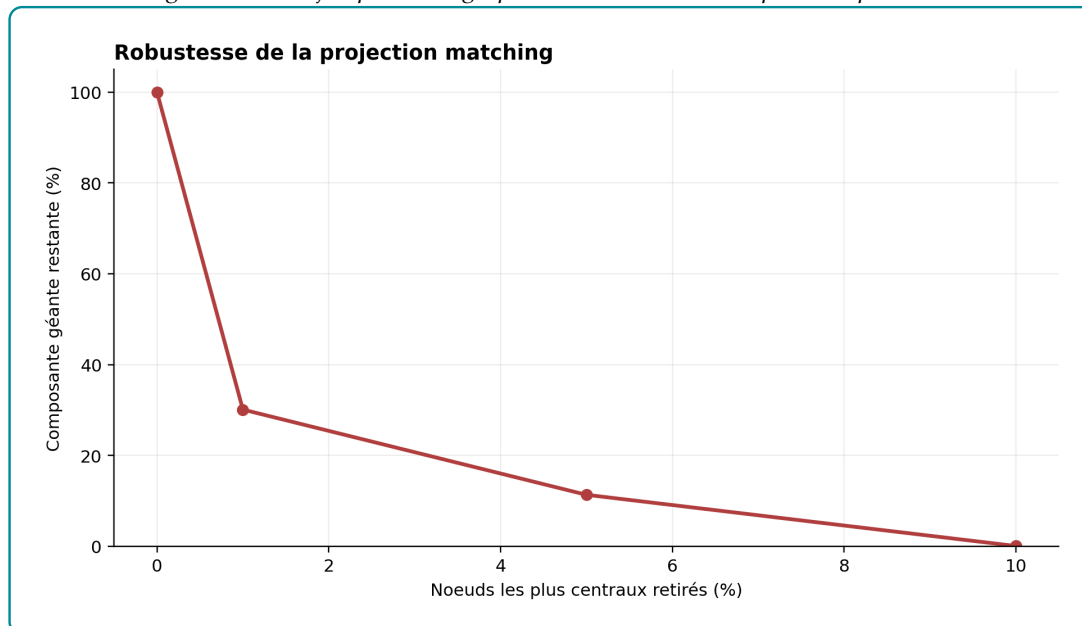


Figure A.8 – Compétences par offre, candidat et métier dans Neo4j

Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.



Figure A.9 – Distribution échantillonnée du skill gap candidat-offre

Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.

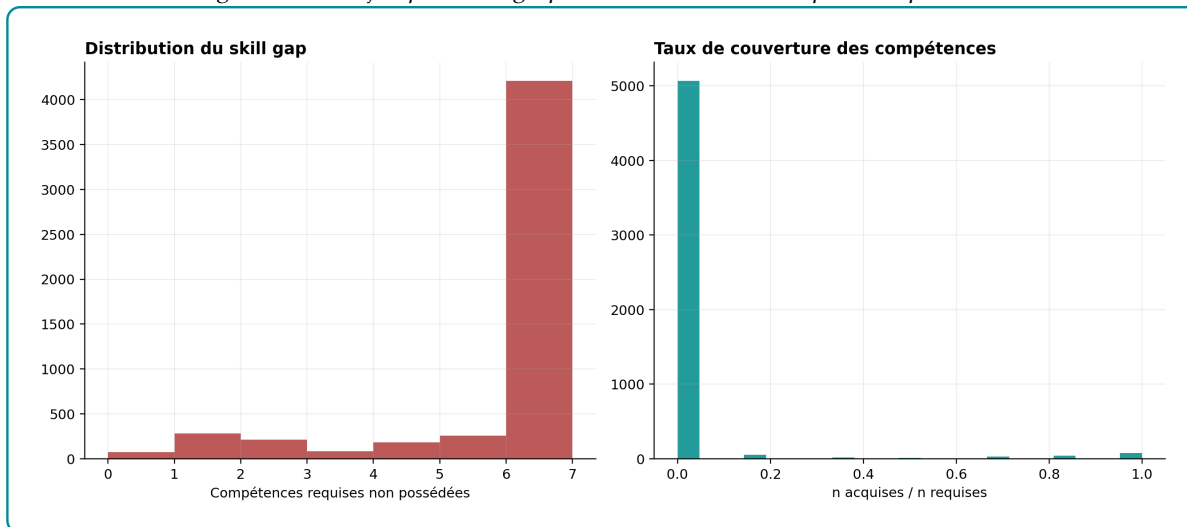


Table A.13 – Résumé du déplacement de rang après reranking Neo4j*Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.*

Indicateur	Valeur
Couples candidat-offre évalués	690
Candidats distincts	69
Rang moyen avant reranking pgvector	9,51
Rang moyen après reranking Neo4j/Optuna	5,50
Déplacement moyen de rang	+4,01
Déplacement médian	+1,00
Offres remontées	59,86 %
Offres rétrogradées	13,77 %
Offres inchangées	26,38 %

Table des matières

Dédicace	i
Remerciements	ii
Liste des sigles et abréviations	v
Liste des tableaux	vi
Liste des figures	viii
Avant-propos	ix
Résumé	x
Abstract	xi
Introduction générale	1
I Cadre théorique et conceptuel	6
1 Fondements théoriques et revue de la littérature sur l'IA générative et les systèmes de recommandation	7
1.1 Vocabulaire opérationnel : du matching à l'Agentic RAG	7
1.2 Fondements économiques et problématique d'appariement	9
1.2.1 Marché du travail, recherche d'emploi et frictions	9
1.2.2 Capital humain, tâches et inadéquation des compétences	10
1.3 Systèmes de recommandation et person-job fit	10
1.3.1 Approches par contenu, collaboratives et hybrides	10
1.3.2 Spécificité du person-job fit	11
1.4 LLMs, embeddings et graphes de connaissances	12
1.4.1 Grands modèles de langage et génération contrôlée	12
1.4.2 Embeddings, recherche sémantique et bases vectorielles	13
1.4.3 Graphes de connaissances et recommandation relationnelle	14
1.5 RAG, GraphRAG et orchestration agentique	14
1.5.1 De la RAG au GraphRAG	14
1.5.2 Agentic RAG, critic et traçabilité	15
1.5.3 Évaluer la performance et la factualité	16
2 Source de données et Approche méthodologique	19

2.1	Cadre méthodologique et sources de données	19
2.1.1	Démarche d'ingénierie appliquée retenue	19
2.1.2	Sources de données et leur rôle dans le système	22
2.1.3	Préparation et normalisation des données	24
2.1.4	Alignement des référentiels métiers et formations	24
2.2	Modélisation sémantique et structuration des connaissances	25
2.2.1	Choix du modèle d'embeddings et fine-tuning	25
2.2.2	Indexation vectorielle dans PostgreSQL et pgvector	26
2.2.3	Construction du graphe de connaissances sous Neo4j	26
2.2.4	Score hybride et logique de skill gap	27
2.3	Orchestration agentique et génération contrôlée	28
2.3.1	Architecture Agentic RAG et orchestration LangGraph	28
2.3.2	Text2Cypher et rôle du LLM	29
2.3.3	Modes d'accès et traçabilité des décisions	30
2.4	Évaluation prévue, limites et synthèse méthodologique	30
2.4.1	Protocole d'évaluation prévu	30
2.4.2	Limites méthodologiques et précautions d'interprétation	31
2.4.3	Ce que l'architecture garantit et ses exigences	31
II	Présentation des résultats	32
3	Construction opérationnelle du système hybride emploi-compétences	33
3.1	Données exploitables et contraintes de qualité	33
3.1.1	Volumes disponibles après nettoyage	33
3.1.2	Qualité des variables utiles au matching emploi-compétences	35
3.1.2.1	Localisation exploitable pour filtrer sans exclure	36
3.1.2.2	Compétences et descriptions : signal utile mais hétérogène	36
3.1.3	Nettoyage réalisé et limites conservées	37
3.2	Adaptation du modèle d'embeddings au domaine emploi-compétences	38
3.2.1	Encodeur retenu et paires contrastives	38
3.2.2	Fine-tuning : contraintes de calcul et preuve de gain	39
3.2.2.1	Paramétrage contraint et dynamique d'apprentissage	40
3.2.2.2	Preuve de gain sur le ranking sémantique	42
3.2.3	Structure de l'espace vectoriel après fine-tuning	43
3.3	Mémoire vectorielle pgvector pour le retrieval	44
3.3.1	Stratégie de chunking des référentiels	46
3.4	Neo4j : mémoire relationnelle du matching	50
3.4.1	Du voisinage vectoriel au chemin explicable	51
3.4.2	Structure du graphe et risques de hubs	53
3.4.2.1	Taille réelle du graphe et couverture de la projection matching	53
3.4.2.2	Noeuds très connectés : signal utile ou bruit de popularité	54
3.4.2.3	Compétences pivots selon degré, PageRank et intermédiarité	55
3.4.2.4	Communautés Louvain et robustesse aux hubs	55
3.4.2.5	Skill gap observé entre profils et offres	56
3.5	Score hybride et reranking des recommandations	57
3.5.1	Formalisation du score hybride	57

3.5.2	Effet du reranking hybride	59
3.6	Orchestration agentique, critic et exposition du système	60
4	Évaluation de la performance du système	61
4.1	Cadre d'évaluation et protocole expérimental	61
4.1.1	Questions évaluées et logique de séparation	61
4.1.2	Données, métriques et limites du protocole	62
4.2	Retrieval sémantique et apport mesuré du graphe	62
4.2.1	Performance du modèle d'embeddings fine-tuné	62
4.2.2	Ablation pgvector seul contre pgvector enrichi par Neo4j	64
4.3	Lecture fonctionnelle du graphe Neo4j et du GraphRAG	65
4.3.1	Recherche métier vers compétences	66
4.3.2	Recherche compétence vers offres	67
4.3.3	Skill gap	67
4.3.4	Compatibilité NCF	67
4.3.5	Comparaison RAG vs GraphRAG	68
4.4	Reranking hybride et calibration des poids	68
4.4.1	Déplacements de rang après reranking par le graphe	68
4.4.2	Pondérations optimales du score hybride avec Optuna	69
4.5	Contrôle de génération par critic et décision accept/revise	72
4.5.1	Calibration contrastive du seuil de fidélité	72
4.5.2	Lecture opérationnelle des décisions accept/revise	73
5	Discussion et recommandations	75
	Conclusion générale	I
	Bibliographie	I
	Bibliographie	III
A	Annexes	VII
A.1	Compléments techniques du chapitre 3	X